

AlgoAnalysis

kiwiizzz

2026 年 06 月 16 日

AlgoAnalysis

kiwiizzz

2026 年 06 月 16 日

1 chapter1: 导论

Stirling 公式:

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right)$$

啊啊啊寶寶妳是一條區 是啊, 吃什么 是啊, 吃什麼 再 san

1.1 基本概念

算法是在有限的步骤内求解某一问题, 所使用的一组定义明确的规则, 也就是在通过一组有限定义的指令来完成一些任务, 给定一个初始状态, 并相应的获取最终结果,

算法应该具有以下特性:

- 可行性;
- 确定性 (每一步都被精确定义, 不存在模糊操作);
- 有限性 (程序可以没有这个特性, 操作系统就是一个不关机就无限期执行的程序);
- 输入、输出,

算法是解决问题的抽象步骤, 程序是算法的具体实现, 算法通过编程语言转为程序, 程序在计算机上运行,

算法分析有事前分析与事后测试两种基本方法,

对于任何一个计算步骤, 如果它的一次执行代价总是以一个时间常数为上界, 与输入数据或者使用何种算法无关, 则称该计算步骤为“元运算”或“元操作”,

算法中某个语句或运算的执行次数称为其频度,

如果算法中的一个元运算具有最高频度, 所有其他元运算频度均在它的频度的常数倍内, 则称这个元运算为基本运算,

1.2 复杂度

1.2.1 O 记号: 渐进上界记号

设 $f(n)$ 和 $g(n) : \mathbb{N}^+ \rightarrow \mathbb{R}^+$, 如果存在正常数 c 和自然数 n_0 , 使得

$$\forall n \geq n_0, f(n) \leq c \cdot g(n)$$

则称 $f(n)$ 是 $O(g(n))$ 的, 记为 $f(n) = O(g(n))$ ¹,

也可以这样理解: $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq \infty \implies f(n) = O(g(n))$

1.2.2 Ω 记号: 渐进下界记号

设 $f(n)$ 和 $g(n) : \mathbb{N}^+ \rightarrow \mathbb{R}^+$, 如果存在正常数 c 和自然数 n_0 , 使得

$$\forall n \geq n_0, f(n) \geq c \cdot g(n)$$

则称 $f(n)$ 是 $\Omega(g(n))$ 的, 记为 $f(n) = \Omega(g(n))$,

也可以这样理解: $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq 0 \implies f(n) = \Omega(g(n))$

1.2.3 Θ -记号: 紧渐进界记号

如果存在 2 个正常数 c_1, c_2 和自然数 n_0 , 使得

$$\forall n \geq n_0, c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

则称 $f(n)$ 是 $\Theta(g(n))$, 记为 $f(n) = \Theta(g(n))$,

也就是说, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c > 0 \implies f(n) = \Theta(g(n))$

可以推断, $f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$, 此时称 $f(n)$ 与 $g(n)$ 同阶,

1.2.4 o -记号

对于任意给定的 $c > 0$, 都存在正整数 n_0 , 使得当 $n \geq n_0$ 时有

$$f(n) < c \cdot g(n)$$

则称函数 $f(n)$ 是 $o(g(n))$ 的, 记为 $f(n) = o(g(n))$,

也就是说: $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \implies f(n) = o(g(n))$.

因此可以有:

$$f(n) = o(g(n)) \Leftrightarrow f(n) = O(g(n)) \wedge g(n) \neq O(f(n))$$

当 n 充分大时, $f(n)$ 的阶比 $g(n)$ 低,

1.2.5 ω -记号: 非紧下界记号

对于任意给定的 $c > 0$, 都存在正整数 n_0 , 使得当 $n \geq n_0$ 时有 $f(n) > c \cdot g(n)$, 则称函数 $f(n)$ 是 $\omega(g(n))$ 的, 记为 $f(n) = \omega(g(n))$,

¹这种"="的记号的本意其实是 $f(n) \in O(g(n))$, 而对于出现在等式或不等式中的渐进记号表示的也是, 其中的某一函数

等价于: $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \implies f(n) = \omega(g(n))$.

1.2.6 渐进记号的性质

1. 传递性: 所有符号
2. 自反性: Θ, O, Ω
3. 对称性: Θ
4. 互对称性:

$$f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$$

$$f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$$

5. 算数计算

- $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$
- $O(f(n)) + O(g(n)) = O(f(n) + g(n))$
- $O(f(n)) * O(g(n)) = O(f(n) * g(n))$
- $O(c \cdot f(n)) = O(f(n))$
- $g(n) = O(f(n)) \Rightarrow O(f(n)) + O(g(n)) = O(f(n))$

算法的空间复杂度是对一个算法在运行过程中**临时使用存储空间的渐进度量**,

1.3 平摊分析

1.3.1 聚集分析

将所有的操作加起来, 然后除以操作数, 得到的结果就是平摊代价,

1.3.2 记账法

对不同操作赋予不同的费用, 这个均摊成本 $\hat{c}_{op}$ 可能与实际成本 c_{op} 不同, 但必须要满足的是, 对于任意的合法的 n 个操作序列, 必须有 $T(n) = \sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i$.

这种方法更常见于对一个已知的均摊代价进行巧妙的证明, 例如: 对于二进制计数器算法, 因为每一个操作 1 翻转为 0 中的 1, 都一定来自于一个操作: 0 翻转为 1, 而最终结果一定会剩余一定量的 1, 所以操作: 0 翻 1 的数量大于 1 翻 0 的数量,

OP	Real Cost C_{OP}	Amortized Cost $\hat{C}_{OP}$
flip (0 $\rightarrow$ 1)	1	2
flip (1 $\rightarrow$ 0)	1	0

表 1 Binary Counter Flip Operations: Real Cost vs. Amortized Cost

$$\begin{aligned}
 T(n) &= \sum_{i=1}^n C_i \\
 &= \#flip(0 \rightarrow 1) + \#flip(1 \rightarrow 0) \\
 &\leq 2 \cdot \#flip(0 \rightarrow 1) \\
 &\leq 2n
 \end{aligned}$$

1.3.3 势能法

势能函数充当能量存储罐的作用，当操作简单时就会势能增加（相当于存钱）；当操作复杂时就会势能减少（相当于取钱）；那么每次操作的平摊代价=实际操作成本+存储罐的变化量，即：

$$a_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

那么对于 n 个操作的总平摊代价为：

$$\sum a_i = \sum c_i + \Phi(D_n) - \Phi(D_0)$$

如果势函数使得对所有的 n ，都有 $\Phi(D_n) \geq \Phi(D_0)$ ，则总平摊代价就是总实际代价的一个上界，为了方便起见有时会定义 $\Phi(D_0) = 0$ 。

1.4 递推关系

（高中部分内容不再展开）

1.4.1 主定理

设 $a \geq 1$ ， $b \geq 1$ 为常数， $f(n)$ 为函数， $T(n)$ 为非负正数，且

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

那么就有

- $f(n) = O(n^{\log_b a - \epsilon})$ ， $\epsilon > 0$ 那么 $T(n) = \Theta(n^{\log_b a})$
- $f(n) = \Theta(n^{\log_b a})$ ，那么 $T(n) = \Theta(n^{\log_b a} \log n)$
- $f(n) = \Omega(n^{\log_b a + \epsilon})$ ， $\epsilon > 0$ ，且对某个常数 $c < 1$ 和所有充分大的 n 有 $a \cdot f(\frac{n}{b}) \leq c \cdot f(n)$ ，那么 $T(n) = \Theta(f(n))$

Proof. 设 $n = b^k$, 则有

$$\begin{aligned} T(n) &= a^k T(1) + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\ &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right), \quad T(1) = c_1 \end{aligned}$$

• 情况 1

$$\begin{aligned} T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\ &= c_1 n^{\log_b a} + O\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \varepsilon}\right) \\ &= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{(b^{\log_b a - \varepsilon})^j}\right) \\ &= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} (b^\varepsilon)^j\right) \\ &= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \cdot \frac{b^{\varepsilon \log_b n} - 1}{b^\varepsilon - 1}\right) \\ &= c_1 n^{\log_b a} + O(n^{\log_b a - \varepsilon} \cdot n^\varepsilon) \\ &= \Theta(n^{\log_b a}) \end{aligned}$$

• 情况 2

$$\begin{aligned} T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\ &= c_1 n^{\log_b a} + \Theta\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right) \\ &= c_1 n^{\log_b a} + \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{a^j}\right) \\ &= c_1 n^{\log_b a} + \Theta(n^{\log_b a} \log n) \\ &= \Theta(n^{\log_b a} \log n) \end{aligned}$$

• 情况 3

$$\begin{aligned}
T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\
&\leq c_1 n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} c^j f(n) \quad \left(af\left(\frac{n}{b}\right) \leq cf(n)\right) \\
&= c_1 n^{\log_b a} + f(n) \frac{c^{\log_b n} - 1}{c - 1} \\
&= c_1 n^{\log_b a} + \Theta(f(n)) \quad (c < 1) \\
&= \Theta(f(n)) \quad (f(n) = \Omega(n^{\log_b a + \epsilon}))
\end{aligned}$$

对于 $T(n) = 2T(\frac{n}{2}) + n \log n$, 就是不能使用主定理的, 因为不存在 $\epsilon > 0$ 使得 $n \log n = \Omega(n^{1+\epsilon})$.

这个可以用递归树进行求解, 恰好这个拆解得到的是平衡树, 只需要对 $f(n)$ 项求和即可,

$$\begin{aligned}
T(n) &= \sum_{i=0}^{\log n} n(\log n - i) \\
&= n \log^2 n = O(n \log^2 n)
\end{aligned}$$

1.4.2 Akra-Bazzi 定理

Akra-Bazzi 定理是分析分治算法时间复杂度的强大工具, 它推广了经典的主定理(Master Theorem), 适用于更广泛的递归关系,

对于递归关系式:

$$T(n) = \sum_{i=1}^k a_i T(b_i n + h_i(n)) + f(n)$$

其中:

- $a_i > 0$ 且 $0 < b_i < 1$ (对所有 $i = 1, \dots, k$)
- $|h_i(n)| = O\left(\frac{n}{\log^2 n}\right)$ (扰动项)
- $f(n)$ 满足某些正则条件

定义 p 为方程的解:

$$\sum_{i=1}^k a_i b_i^p = 1$$

则时间复杂度为:

$$T(n) = \Theta\left(n^p \left(1 + \int_1^n \frac{f(u)}{u^{p+1}} du\right)\right)$$

特征	Akra-Bazzi 定理
适用范围	非均匀划分的递归式
扰动项	允许 $h_i(n)$ 存在
驱动函数	$f(n)$ 形式更灵活
解的形式	积分表达式

表 2 主定理与 Akra-Bazzi 定理对比

1.4.3 应用示例

分析递归式 $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + n$:

- 1. 确定参数: $a_1 = a_2 = 1, \quad b_1 = b_2 = 1/2$
- 2. 解方程: $2 \cdot (1/2)^p = 1 \Rightarrow p = 1$
- 3. 计算积分: $\int_1^n \frac{u}{u^2} du = \ln n$
- 4. 最终解: $T(n) = \Theta(n(1 + \ln n)) = \Theta(n \log n)$

当递归式满足主定理条件时, Akra-Bazzi 定理给出的结果与主定理一致, 但前者适用性更广,

不过其实用数学归纳法会更舒适一些, 例如估计以下复杂度:

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + \Theta(n)$$

假设对任意的 $m < n$, 都有 $T(m) < cm$ 恒成立, 其中 c 为常数, 下面证明 $T(n) < cn$ 即可证明出 $T(n) = \Theta(n)$

$$\begin{aligned} T(n) &= T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + \Theta(n) \\ &< T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + dn \\ &< c\frac{n}{5} + c\frac{7n}{10} + dn \\ &= \left(\frac{c}{5} + \frac{7c}{10} + d\right)n < cn \end{aligned}$$

因而只需取 $c > 10d$ 即可,

1.4.4 递推关系求解练习

- 1. $T(n) = T(\frac{n}{2}) + T(\frac{n}{4}) + cn$

令 $n = 2^k$, $T(2^k) = T(2^{k-1}) + T(2^{k-2}) + c \cdot 2^k$

构建递归树知, 若设 $T(2^{k-i})$ 的结点数量是 $f[i]$, 则有 $f[i] = f[i-1] + f[i-2]$

且 $f[0] = f[1] = 1$, 因此叶子结点数量为 $f[k] = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^{k+1} - \left(\frac{1-\sqrt{5}}{2} \right)^{k+1} \right]$

$$\frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^{k+1} - \left| \left(\frac{1-\sqrt{5}}{2} \right)^{k+1} \right| \right] \leq f[k] \leq \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^{k+1}$$

$$\therefore f[k] = O(a^k) = O(n^{\log_2 a}), \text{ 其中 } a = \frac{1+\sqrt{5}}{2}$$

而对于计算耗时为: $cn \cdot \sum_{i=1}^n \left(\frac{3}{4} \right)^{i-1} = O(n) > O(n^{\log_2 a})$

$$\therefore T(n) = O(n)$$

$$2. T(n) = T(n-1) + \log n$$

$$\text{原式} = T(1) + \sum_{i=2}^n \log i$$

下证: $\sum_{i=2}^n \log i$ 与 $n \log n$ 同阶

由 Stolz 定理知, $\lim_{n \rightarrow \infty} \frac{\sum_{i=2}^n \log i}{n \log n} = \lim_{n \rightarrow \infty} \frac{\log n}{n \log n - (n-1) \log(n-1)} = \lim_{n \rightarrow \infty} \frac{\log n}{\log(n-1)}$

$$\therefore T(n) = \Theta(n \log n)$$

1.5 本章作业

1.5.1 势函数规范化构造及其平摊代价不变性证明

假定有势函数 Φ , 对所有 $i \geq 1$ 满足 $\Phi(D_i) \geq \Phi(D_0)$, 但 $\Phi(D_0) \neq 0$, 证明: 存在势函数 Φ' , 使得 $\Phi'(D_0) = 0$, 对于所有 $i \geq 1$, 满足 $\Phi'(D_i) \geq 0$, 且使用 Φ' 的平摊代价与使用 Φ 的平摊代价相同,

Proof. 设新的势函数为:

$$\Phi'(D_i) = \Phi(D_i) - \Phi(D_0) \quad (i = 0, 1, 2, \dots)$$

1. **零势初始化:**

$$\Phi'(D_0) = \Phi(D_0) - \Phi(D_0) = 0$$

2. **非负性保持** ($\forall i \geq 1$):

$$\Phi'(D_i) = \Phi(D_i) - \Phi(D_0) \geq 0 \quad (\text{由已知条件 } \Phi(D_i) \geq \Phi(D_0))$$

3. **平摊代价不变性**: 对于任意操作, 设实际代价为 c_i , 有:

$$\begin{aligned}\hat{c}'_i &= c_i + \Phi'(D_i) - \Phi'(D_{i-1}) \\ &= c_i + (\Phi(D_i) - \Phi(D_0)) - (\Phi(D_{i-1}) - \Phi(D_0)) \\ &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= \hat{c}_i\end{aligned}$$



1.5.2 聚集法计算平摊代价

假定对一个数据结构执行一个由 n 个操作组成的操作序列, 当 i 严格为 2 的幂时, 第 i 个操作的代价为 i , 否则代价为 1, 请使用聚集法确定每个操作的平摊代价,

考虑 n 个操作序列的总代价:

$$T(n) = \sum_{\substack{k=0 \\ 2^k \leq n}}^{\lfloor \log_2 n \rfloor} 2^k + (n - \lfloor \log_2 n \rfloor - 1)$$

展开后可得:

$$T(n) = (2^{\lfloor \log_2 n \rfloor + 1} - 1) + (n - \lfloor \log_2 n \rfloor - 1) \leq 3n$$

因此平摊代价为:

$$\frac{T(n)}{n} \leq 3 = O(1)$$

1.5.3 主定理运用

设递推方程 $T(n) = 7T(\frac{n}{2}) + n^2$ 给出了算法 A 在最坏情况下的时间复杂度函数, 算法 B 在最坏情况下的时间复杂度函数 $W(n)$ 满足递推方程 $W(n) = aW(\frac{n}{4}) + n^2$, 试确定最大的正整数 a , 使得 $W(n)$ 的阶低于 $T(n)$ 的阶,

$$\therefore T(n) = 7T\left(\frac{n}{2}\right) + n^2$$

$$\therefore a = 7, b = 2, f(n) = n^2$$

$$\therefore n^{\log_b a} = n^{\log_2 7}, f(n) = n^2 = O(n^{\log_2 7 - \varepsilon}), \varepsilon > 0$$

$$\therefore T(n) = \Theta(n^{\log_2 7})$$

$$\text{又} \therefore W(n) = aW\left(\frac{n}{4}\right) + n^2$$

$$\therefore n^{\log_b a} = n^{\log_4 a} = n^{\frac{1}{2} \log_2 a}, f'(n) = n^2$$

if $a = 16$, $f'(n) = \Theta(n^{\log_b a})$, $W(n) = \Theta(n^2 \log n) < \Theta(n^{\log_2 7})$, 满足题意

if $a > 16$, $f'(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$, $W(n) = \Theta(n^{\log_b a}) < \Theta(n^{\log_2 7})$

$$\therefore \sqrt{a} < 7, a < 49$$

$$\therefore a_{\max} = 48 > 16$$

$$\therefore a_{\max} = 48$$

注意 $a = 16$ 的讨论是必要的,

您的文档标题

作者姓名

2026 年 06 月 16 日

2 chapter2 归纳法

将一个问题分割成多个子问题, 这些子问题与原来的问题有相同的结构, 分而治之。

2.1 归纳算法举例

2.1.1 多项式求值

假设有 $n + 2$ 个实数 $a_0, a_1, \dots, a_n$ 和 x 的序列, 现在要对以下多项式求值:

$$P_{n(x)} = \sum_{i=0}^n a_i x^i$$

我们发现:

$$p_{n(x)} = (((((a_n x + a_{n-1})x + a_{n-2})x + a_{n-3}) \dots))x + a_0$$

所以, 我们得到具体表达式为:

$$p_{n-1}(x) = a_n x^{n-1} + a_{n-1} x^{n-2} + \dots + a_2 x + a_1$$

所以, 有递推关系:

$$p_{j(x)} = x p_{j-1}(x) + a_{n-j}$$

“Horner rule 是什么? 好吧就是上面那个”

2.1.2 生成排列

perm1(1)

perm1(m)

显然, 对于重复次数 $f(n)$ 有以下递推关系:

$$f(n) = n f(n-1) + n, n \geq 2$$

令 $f(n) = n! h(n)$, 那么

$$n! h(n) = n(n-1)! h(n-1) + n$$

即

$$h(n) = h(n-1) + \frac{1}{(n-1)!}$$

解得

$$h(n) = h(1) + \sum_{i=1}^{n-1} \frac{1}{i!} \leq e - 1$$

所以

$$f(n) = n!h(n) < (e - 1)n! < 2 \cdot n!$$

而输出语句的复杂度是 $\Theta(n)$ ，所以最终 $T(n) = \Theta(n \cdot n!)$

这里用到的思想其实是一种累加法的变体。也就是考虑递推式

$$f(n) = g(n)f(n-1) + h(n)$$

只需进行合适的换元，令 $p(n)$ 满足：

$$f(n) = \left(\prod_{i=1}^n g(i) \right) \cdot p(n)$$

这样得到：

$$\left(\prod_{i=1}^n g(i) \right) \cdot p(n) = \left(\prod_{i=1}^n g(i) \right) \cdot p(n-1) + h(n)$$

$$p(n) = p(n-1) + \frac{1}{\prod_{i=1}^n g(i)} h(n)$$

$$p(n) = p(1) + \sum_{i=1}^n \frac{1}{\prod_{j=1}^i g(j)} h(i)$$

由此，即可求出 $f(n)$ ，不过要注意无穷级数的作用。

“寻找下一个排列”的接口：std::next_permutation C++ 头文件中提供了 std::next_permutation(begin, end)。

功能：将序列重排为字典序中的下一个排列。如果当前已经是最大排列（降序），则重排为最小排列（升序）并返回 false，否则返回 true。

时间复杂度： $O(N)$ ，其中 N 是元素个数。平均情况下非常快（均摊常数时间）。实现原理（字典序算法）：它不需要知道总共有多少个排列，也不需要计算阶乘。它只通过比较相邻元素的大小来操作。

算法步骤（以 $[1, 3, 2]$ 为例）：从右向左查找第一对满足 $a[i] < a[i+1]$ 的元素。找到分界点 i 。在 $[1, 3, 2]$ 中， $1 < 3$ ，所以 $i=0$ （元素 1）。从右向左查找第一个大于 $a[i]$ 的元素 $a[j]$ 。在 $[1, 3, 2]$ 中，右边比 1 大的有 3 和 2，最右边的是 2。所以 $j=2$ （元素 2）。交换 $a[i]$ 和 $a[j]$ 。交换 1 和 2 $\rightarrow [2, 3, 1]$ 。反转 $i+1$ 到末尾的所有元素。反转 3, 1 $\rightarrow [2, 1, 3]$ 。

结果： $[2, 1, 3]$ 就是 $[1, 3, 2]$ 的下一个排列。

2.1.3 基数排序

令 $L = \{a_1, a_2, \dots, a_n\}$ 是一张有 n 个数的集合，其中每个数恰好有 k 位数字，就是说每个数具有 $d_k d_{k-1} \dots d_1$ 形式，这里 d_i 是 0 到 9 中的一个数字。

我们研究一种排序算法对 L 进行非降序排列，使算法在几乎所有的实际应用中都以线性时间运行。

在这个问题中，我们对整数的大小 k 施行归纳，而不是对数 n 这个对象来用归纳法。

时间复杂度： $\Theta(kn)$ ，空间复杂度： $\Theta(10n)$

2.1.4 寻找多数元素

令 $A[1..n]$ 是一个整数序列， A 中的整数 a 如果在 A 中出现的次数多于 $\lfloor n/2 \rfloor$ ，那么 a 称为多数元素。

- 初始，令候选元素 $c = A[1]$ ，并将计数器置 1
- 从 $A[2]$ 开始，逐个扫描元素，如果被扫描的元素和 c 相等，则计数器加 1；如果元素不等于 c ，则计数器减 1
- 如果所有的元素都已经扫描完毕并且计数器大于 0，那么返回 c 作为多数元素的候选者
- 如果在 c 和 $A[j]$ ($1 < j < n$) 比较时计数器为 0，那么对于 $A[j+1..n]$ 上的元素递归调用 candidate 过程。

$j \leftarrow m; c \leftarrow A[m]; count \leftarrow 1$

$c \leftarrow \text{candidate}(1) \quad count \leftarrow 0$

时间复杂度 $\Theta(n)$ ，空间复杂度 $\Theta(1)$

2.1.5 棋子移动游戏

洛谷题：<https://www.luogu.com.cn/problem/P1259>

对于最后处理的 4 对棋子是需要特殊对待的，不然会在 2 白 2 空 2 黑的时候卡死。

2.2 本章习题

2.2.1 RADIXSORT 算法时间复杂性

当输入由下列区间中的 n 个正整数组成时，以 n 为大小说明算法 RADIXSORT 的时间复杂性。

- 1. $[1 \dots n]$
- 2. $[1 \dots n^2]$
- 3. $[1 \dots 2^n]$

输入区间	时间复杂度	解释
$[1 \dots n]$	$O(n)$	最大位数 $d = \lceil \log_{10} n \rceil \approx O(1)$ ，视为常数，故线性时间。
$[1 \dots n^2]$	$O(n^2)$	位数 $d = \lceil 2 \log_{10} n \rceil$ ，但数字范围扩大，近似为 $d = O(n)$ 。
$[1 \dots 2^n]$	$O(n \cdot 2^n)$	位数 $d = \lceil n \log_{10} 2 \rceil = \Theta(n)$ ，且输入规模为 2^n 量级。

表 3 基数排序 (RADIXSORT) 的时间复杂性分析

2.2.2 寻找名人问题

在一个房间里有 n 个人，其中一个是名人。所谓名人，就是大家都认识他，但是他却不认识任何其他人。名人之外的其他人可能认识房间里另外的一部分人。你可以问房间里任何人问题，但是问题只能是：“你认识某某吗？”，对方回答“**Yes**”或者“**No**”。请设计一个时间复杂度为 $\Theta(n)$ 的归纳算法，通过问问题，把房间中的名人找出来。

- 1. **初始化候选人**：假设第一个人 为候选名人 `candidate = 0`。
- 2. **遍历排除非名人**：
- 3. 从第 2 个人到第 n 个人，依次与候选人比较：
 - 若候选人认识当前人，则候选人不是名人，更新候选人为当前人。
 - 若候选人不认识当前人，则当前人不是名人，保留候选人。
- 4. **验证候选人合法性**：
 - 检查是否所有人认识候选人。
 - 检查候选人是否不认识任何人。

您的文档标题

作者姓名

2026 年 06 月 16 日

3 chapter3 分治

把规模大的问题分解成小规模问题, 要求子问题**互相独立且与原问题形式相同**, 递归解决这些子问题, 再把他们的解合并到原问题中, 这样的策略称为**分治法**。

归纳法是**尾递归**思想, 通过扩展子问题解的方式得到原问题的解; 而分治法是划分的思想, 再合并得到原问题的解。

3.1 分治思想

分治范式通常由以下的步骤组成:

- **划分步骤**。在算法的这个步骤中, 输入被分成 $p \geq 1$ 个部分, 每个子实例的规模严格小于 n , n 是原始实例的规模。尽管比2大的其他小的常数很普遍, 但 p 的最常用的值是2。 p 也可以具有与 $\log n$, n^ϵ 一样高的数量级($0 < \epsilon < 1$)。
- **治理步骤**。如果问题的规模大于某个预定的阈值 n_0 , 这个步骤由执行 p 个递归调用组成, 阈值是由算法的数学分析导出的。
- **组合步骤**。这个步骤是组合 p 个递归调用的解来得到期望的输出值。

分治法所能解决的问题一般具有以下几个特征:

- 该问题的规模缩小到一定的程度就可以容易地解决;
- 该问题可以分解为若干个规模较小的相同问题, 即具有最优子结构性质 (问题的最优解所包含的子问题的解也是最优的);
- 利用该问题分解出的子问题的解可以合并为该问题的解;
- 该问题所分解出的各个子问题是相互独立的, 即子问题之间不包含公共的子问题。

3.2 分治算法举例

3.2.1 归并排序

如果 n 是2的幂时, 两个算法的执行比较次数相同。合并两个排好序的子数组, 所需要的元素比较次数在 $\frac{n}{2}$ 和 $n-1$ 之间, 所以最小比较次数是:

$$c(n) = 2c\left(\frac{n}{2}\right) + \frac{n}{2}, \quad n \geq 2$$

最多比较次数:

$$c(n) = 2c\left(\frac{n}{2}\right) + n - 1, \quad n \geq 2$$

由主定理知, $c(n) = \Theta(n \log n)$ 如果 n 是任意的正整数, 则算法执行元素比较次数是:

$$c(n) = c(\lfloor n/2 \rfloor) + c(\lceil n/2 \rceil) + bn, \quad n \geq 2$$

时间复杂度是 $\Theta(n \log n)$, 空间复杂度是 $\Theta(n)$ (开出一个临时数组就够用)。

3.2.2 选择问题: 寻找第 k 小元素

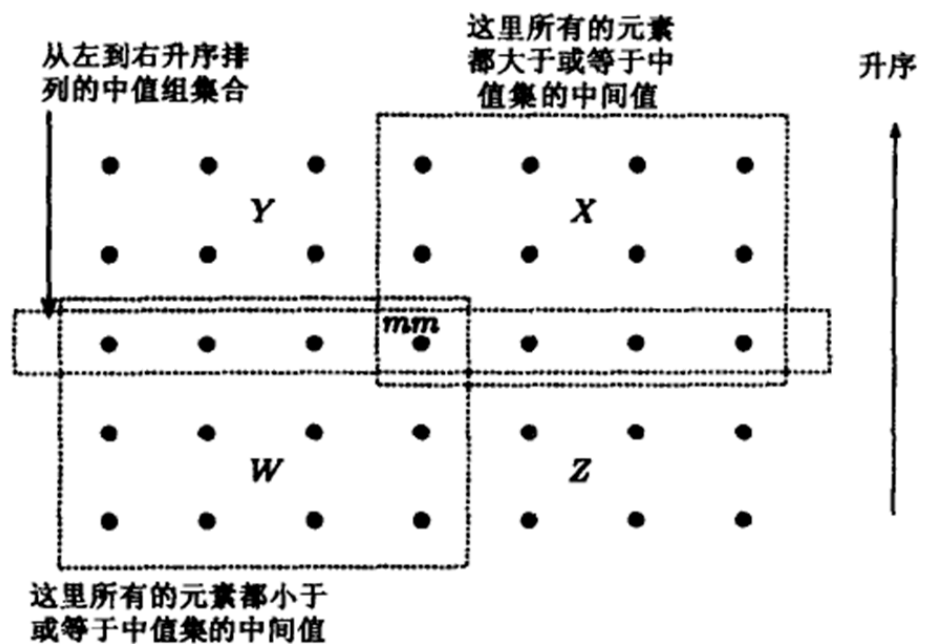
分组: 将 n 个元素每 5 个一组, 分成 $\lfloor \frac{n}{5} \rfloor$ 组。

找中位数: 对每组进行内部排序 (因为每组只有 5 个元素, 复杂度是 $O(1)$), 找出每组的中位数。

递归找“中位数的中位数”: 将上一步找到的所有中位数组成一个新集合 M , 递归调用算法, 找到 M 中的中位数, 记为 mm 。

划分 (Partition): 以 mm 为基准, 将原数组分为三部分:

- A_1 : 小于 mm 的元素。
- A_2 : 等于 mm 的元素。
- A_3 : 大于 mm 的元素。



设 A'_1 表示小于等于 mm 的元素集, A'_3 表示大于等于 mm 的元素集。

由于 A'_1 至少与 W 区域同样大, 然后用总的元素个数减去 A'_1 就是 A_3 :

$$|A'_1| \geq 3 \lceil \frac{\lfloor n/5 \rfloor}{2} \rceil \geq \frac{3}{2} \lfloor \frac{n}{5} \rfloor$$

$$|A_3| \leq n - \frac{3}{2} \lfloor \frac{n}{5} \rfloor \leq n - \frac{3}{2} (\frac{n-4}{5}) = n - 0.3n + 1.2 = 0.7n + 1.2$$

令 $0.7n + 1.2 \leq \lfloor 0.75n \rfloor$, 解得 $n \geq 44$ (这里的 $\frac{3}{4}$ 选的很好, 不到 $\frac{4}{5}$, 又超过 $\frac{2}{3}$)

所以有:

$$T(n) \leq \begin{cases} c & \text{if } n < 44 \\ T(\lfloor \frac{n}{5} \rfloor) + T(\lfloor \frac{3n}{4} \rfloor) + cn & \text{if } n \geq 44 \end{cases}$$

因而 $T(n) \leq \frac{cn}{1 - \frac{1}{5} - \frac{3}{4}} = 20cn$ 。

对于代码实现, 我们可以对快排进行改造:

C++

C++

```
void qsort(int l, int r){
    // if(l ≥ r) return;
    // 这个必须要注释掉, 否则就可能不输出了
    int i = l, j = r, x = a[(l + r) >> 1];
    while(i ≤ j){
        while(a[i] < x) i++;
        while(a[j] > x) j--;
        if(i ≤ j){
            swap(a[i], a[j]);
            i++; j--;
        }
    }
    if(k ≤ j) qsort(l, j);
    else if(k ≥ i) qsort(i, r);
    else{
        printf("%d\n", a[j + 1]);
        exit(0);
    }
}
```

快排是需要 $O(n)$ 的空间来存储递归中的左右边界的。运行时间最坏情况下为 $\Theta(n^2)$, 整体复杂度为 $\Theta(n \log n)$ 。

3.2.3 最值选择算法的改进

3.2.3.1 原始 MINMAX 算法

原始算法要求输入规模 n 必须为 2 的幂次 ($n = 2^k$), 其伪代码如下:

时间复杂度分析:

- 比较次数递推式:

$$C(n) = \begin{cases} 1 & n = 2 \\ 2C(n/2) + 2n & n > 2 \end{cases}$$

- 精确解: $C(n) = \frac{3n}{2} - 2$

3.2.3.2 改进后的 MINMAX 算法

改进算法通过以下修改支持任意正整数 n :

3.2.3.3 改进要点分析

- **新增递归基**: 处理 $n = 1$ 的情况 (第 3-4 行)
- **非对称划分**: 采用 $\lfloor n/2 \rfloor$ 和 $\lceil n/2 \rceil$ 划分
- **通用性**: 支持任意正整数输入规模

3.2.3.4 比较次数分析

改进后的比较次数满足递推关系:

$$T(n) = \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 2n & n > 2 \end{cases}$$

3.2.3.5 理论结论

- 改进算法比较次数趋近于 $\lceil \frac{3n}{2} \rceil - 2$ 但非严格相等
- 当 $n = 2^k$ 时与原始算法完全一致
- 最坏情况下比较次数上界为 $3\lceil n/2 \rceil - 2$

3.2.4 有序矩阵找第 k 小的数

给定一个 $n \times n$ 的矩阵, 其中每一行和每一列的元素均按升序排列。需要找到矩阵中第 k 小的元素。

1. **二分查找区间**: 初始区间为矩阵的最小值和最大值:

$$l = \text{matrix}[0][0], \quad r = \text{matrix}[n-1][n-1]$$

2. 统计小于等于mid的元素个数:

- 对于中点mid, 统计矩阵中 $\leq \text{mid}$ 的元素个数count
- 利用每行有序的性质, 在每行中用二分查找 (upper_bound) 统计

3. 调整区间:

$$\text{count} < k \Rightarrow l = \text{mid} + 1$$

$$\text{count} \geq k \Rightarrow r = \text{mid} - 1$$

4. 终止条件: 当 $l > r$ 时, l 即为第 k 小的元素

- 时间复杂度: $O(n \log n \log C)$
 - 二分查找次数: $O(\log C)$ ($C = \text{max} - \text{min}$)
 - 每次统计count: $O(n \log n)$
- 空间复杂度: $O(1)$

$n \leftarrow \text{len}(\text{matrix})$ $l \leftarrow \text{matrix}[0][0]$ $r \leftarrow \text{matrix}[n-1][n-1]$

3.2.5 多数元素问题

根据多数元素的定义可知多数元素必定在数组中出现次数最多, 而若将原数组分成2部分, 则在其中一部分出现次数最多的必定是多数元素。

该算法的时间复杂度是 $O(n \log n)$, 空间复杂度是 $O(\log n)$.

3.2.6 芯片测试问题

问题描述: 有 n 片芯片, 已知其中好芯片至少比坏芯片多 1 片, 需要通过测试从中找出 1 片好芯片。如何设计一个算法, 使用最少测试次数, 从中挑出 1 片好芯片?

测试方法: 将芯片放到测试台上, 2 片芯片互相测试, 并报告结果“好”或“坏”。好芯片报告结果一定是正确的, 坏芯片的报告不可靠 (可能对也可能不对)。

如果 n 是偶数, 那么将 n 片芯片两两一组做淘汰测试, 剩下芯片构成子问题, 进入下一轮分组淘汰。

淘汰规则: 如果报告结果为“好, 好”, 则任留 1 片, 否则全部抛弃, 对于 $n = 3$ 单独测试, 对于 $n = 2, n = 1$ 的情况, 则任取其一就是好芯片。

要证明分治算法的正确性, 只需要证明子问题与愿问题性质相同且算法是有退出的出口的。

如果 n 是奇数, 则任选1个芯片, 用其他 $n-1$ 个芯片对他进行测试, 若是好芯片, 则算法退出, 否则丢弃他, 转化为是偶数个的情况。

设 $W(n)$ 是最坏情况下的测试次数, 那么有:

$$W(n) = W\left(\frac{n}{2}\right) + O(n)$$

由主定理知, $W(n) = O(n)$

3.2.7 循环赛日程表

设有 $n = 2^k$ 个运动员要进行网球循环赛。现要设计一个满足以下要求的比赛日程表:

1. 每个选手必须与其他 $n-1$ 个选手各赛一次;
2. 每个选手一天只能赛一次;
3. 循环赛一共进行 $n-1$ 天。

```
void sol(int x, int y, int size, int st){
    if(size ≥ 2){
        sol(x, y, size - 1, st);
        sol(x + (1 << (size - 1)), y, size - 1, st +
(1 << (size - 1)));
        sol(x, y + (1 << (size - 1)), size - 1, st +
(1 << (size - 1)));
        sol(x + (1 << (size - 1)), y + (1 << (size -
1)), size - 1, st);
    }
    else{
        plan[x][y] = plan[x + 1][y + 1] = st;
        plan[x + 1][y] = plan[x][y + 1] = st + 1;
    }
}
```

3.2.8 向量卷积

给定向量 $a = (a_0, a_1, \dots, a_n), b = (b_0, b_1, \dots, b_n)$, 则有二者的卷积为 $c = (c_0, c_1, \dots, c_n)$, 其中

$$c_i = \sum_{j=0}^i a_j b_{i-j}$$

而卷积与多项式乘法等价，即令 $A(x) = a_0 + a_1x + \dots + a_{m-1}x^{m-1}$, $B(x) = b_0 + b_1x + \dots + b_{n-1}x^{n-1}$ ，则 $C(x) = A(x)B(x) = a_0b_0 + (a_0b_1 + a_1b_0)x + \dots + a_{m-1}b_{n-1}x^{m+n-2}$

如果采用暴力做法，先逐项相乘，再同类合并，那么复杂度将是 $O(n^2)$ 。

我们设计特定值 $x_1, x_2, \dots, x_{2n}$ 求出 $A(x_j)$ 和 $B(x_j)$, $j = 1, 2, \dots, 2n$ ，进一步地计算出 $C(x_j)$ ，最后利用多项式插值方法，根据 $C(x)$ 在 $x = x_1, x_2, \dots, x_{2n}$ 的插值点，求出 $C(x)$ 。

而基于此，我们最关键的计算提速步骤就在于利用分治算法的思想对 $A(x_i)$ 的求值步骤进行加速：

$$\begin{aligned} A_{\text{even}}(x) &= a_0 + a_2x + a_4x^2 + \dots + a_{n-2}x^{\frac{n-2}{2}} \\ A_{\text{odd}}(x) &= a_1 + a_3x + a_5x^2 + \dots + a_{n-1}x^{\frac{n-2}{2}} \\ A(x) &= A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2) \end{aligned}$$

这里相当于把本来 x 是 1 的 $2n$ 次方根，变成了 x^2 是 1 的 n 次方根，实现了问题的降级，从根本上使得复杂度从 $O(n^2)$ 变成了 $O(n \log n)$ 。

而在这个过程中，我们利用复数单位根的对称性可以进一步地加速，即：

$$\begin{aligned} A(w_j) &= A_{\text{even}}(w_j^2) + w_j A_{\text{odd}}(w_j^2) \\ A(w_{j+n}) &= A_{\text{even}}(w_j^2) - w_j A_{\text{odd}}(w_j^2) \end{aligned}$$

而如何通过 $C(w_i)$ 求出 c_i ，方法如下，但是不要深究。

构造多项式：

$$D(x) = C(w_0) + C(w_1)x + \dots + C(w_{2n-1})x^{2n-1}$$

可以证明（别管怎么证的）：

$$D(w_i) = 2nc_{2n-j}$$

至此，时间复杂度被优化到了 $O(n \log n)$

3.2.9 棋盘覆盖

其实就是这个题，分治思想：<https://www.luogu.com.cn/problem/P1228>

由此可想到将一个边长为 2^k 的正方形分成四个边长为 2^{k-1} 的正方形，特殊方格必在其中的一个正方形中。然后再中心摆上一个地毯，地毯缺少的一块朝向特殊方格所在正方形，这样相当于每个小正方形中有一个特殊方格。一直分治下去，直到边长为 2 结束。

3.2.10 最大子序列和问题

题目: <https://www.luogu.com.cn/problem/P1115>

1. 第一个数为一个有效序列
2. 如果一个数加上上一个有效序列得到的结果比这个数大, 那么该数也属于这个有效序列。
3. 如果一个数加上上一个有效序列得到的结果比这个数小, 那么这个数单独成为一个新的有效序列
4. 在执行上述处理的过程中实时更新当前有效序列的所有元素之和并取最大值。

这样代码就是:

```
for(int i=0;i<n;i++){
    cin>>a;
    f=max(f+a,a);
    ans=max(f,ans);
}
```

而如果用分治算法解决这道题, 只要求出左半部分和右半部分的最大子序列和, 最后计算一定要包含中间的最大子序列和, 三者取最大值就是答案。

```
int g(int l,int m,int r){//l,右边界、r,左边界、m,中间数
    int max1=a[m],max2=a[m+1];//最大值
    int sum1=0,sum2=0;//和, 循环用
    //从m向左延伸
    for(int i=m;i>=l;i--){
        sum1+=a[i];
        max1=max(max1,sum1);
    }
    //从m+1向右延伸
    for(int i=m+1;i<=r;i++){
        sum2+=a[i];
        max2=max(max2,sum2);
    }
    return max1+max2;
}
```

```
int he(int l,int r){
    if(l==r){
        return a[l];
    }
    int mid=(l+r)/2;
    return
    max(he(l,mid),max(he(mid+1,r),g(l,mid,r)));
}
```

3.3 分治算法的改进

主要就是两个渠道：减少子问题数量、减少递归内部计算量。

3.3.1 大整数乘法

设 n 是2的幂, $u = w \cdot 2^{\frac{n}{2}} + x, v = y \cdot 2^{\frac{n}{2}} + z$, 那么 $uv = wy2^n + (xy + wz)2^{\frac{n}{2}} + xz$, 但是这样时间复杂度是 $\Theta(n^2)$ 的。

我们可以考虑 $wz + xy = (w + x)(y + z) - wy - xz$, 这样就只需要进行3个 $\frac{n}{2}$ 的子问题了, 复杂度为 $\Theta(n^{\log 3}) = O(n^{1.59})$

3.3.2 矩阵乘法

暴力做法是 $\Theta(n^3)$ 的, 而如果只是粗暴的分区处理, 仍然是 $\Theta(n^3)$ 的, 这里我们介绍 STRASSEN 算法, 增加加减法的次数来减少乘法次数。

1. 与朴素分治算法相同, 分解出左上, 左下, 右上, 右下四个子矩阵。
2. 创建10个 $\frac{n}{2} \times \frac{n}{2}$ 的矩阵 S_i , 每个 S_i 保存两个子矩阵的和或差。
3. 用子矩阵和 S_i 相乘, 递归地计算7个 $\frac{n}{2} \times \frac{n}{2}$ 的矩阵 P_i 。
4. 通过 P_i 的不同组合进行加减, 得到 C 的子矩阵。

$$\begin{aligned}
S_1 &= B_{1,2} - B_{2,2} & S_2 &= A_{1,1} + A_{1,2} \\
S_3 &= A_{2,1} + A_{2,2} & S_4 &= B_{2,1} - B_{1,1} \\
S_5 &= A_{1,1} + A_{2,2} & S_6 &= B_{1,1} + B_{2,2} \\
S_7 &= A_{1,2} - A_{2,2} & S_8 &= B_{2,1} + B_{2,2} \\
S_9 &= A_{1,1} - A_{2,1} & S_{10} &= B_{1,1} + B_{1,2} \\
P_1 &= A_{1,1} \cdot S_1 & P_2 &= S_2 \cdot B_{2,2} \\
P_3 &= S_3 \cdot B_{1,1} & P_4 &= A_{2,2} \cdot S_4 \\
P_5 &= S_5 \cdot S_6 & P_6 &= S_7 \cdot S_8 \\
P_7 &= S_9 \cdot S_{10} \\
C_{1,1} &= P_5 + P_4 - P_2 + P_6 & C_{1,2} &= P_1 + P_2 \\
C_{2,1} &= P_3 + P_4 & C_{2,2} &= P_5 + P_1 - P_3 - P_7
\end{aligned}$$

对于矩阵规模不是 2 的幂的情况，只需要补 0 即可。

类似地，试只用三次乘法完成复数相乘 $(a + bi)(c + di) = (ac - bd) + (ad + bc)i$ 。

设

$$\begin{aligned}
\alpha &= ac \\
\beta &= bd \\
\gamma &= (a + b)(c + d)
\end{aligned}$$

则：

$$(a + bi)(c + di) = (\alpha - \beta) + (\gamma - \alpha - \beta)i$$

3.3.3 最近点对问题

方法是前面提到过的，按找 x 坐标排序后割开成两部分，最终答案就是 $\delta_l, \delta_r, \delta'$ 之间的最小值。但是关键的问题在于 δ' （左右之间的最小值）该怎么求。

直接用前面求出来的 $\delta = \min(\delta_l, \delta_r)$ ，在中线划分出一个带状区域： $[\text{mid_x} - \text{delta}, \text{mid_x} + \text{delta}]$ 范围内的点。这些点是“可疑分子”，可能构成更小的距离。

然后在带状区域中寻找更小的距离。在矩形带内从下到上滑动一个高为 δ ，长为 2δ 的矩形。显然对于一个边长为 δ 的正方形来说，两两距离超过边长的点最多有 4 个，所以最多有 8 个点能共存于这样高为 δ 长为 2δ 的矩形，故只需要从矩形带最下方的点开始，对每个点构造这样的矩形，最多枚举 7 个点即可。

PYTHON

PYTHON

```

import math

# 辅助函数：计算两点间的欧几里得距离
def dist(p1, p2):
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

# 暴力法：当点数很少时 ( $\leq 3$ ), 直接两两比较
def brute_force(points):
    min_dist = float('inf')
    n = len(points)
    for i in range(n):
        for j in range(i + 1, n):
            d = dist(points[i], points[j])
            if d < min_dist:
                min_dist = d
    return min_dist

# 核心递归函数
# points_x: 当前范围内按 X 排序的点列表
# points_y: 当前范围内按 Y 排序的点列表 (这是为了保持  $O(N \log N)$  的关键)
def closest_pair_recursive(points_x, points_y):
    n = len(points_x)

    # 1. 递归基：如果点很少，直接用暴力法
    if n <= 3:
        return brute_force(points_x)

    # 2. 分解 (Divide)
    mid = n // 2
    mid_point = points_x[mid]

    # 将 points_y 分割成左右两部分 (保持 Y 有序)
    # 注意：这里不能简单切片，因为 points_y 是全局 Y 有序的，
    # 需要根据 X 坐标判断归属
    left_y = [p for p in points_y if p[0] <= mid_point[0]]
    right_y = [p for p in points_y if p[0] > mid_point[0]]

    # 对应的 X 有序列表直接切片即可
    left_x = points_x[:mid]
    right_x = points_x[mid:]

    # 3. 解决 (Conquer) - 递归求解左右两边
    delta_left = closest_pair_recursive(left_x, left_y)
    delta_right = closest_pair_recursive(right_x, right_y)

```

```

# 取左右两边的较小值作为当前的最小距离 delta
delta = min(delta_left, delta_right)

# 4. 合并 (Combine) - 处理跨越中线的点对

# 4.1 构建带状区域 T (Strip)
# 从 Y 有序列表中, 筛选出 X 坐标在 [mid_x - delta,
mid_x + delta] 范围内的点
# 这些点是“可疑分子”, 可能构成更小的距离
strip = []
mid_x = mid_point[0]
for p in points_y:
    if abs(p[0] - mid_x) < delta:
        strip.append(p)

# 4.2 在带状区域内寻找更小的距离
# 关键优化: 对于 strip 中的每个点, 只需要检查它后面最多
7 个点
min_strip_dist = delta
for i in range(len(strip)):
    # j 从 i+1 开始, 且限制最多检查 7 个点
    # 同时如果 Y 方向距离已经超过 delta, 也可以提前停止
    (双重保险)
    for j in range(i + 1, min(i + 8,
len(strip))):
        if strip[j][1] - strip[i][1] ≥ delta:
            break
        d = dist(strip[i], strip[j])
        if d < min_strip_dist:
            min_strip_dist = d

# 返回最终的最小值
return min(min_strip_dist, delta)

# 主函数: 初始化排序并调用递归
def closest_pair(points):
    # 预处理: 分别生成按 X 和按 Y 排序的列表
    # 这一步只做一次, 复杂度 O(N log N)
    points_x = sorted(points, key=lambda p: p[0])
    points_y = sorted(points, key=lambda p: p[1])

    return closest_pair_recursive(points_x, points_y)

```

3.4 本章作业

3.4.1 快排空间复杂度

证明算法 QUICKSORT 的空间复杂度在 $\Theta(\log n)$ 与 $\Theta(n)$ 之间变动。并计算该算法的平均空间复杂度是多少？

快速排序的空间复杂度主要由递归调用栈的深度（栈中存储的是枢轴量）决定，而递归深度取决于划分的平衡性。

快速排序的平均空间复杂度由递归树的平均深度决定。通过分析划分的期望行为，可以证明平均深度为 $O(\log n)$ 。

划分点是随机选择的，划分后左右子数组大小的比例为任意 $(k, n - k - 1)$ 的概率均等。设 $D(n)$ 为递归树深度的期望值，满足以下递推关系：

$$D(n) = 1 + \frac{1}{n} \sum_{k=0}^{n-1} \max(D(k), D(n - k - 1))$$

由于 $D(k)$ 和 $D(n - k - 1)$ 对称，可简化为：

$$D(n) = 1 + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} D(k)$$

假设 $D(k) \leq c \log k$ 对所有 $k < n$ 成立，则：

$$D(n) \leq 1 + \frac{2c}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} \log k$$

通过积分近似求和（ $\sum \log k \approx n \log n - n$ ），可得：

$$D(n) \leq 1 + \frac{2c}{n} \left(n \log n - \frac{n}{2} \log \frac{n}{2} \right) \approx c \log n$$

因此， $D(n) = O(\log n)$ 。

3.4.2 多选问题

设 S 是包含 n 个不等数的集合，给定整数 r ($1 \leq r \leq n$)， $K = \{k_1, k_2, \dots, k_r\}$ 且 $1 \leq k_1 < k_2 < \dots < k_r \leq n$ 。要求在 S 中找出第 k_1 小，第 k_2 小，...，第 k_r 小的 r 个元素。例如， $r = 3$ ， $K = \{2, 4, 7\}$ ，则输出 S 中的第 2 小，第 4 小，第 7 小的元素。当 $r = 1$ 时，该问题就是一般选择问题。当 $r = n$ 时，该问题就变成了排序问题。

1. 设计一个最坏情况下时间复杂度为 $O(nr)$ 的算法。
2. 若 $r > 1$ ，设计一个最坏情况下时间复杂度为 $O(n \log r)$ 的算法。

时间复杂度 $O(nr)$ 的算法

对 K 进行排序，确保 $k_1 < k_2 < \dots < k_r$ 初始化结果数组 $R[1..r]$ R

时间复杂度 $O(n \log r)$ 的算法

对 K 进行排序，确保 $k_1 < k_2 < \dots < k_r$ $k_{mid} \leftarrow k_{\lfloor r/2 \rfloor}$ $pivot \leftarrow \text{SELECT}(S, k_{mid})$ 将 S 划分为： $L \leftarrow \{x \in S \mid x < pivot\}$ $E \leftarrow \{x \in S \mid x = pivot\}$ $G \leftarrow \{x \in S \mid x > pivot\}$ 初始化子问题的 K 集合： $K_L \leftarrow \{k_i \in K \mid k_i < k_{mid}\}$ $K_G \leftarrow \{k_i \in K \mid k_i > k_{mid}\}$ 递归解决子问题： $R_L \leftarrow \text{MultipleSelectOptimized}(L, K_L, |L|, |K_L|)$ $R_G \leftarrow \text{MultipleSelectOptimized}(G, K_G - |L| - |E|, |G|, |K_G|)$ $R_L \cup \{pivot\} \cup R_G$

1. 基本情况处理：

- 若 $r = 1$ ，直接调用 SELECT 算法（时间复杂度 $O(n)$ ）返回第 k_1 小的元素

2. 预处理：

- 对 K 排序，确保 $k_1 < k_2 < \dots < k_r$

3. 选取枢纽元：

- 计算 K 的中位数位置： $k_{mid} = k_{\lfloor r/2 \rfloor}$
- 使用 SELECT 算法在 S 中找到第 k_{mid} 小的元素 $pivot$

4. 划分数组：

- $L = \{x \in S \mid x < pivot\}$
- $E = \{x \in S \mid x = pivot\}$
- $G = \{x \in S \mid x > pivot\}$

5. 递归处理子问题：

- 左子问题 $K_L = \{k_i \in K \mid k_i < k_{mid}\}$ ：在 L 中递归求解
- 右子问题 $K_G = \{k_i \in K \mid k_i > k_{mid}\}$ ：在 G 中递归求解，调整查询排名 $k'_i = k_i - |L| - |E|$

6. 合并结果：

- 返回 $R_L \cup \{pivot\} \cup R_G$

时间复杂度分析

- 递归深度： $O(\log r)$ （每次 K 规模减半）
- 每层工作量： $O(n)$ （SELECT 和划分）
- 总时间复杂度： $O(n \log r)$

您的文档标题

作者姓名

2026 年 06 月 16 日

4 chapter4 动态规划

当子问题不独立时,如果能够保存已解决的子问题的答案,而在需要时再找出已求得的答案,就可以避免大量重复计算,从而得到多项式时间算法。而动态规划算法只解决每个子问题一次,然后将其答案保存起来,从而避免了每次遇到子问题时重新计算答案的工作。即动态规划通过空间开销来节省由于重复计算带来的时间开销。

最优性原理:如果问题的最优解所包含的子问题的解也是子问题的最优解,就称该问题具有最优子结构,即满足最优性原理(优化原则)。

并不是所有优化问题都适用优化原则,有时对某个子问题的解不一定最优,但延伸成整个问题的解时反而最优。

在证明一个问题满足最优子结构性性质时,可以考虑反证法,我们后面在 0-1 背包问题 4 题中具体展开。

4.1 最长公共子序列

Definition 4.1.1

题面: 给定两个序列 $X = \langle x_1, \dots, x_m \rangle$ 和 $Y = \langle y_1, \dots, y_n \rangle$, 求它们的最长公共子序列 (LCS) 的长度。子序列是指从原序列中删除若干元素(可能为 0 个)后保持原有相对顺序的序列。例如, 序列 $(1, 3, 5)$ 是 $(1, 2, 3, 4, 5)$ 的子序列, 而 $(1, 5, 3)$ 不是。公共子序列是指同时是两个序列子序列的序列。

要证明最长公共子序列问题满足最优子结构性性质, 设 $X = \langle x_1, \dots, x_m \rangle$ 和 $Y = \langle y_1, \dots, y_n \rangle$ 的 LCS 为 $Z = \langle z_1, \dots, z_k \rangle$ 。当 $x_m = y_n$ 时必有 $z_k = x_m$, 否则可将 x_m 追加到 Z 得到更长子序列导致矛盾, 此时 Z_{k-1} 必须是 X_{m-1} 和 Y_{n-1} 的 LCS, 若存在更长的 W 则 $W \oplus x_m$ 将形成更长公共子序列; 当 $x_m \neq y_n$ 时, 若 $z_k \neq x_m$ 则 Z 是 X_{m-1} 和 Y 的 LCS, 若 $z_k \neq y_n$ 则 Z 是 X 和 Y_{n-1} 的 LCS, 否则可通过选择更长子序列构造矛盾。由此 LCS 的最优解总由子问题最优解构成, 满足最优子结构性性质。

$s_{[-1]}$ 表示序列 s 的最后一个元素, $s_{[: -1]}$ 表示 s 去掉最后一个元素后的子序列

$$\text{LCS}(s_1, s_2) = \begin{cases} 0, & \text{if } s_1 \text{ or } s_2 \text{ is empty} \\ \text{LCS}(s_{1[: -1]}, s_{2[: -1]}) + 1, & \text{if } s_{1[-1]} = s_{2[-1]} \\ \max(\text{LCS}(s_{1[: -1]}, s_2), \text{LCS}(s_1, s_{2[: -1]})), & \text{otherwise} \end{cases}$$

```

while (cin >> s1 >> s2){
    memset(dp, 0, sizeof(dp));
    int n1 = strlen(s1), n2 = strlen(s2);
    for (int i = 1; i ≤ n1; ++i)
        for (int j = 1; j ≤ n2; ++j)
            if (s1[i - 1] == s2[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
    cout << dp[n1][n2] << endl;
}

```

4.2 最长上升子序列

题面：给定一个序列 $A = \langle a_1, \dots, a_n \rangle$ ，求其最长上升子序列 (LIS) 的长度。上升子序列是指从原序列中删除若干元素后得到的子序列，且子序列中的元素严格递增（即对于下标 $i_1 < i_2 < \dots < i_k$ ，有 $a_{i_1} < a_{i_2} < \dots < a_{i_k}$ ）。例如，序列 $(2, 5, 7)$ 是 $(1, 2, 5, 3, 7, 4)$ 的上升子序列。

先说一种最普遍的方法，因为所求为子序列，所以这道题很容易想到一种线性动态规划。我们需要求最长上升子序列，为了上升我们肯定要知道我们当前阶段最后一个元素为多少，为了最长我们还要知道当前我们的序列有多长。我们可以用前者来充当第一维描述：设 $F[i]$ 表示以 $A[i]$ 为结尾的最长上升子序列的长度，为了保证元素单调递增我们肯定只能从 i 前面的且末尾元素比 $A[i]$ 小的状态转移过来：

$$F[i] = \max_{0 \leq j < i, A[j] < A[i]} \{F[j] + 1\}$$

初始值为 $F[0] = 0$ ，而答案可以是任何阶段中只要长度最长的那一个，所以我们边转移边统计答案。

复杂度： $O(n^2)$

贪心加二分查找。

我们肯定要知道我们当前阶段最后一个元素为多少，还有当前我们的序列有多长。前两种方法都是用前者做状态，我们为什么不可以用后者做状态呢？

设 $F[i]$ 表示长度为 i 的最长上升子序列的末尾元素的最小值，我们发现这个数组的权值一定单调不降（仔细想一想，这就是我们贪心的来由）。于是我们按顺序枚举数组 A ，每一次对 $F[]$ 数组二分查找，找到小于 $A[i]$ 的最大的 $F[j]$ ，并用它将 $F[j + 1]$ 更新。

期望复杂度： $O(n \log n)$

```

int lengthOfLIS(vector<int>& nums) {
    vector<int> tails; // tails[i]表示长度为i+1的LIS的
    最小末尾元素
    for (int num : nums) {
        // 使用二分查找找到第一个大于等于num的元素位置
        auto it = lower_bound(tails.begin(),
tails.end(), num);
        if (it == tails.end()) {
            // 如果num比所有元素都大, 可以扩展LIS长度
            tails.push_back(num);
        } else {
            // 否则替换第一个大于等于num的元素, 保持tails数
            组的最小性
            *it = num;
        }
    }
    return tails.size(); // tails的大小就是LIS的长度
}

```

要同时求出最长公共子序列的长度和序列, 首先初始化二维数组 $c[m+1][n+1]$ (存储长度) 和 $b[m+1][n+1]$ (存储方向), 将 c 的第一行和第一列初始化为 0。然后遍历字符串 X 和 Y : 当 $X[i-1] = Y[j-1]$ 时, 令 $c[i][j] = c[i-1][j-1] + 1$ 并在 $b[i][j]$ 标记 " $\nwarrow$ "; 否则比较 $c[i-1][j]$ 和 $c[i][j-1]$, 若 $c[i-1][j] \geq c[i][j-1]$ 则令 $c[i][j] = c[i-1][j]$ 且标记 " $\uparrow$ ", 否则令 $c[i][j] = c[i][j-1]$ 且标记 " $\leftarrow$ ". 最终 $c[m][n]$ 即为 LCS 长度。回溯时从 $b[m][n]$ 开始: 若标记为 " $\nwarrow$ " 则递归 $b[i-1][j-1]$ 并记录 $X[i-1]$; 若为 " $\uparrow$ " 则递归 $b[i-1][j]$; 若为 " $\leftarrow$ " 则递归 $b[i][j-1]$ 。最后将记录的字符反转即得 LCS 序列。

$(c[m][n], b)$

4.3 矩阵链乘法

题面: 给定 n 个矩阵的链 $\langle A_1, A_2, \dots, A_n \rangle$, 其中矩阵 A_i 的维度为 $p_{i-1} \times p_i$, 求计算该矩阵链乘积所需的最少标量乘法次数。矩阵乘法满足结合律但不满足交换律, 因此不同的括号化方案会导致不同的计算代价。

例如, 计算 $A_{10 \times 100} \cdot B_{100 \times 5} \cdot C_{5 \times 50}$, 若按 $(AB)C$ 计算需 $10 \cdot 100 \cdot 5 + 10 \cdot 5 \cdot 50 = 7500$ 次乘法, 若按 $A(BC)$ 计算需 $100 \cdot 5 \cdot 50 + 10 \cdot 100 \cdot 50 = 75000$ 次乘法。

给定 n 个矩阵的链 $\langle A_1, A_2, \dots, A_n \rangle$, 其中矩阵 A_i 的维度为 $p_{i-1} \times p_i$

设一个由 n 个矩阵组成的序列的可选的括号化方案为 $P(n)$ 。则 $P(n)$ 递归式(卡特兰数)为:

$$P(n) = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n \geq 2 \end{cases}$$

其闭合形式解为:

$$P(n) = \frac{1}{n} \binom{2n-2}{n-1} = \frac{(2n-2)!}{n((n-1)!)^2} \approx \frac{4^n}{4\sqrt{\pi}n^{1.5}}$$

我们的求解过程分以下几步:

1. 描述最优解的结构特征。
2. 递归定义最优解的值。
3. 计算最优解的值。
4. 根据计算信息构造最优解。

定义 $A_{i:j}$ 为 $A_i A_{i+1} \cdots A_j$ 的乘积, 其中 $i \leq j$ 。如果 $A_{i:j}$ 可以继续分割(即 $i < j$), 利用括号化分割成 $A_{i:k}$ 和 $A_{k+1:j}$, 其中 $i \leq k < j$ 。

$A_{i:j}$ 的括号化后的计算代价为:

$$\text{计算代价}(A_{i:j}) = \text{计算代价}(A_{i:k}) + \text{计算代价}(A_{k+1:j}) + \text{相乘代价}(A_{i:k}, A_{k+1:j})$$

关键性质:

- 子问题的最优括号化方案包含在原问题的最优括号化方案中
- 通过求出所有子问题的最优解, 就能组成原问题的最优解
- 在求解子问题的最优分割方案时, 需要对所有可能的分割点 k 进行考察

假设矩阵链乘问题不满足最优子结构性质, 即存在某个子问题的最优解不是构成原问题最优解的一部分。

考虑矩阵链 $A_1 A_2 \cdots A_n$ 的最优括号化方案, 设其在位置 k 处分割:

$$A_{1:n} = (A_{1:k})(A_{k+1:n})$$

根据假设, 存在以下两种情形之一:

1. $A_{1:k}$ 不是子问题 $A_{1:k}$ 的最优括号化方案
2. $A_{k+1:n}$ 不是子问题 $A_{k+1:n}$ 的最优括号化方案

以第一种情形为例(第二种情形对称), 设存在更优的 $A_{1:k}^*$ 使得:

$$m^*[1, k] < m[1, k]$$

则可构造新的括号化方案:

$$A'_{1:n} = (A_{1:k}^*)(A_{k+1:n})$$

其总计算代价为：

$$\begin{aligned} m'[1, n] &= m^*[1, k] + m[k+1, n] + p_0 p_k p_n \\ &< m[1, k] + m[k+1, n] + p_0 p_k p_n \\ &= m[1, n] \end{aligned}$$

这与 $m[1, n]$ 是最优解的初始假设矛盾。因此，原假设不成立，矩阵链乘必须满足最优子结构性质。

定义 $m[i, j]$ 表示计算 $A_{i:j}$ 的乘积所需标量乘法的最少次数。计算 $A_1 A_2 \cdots A_n$ 的乘积所需标量乘法的最少次数可表示为 $m[1, n]$ 。

我们可以递归求解 $m[i, j]$ ：

- 当 $i = j$ 时， $A_{i:i} = A_i$ ， $m[i, i] = 0$ ，其中 $i = 1, 2, \dots, n$
- 当 $i < j$ 时，利用最优括号化方案：

$$m[i, j] = m[i, k] + m[k+1, j] + p_{i-1} p_k p_j, \quad \text{其中 } i \leq k < j$$

分段函数形式表示为：

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \} & i < j \end{cases}$$

定义 $s[i, j]$ 表示 $A_{i:j}$ 的最优分割方案中的值 k ，即满足：

$$m[i, j] = m[i, k] + m[k+1, j] + p_{i-1} p_k p_j$$

算法的时间复杂度是 $\Theta(n^3)$ ，空间复杂度是 $\Theta(n^2)$

4.4 多源最短路径问题

题面：给定带权有向图 $G = (V, E, w)$ ，其中每条边 $(i, j) \in E$ 的权重为 $w_{i,j}$ （可能为负值），求所有顶点对之间的最短路径长度。最短路径是指从源点到目标点的所有路径中，边权之和最小的路径。若从顶点 i 到 j 不可达，则距离为无穷大。

Floyd-Warshall 算法通过动态规划，以 $O(|V|^3)$ 的时间复杂度求解所有点对的最短路径。

$D \leftarrow l$ 将输入矩阵 l 复制到 D

4.5 0/1 背包问题

题面：给定 n 个物品，每个物品有重量 w_i 和价值 v_i ，背包容量为 C 。每个物品只能选择装入或不装入（0/1），求能装入背包的物品总价值

最大值。0/1 背包意味着每个物品最多只能选择一次，不能重复选择，也不能选择部分。例如，有 3 个物品 $[(w_1 = 2, v_1 = 3), (w_2 = 3, v_2 = 4), (w_3 = 4, v_3 = 5)]$ ，背包容量 $C = 5$ ，最优解是选择物品 1 和物品 2，总价值为 $3 + 4 = 7$ 。

$$\begin{aligned} & \max \sum_{i=1}^n v_i x_i \\ & s.t. \begin{cases} \sum_{i=1}^n w_i x_i \leq C \\ x_i \in \{0, 1\} & i = 1 \\ \dots \\ n \end{cases} \end{aligned}$$

设 $(y_1, y_2, \dots, y_n)$ 是上式的一个最优解，下证 $(y_2, y_3, \dots, y_n)$ 是下面子问题的最优解：

$$\begin{aligned} & \max \sum_{i=2}^n v_i x_i \\ & s.t. \begin{cases} \sum_{i=2}^n w_i x_i \leq C - w_1 y_1 \\ x_i \in \{0, 1\} & i = 2 \\ \dots \\ n \end{cases} \end{aligned}$$

考虑反证，假设 $(z_2, z_3, \dots, z_n)$ 是上述子问题的最优解，则：

$$\begin{aligned} \sum_{i=2}^n v_i z_i & \geq \sum_{i=2}^n v_i y_i \\ w_1 y_1 + \sum_{i=2}^n w_i z_i & \leq C \end{aligned}$$

所以有

$$v_1 y_1 + \sum_{i=2}^n w_i z_i \geq \sum_{i=1}^n v_i y_i$$

这说明 $(y_1, z_2, z_3, \dots, z_n)$ 是比 $(y_1, y_2, \dots, y_n)$ 更优的一个解，矛盾！

背包问题的最优解能够在 $\Theta(nC)$ 的时间内和 $\Theta(C)$ 的空间内得到。

$V[n, C]$

4.6 图像压缩问题

题面：给定 n 个像素点 $p_1, \dots, p_n$ ，每个像素用若干位二进制表示。将像素序列分割成若干连续段，同一段内像素用相同的位数存储，使总存储空间最小。背景：在灰度图像中，每个像素的灰度值在 0-255 之间，通常需要 8 位存储。但如果某个段的像素灰度值范围较小（如 0-15），

则只需 4 位存储即可。压缩策略是将连续像素按段分组，同一段内用统一的位数，需要额外存储段头信息（段内像素数和位数）。

对于给定的 n 个像素点 $p_1, p_2, p_3, \dots, p_n$ ，将其分割为连续的 m 个段 $S_1, S_2, S_3, \dots, S_m$ 。设第 i 个像素段 S_i 中，有 $N[i]$ 个像素（ $1 \leq i \leq m$ ），每个像素用 $b[i]$ 位表示（比如 0-15 的数字，要用 4 位表示）。

设 h_i 为分段 S_i 中最大灰度值所对应的二进制数的位数，即

$$h_i = \left\lceil \log \left(\max_{t[i]+1 \leq k \leq t[i]+N[i]} p_k + 1 \right) \right\rceil$$

由于 $p_k \leq 255$ ，则 $h_i \leq b[i] \leq 8$ ，需要用 3 位来表示 $b[i]$ 。我们给定 $N[i]$ 最多是 255，这样需要 8 位来存储。像素段 S_i 需要的总内存空间是 $N[i] * b[i] + 11$ 。

像素序列 $\{p_1, p_2, p_3, \dots, p_n\}$ 所需存储空间是：

$$\sum_{i=1}^m N[i] \times b[i] + 11m$$

假设图像压缩问题不满足最优子结构性质，即存在某个最优分段方案，其中至少一个子段的分割不是局部最优的。

考虑像素序列 p_1 到 p_n 的最优分割方案 $S_1, \dots, S_m$ ，设其中子段 S_k （ $1 \leq k \leq m$ ）不是该子区间的最优分割。则存在更优的分割 S_k^* 使得：

$$N[k] \times b[k] + 11 > N^*[k] \times b^*[k] + 11$$

将原方案中的 S_k 替换为 S_k^* ，得到新方案的总存储空间：

$$\begin{aligned} \text{Total}^* &= \sum_{i \neq k} (N[i] \times b[i] + 11) + (N^*[k] \times b^*[k] + 11) \\ &< \sum_{i=1}^m (N[i] \times b[i] + 11) = \text{Total} \end{aligned}$$

这与原方案的最优性矛盾。因此，最优分割方案中的每个子段必须自身是最优分割。

定义函数 $bm(i, j)$ 表示从像素 i 到 j 中像素灰度值 p_k 最大值所需的最少二进制位数：

$$bm(i, j) = \left\lceil \log \left(\max_{i \leq k \leq j} \{p_k\} + 1 \right) \right\rceil \quad (1 \leq i \leq n)$$

根据动态规划原理，存储长度满足：

$$S[i] = \min_{1 \leq k \leq \min\{i, 256\}} \left\{ S[i-k] + k \times bm(i-k+1, i) \right\} + 11$$

```

//s[i]记录1-i像素所需的最小空间, I[i]记录1-i像素的最优划分的
最后一个段的大小,
//b[i]由p[i]得出, 它记录p[i]需要几个比特来存储。
void compress2(int* p, int n, int* s, int* I, int* b)
{
    int header = 11; //一个段需要3位来存储每个像素点需要几个
    比特来存储, 8位来存储这个像素段有多少个像素点。
    int Imax = 256; //N[i]的最大值
    s[0] = 0;
    //这个循环是为了计算s[i]数组, 用I[i]数组保存在哪里分
    段的信息, 其它数组、变量都是辅助。
    for (int i = 1; i ≤ n; i++) {
        //注意s[i]初始化为INT_MAX, 而不是初始化为0, 因为
        s[i]要寻找最小值
        b[i] = length(p[i]);
        int bmax = 0; //我们要算出在一个像素段中, 最大的那个
        像素点需要用几个比特来存储, 然后这个像素段中所有点都用那么多比
        特来存储。
        s[i]=INT_MAX; //s[i]求的是最小值, 所以初始化为大整
        数。
        I[i];
        for (int k = 1; k ≤ i && k ≤ Imax; k++) {
            if (bmax < b[i - k + 1]) bmax = b[i - k +
            1]; //如果像素段出现更大像素点, 需要更大存储空间, 刷新bmax
            if (s[i] > s[i - k] + k * bmax) { //出现更
            小的s[i]则刷新
                s[i] = s[i - k] + k * bmax;
                I[i] = k; 记录分割位置: 第1-i像素点的最后
            一个段有多少数字。
            }
        }
        s[i] += header;
    }
}

```

时间复杂度 $O(n)$

4.7 最大子段和

题面：给定一个整数序列（可能包含负数），求其连续子段的最大和（即找出连续的若干个数，使它们的和最大）。子段是指序列中连续的一段，与子序列不同，子段必须保持连续性。例如，序列 $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$ 的最大子段和为 $4 + (-1) + 2 + 1 = 6$ ，对应的子段是 $[4, -1, 2, 1]$ 。如果所有数都是负数，则最大子段和为最大的那个负数（长度为1）。

```

cin>>n;
for(int i=0;i<n;i++){
    cin>>a;
    f=max(f+a,a);
    ans=max(f,ans);
}

```

4.8 多段图问题

题面：给定一个多段有向图，每条边有权重，求从源点到汇点的最短路径（或最大成本路径）。多段图是指顶点可以划分为若干个阶段（层），边只能从前一阶段的顶点指向后一阶段的顶点，不存在反向边或同阶段间的边。这种结构使得可以按照阶段顺序进行动态规划，每阶段只需考虑从上一阶段转移过来的最优解。

- $P(i, j)$: 从 V_i 中结点 j 到汇点 t 的最短路径
- $COST(i, j)$: $P(i, j)$ 的路径成本
- $D(i, j)$: V_i 中结点 j 的最优决策（选择的后继节点）

$$COST(i, j) = \begin{cases} 0 & \text{当 } i = k \text{ (终止条件)} \\ \min_{\substack{l \in V_{i+1} \\ \langle j, l \rangle \in E}} \{ c(j, l) + COST(i+1, l) \} & \text{当 } i < k \text{ (递推规则)} \end{cases}$$

- $BP(i, j)$: 从起点到阶段 V_i 节点 j 的最短路径
- $BCOST(i, j)$: 该路径的累计成本
- $D(i, j)$: 节点 j 选择的最优前驱节点

$$BCOST(i, j) = \begin{cases} 0 & \text{基础情况 (起点本身)} \\ \min_{\substack{l \in V_{i-1} \\ \text{边 } \langle l, j \rangle \in E}} [BCOST(i-1, l) + c(l, j)] & \text{递推情况} \end{cases}$$

- 将 n 个资源分配给 r 个项目
- 分配 j 个资源给项目 i 时，获得净利 $N(i, j)$
- 目标：找到使总净利最大的分配方案

1. 阶段划分：

- 构造 $r+1$ 段有向图（ r 个项目对应 r 个决策阶段+1 个终止阶段）
- 段 i ($1 \leq i \leq r$) 表示项目 i 的分配决策

2. 节点设计：

- 源节点 $s = V(1, 0)$
- 中间节点 $V(i, j) = \begin{cases} \text{共分配 } j \text{ 个资源给项目 } 1, \dots, i-1, 2 \leq i \leq r \\ j \in \{0, 1, \dots, n\} \end{cases}$
- 汇节点 $t = V(r+1, n)$

3. 边构造规则:

- 边形式: $\langle V(i, j), V(i+1, l) \rangle$, 满足 $j \leq l$ 且 $1 \leq i \leq r$
- 边权重:

$$w(\langle V(i, j), V(i+1, l) \rangle) = \begin{cases} N(i, l-j), & 1 \leq i < r \\ \max\{N(r, p) \mid 0 \leq p \leq n-j\}, & i = r \end{cases}$$

4. 求解方法:

- 最优解对应 s 到 t 的 **最大成本路径**
- 通过将边权重取负 ($-N(i, k)$) 转换为多段图最短路径问题
- 动态规划递推式:

$$\text{COST}(i, j) = \max_{\substack{l \geq j \\ \langle V(i, j), V(i+1, l) \rangle \in E}} \{ w(\langle V(i, j), V(i+1, l) \rangle) + \text{COST}(i+1, l) \}$$

4.9 最优二分检索树问题

题面: 给定 n 个有序关键字 $a_1 < a_2 < \dots < a_n$, 每个关键字 a_i 被检索的概率为 p_i , 未命中区间 E_j 的概率为 q_j . 构造一棵二分检索树, 使得加权平均检索次数最小。二分检索树是一棵二叉树, 每个节点存储一个关键字, 左子树的关键字都小于节点关键字, 右子树的关键字都大于节点关键字。检索时从根节点开始, 与节点关键字比较, 相等则命中, 不等则进入左或右子树继续比较。未命中区间 E_j 是指满足 $a_j < x < a_{j+1}$ 的值 x 的范围 (E_0 表示 $x < a_1$, E_n 表示 $x > a_n$), 即检索失败时 x 可能落入的区间。每次比较 (无论成功或失败) 都计为 1 次操作。

通过动态规划选择树结构, 使得:

- 成功检索 (命中关键字) 和
- 不成功检索 (未命中) 的 **加权平均比较次数最少**

$$\min \left(\sum_{i=1}^n p_i \cdot (\text{depth}(a_i) + 1) + \sum_{j=0}^n q_j \cdot \text{depth}(E_j) \right)$$

p_i	: 关键字 a_i 的命中概率
q_j	: 区间 E_j (失败检索区间) 的未命中概率
$\text{depth}(\cdot)$	: 结点在树中的深度

$$C(i, j) = \min_{i \leq k \leq j} \{C(i, k-1) + C(k+1, j) + W(i, j)\}$$

(e, root)

4.10 投资问题

题面：给定 m 元资金和 n 个投资项目，每个项目 i 投入 x 元的收益为 $f_i(x)$ 。求如何分配资金使总收益最大。投资分配问题是资源分配的典型应用，每个项目可以投入任意金额（包括 0），且收益函数 $f_i(x)$ 通常满足边际收益递减（即投入越多，单位投入的收益越低）。约束是所有项目的投入之和不能超过总资金 m ，目标是最大化总收益 $\sum_{i=1}^n f_i(x_i)$ 。

$\text{remaining} \leftarrow m \ (F[n][m], \text{alloc})$

给定 m 元资金和 n 个投资项目，每个项目的效益函数为 $f_i(x)$ ，表示投入 x 元到第 i 个项目的收益。目标是通过资金分配最大化总效益：

$$\max \sum_{i=1}^n f_i(x_i) \quad \text{s.t.} \quad \sum_{i=1}^n x_i = m, x_i \in \mathbb{N}$$

定义状态函数：

$F_k(x)$ = 将 x 元分配给前 k 个项目的最大效益

递推关系与边界条件：

$$F_k(x) = \max_{0 \leq x_k \leq x} \{f_k(x_k) + F_{k-1}(x - x_k)\}, \quad k > 1$$

$$F_1(x) = f_1(x)$$

4.11 本章作业

4.11.1 改进 floyd 算法

题面：Floyd-Warshall 算法可以求解所有点对的最短路径。请修改该算法，使其能够检测图中是否存在负权圈。负权圈是指图中存在一个环路，环路中所有边的权重之和为负数。当存在负权圈时，从某个顶点出发，可以无限绕行该圈使路径长度趋向负无穷，因此最短路径问题无意义。Floyd 算法在存在负权圈时无法收敛，需要通过检查距离矩阵的对角线元素来检测：若某个顶点到自身的距离变为负数，则说明存在经过该顶点的负权圈。

请修改求解所有点对的 Floyd 算法，使它可以检测到负权圈（一个负权圈是它的总长度为负的圈）。

- **负权圈问题**: 当图中存在总权重为负的环路时, Floyd 算法会陷入无限更新状态。例如, 若路径 $i \rightarrow j$ 包含负权圈, 每次绕行该圈都会使总路径权重减小, 导致算法无法收敛。

数学表现为:

$$\exists \text{ 环 } C : \sum_{e \in C} w(e) < 0 \Rightarrow \lim_{k \rightarrow \infty} dist^{(k)}[i][j] = -\infty$$

- **最优子结构破坏**: 动态规划要求子问题的最优解能构成全局最优解。负权圈使得通过无限绕行可不断降低路径权重, 导致:

局部最优 $\nrightarrow$ 全局最优

$$dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j]) \quad (\text{不收敛})$$

改进思路: 在算法完成后检查距离矩阵对角线元素, 满足:

$$\exists i \in V : dist[i][i] < 0 \Rightarrow \text{存在包含顶点 } i \text{ 的负权圈}$$

4.11.2 有向图的所有点对最短路径问题

题面: 给定一个带非负边权重的有向图, 求图中每个顶点到其他所有顶点的最短路径。请证明该问题满足最优子结构性质。最优子结构性是动态规划的核心特征, 指问题的最优解可以由其子问题的最优解组合而成。对于最短路径问题, 如果一条从 u 到 v 的最短路径经过中间节点 w , 那么从 u 到 w 和从 w 到 v 的子路径也必须分别是最短路径, 否则可以用更短的子路径替换, 得到更短的 u 到 v 路径, 产生矛盾。

设 G 是一个有向图, 其中每一条边有一个非负的长度, 如果两点之间无边, 令其长度为无穷。求图中每个顶点到其他顶点的最短路径。请证明该问题满足最优子结构性质。

假设最短路径问题不满足最优子结构性质, 则存在以下情形:

1. **假设**: 设 $p = \langle v_1, v_2, \dots, v_k \rangle$ 是从 v_1 到 v_k 的最短路径, 但存在子路径 $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ 不是 v_i 到 v_j 的最短路径。
2. **构造矛盾**:
 - 存在更优路径 p'_{ij} 满足 $w(p'_{ij}) < w(p_{ij})$
 - 构造新路径 $p' = \langle v_1, \dots, v_{i-1} \rangle \circ p'_{ij} \circ \langle v_{j+1}, \dots, v_k \rangle$
 - 计算总权重:

$$w(p') = w(p) - w(p_{ij}) + w(p'_{ij}) < w(p)$$

3. **矛盾结论**: 这与 p 是最短路径的初始假设矛盾, 故原命题得证。

4.11.3 k-shattering 问题

题面：给定一棵边带正权的完全二叉树，求删除权值总和最小的边集，使得删除后每棵子树的节点数都不超过 k 。 k -shattering 是指将树“打散”成若干个大小受限的子树。完全二叉树是指除最后一层外，每层节点都达到最大数量，且最后一层节点从左到右排列。删除一条边相当于将树分割为两棵子树。目标是通过删除一些边，使剩下的连通分量（子树）的节点数都不超过 k ，同时被删除边的总权值最小。这是一个典型的树形动态规划问题，需要在保持子树规模约束的同时最小化删除代价。

4.11.3.1 问题定义

给定边带权值的完全二叉树 $T = (V, E, w)$ ，其中 $w : E \rightarrow \mathbb{R}^+$ 。一个 k -shattering 集合是边集 $S \subseteq E$ 满足：

- 删除 S 后， T 被划分为 m 棵子树 $\{T_1, \dots, T_m\}$
- 每棵子树 T_i 的节点数 $|V(T_i)| \leq k$

求使 $\sum_{e \in S} w(e)$ 最小的 k -shattering 集合。

- **状态定义：** $dp[i][m]$ 表示以 i 为根的子树保留 m 个节点的最小代价
- **边界条件：**

$$dp[i][m] = 0 \quad \forall i > \lfloor n/2 \rfloor$$

- **状态转移：**
- 这里代码有问题,检查
- **时间复杂度：** $O(nk^2)$

最小代价： $\min(16, 9) = 9$ ，对应删除边集 $\{\langle b, d \rangle, \langle b, e \rangle, \langle a, c \rangle\}$ 。

shabi

作者姓名

2026 年 06 月 16 日

5 chapter5: 贪心算法

贪心算法很难有具体可行的方法对其适用性进行判定,但是一般具有2个重要的性质:**贪心选择性质**和**最优子结构性质**。

所谓贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择,即贪心选择来达到。这意味着每一步的最优选择不会影响未来的选择,即当前的选择只依赖于当前的状态,而不依赖于未来的状态。

对于一个约束优化问题:

1. **可行解**: 如果决策序列是关于一个问题的完整解的形式,且不破坏问题的约束条件,则称它为该问题的可行解。
2. **部分可行解**: 如果一个决策序列是问题的完整解形式的一部分,且满足约束条件,则称它为部分可行解。
3. **不可行的(部分)解**: 不满足问题约束条件的完整或部分解形式的决策序列,称为不可行的(部分)解。
4. **最优解**: 使目标函数值取最优值的可行解,称为最优解。
5. **贪心解**: 通过贪心算法得到的可行解,称为贪心解。
6. **贪心解性质**: 贪心解不一定是最优解。

5.1 贪心算法思想

贪心算法一般框架

Algorithm GREEDY(A, n) $solution \leftarrow \emptyset$ return ($solution$)

1. **贪心选择性质**: 通过SELECT函数保证每一步选择都是当前最优,且该选择最终能导向全局最优解(需进行数学证明)
2. **最优子结构**: FEASIBLE 函数确保解集的逐步扩展不破坏问题结构。
3. **终止条件**: 循环 n 次确保覆盖所有候选元素,最终解集 $solution$ 必然完备。

贪心算法产生全局最优解的最重要因素是选择策略,采用的贪心准则/贪心度量。

5.2 贪心算法举例

以下是为你整理的关于“最大股票收益问题”和“分数背包问题”的 Typst 格式笔记。针对这些算法概念,我结合了你偏好的“5-MINUTE LEARNINGS”与“BOTTOM-UP”步骤进行解析。

5.2.1 最大股票收益问题 (Best Time to Buy and Sell Stock)

题面: 给定一个数组 P , 其中 $P[i]$ 表示某支股票第 i 天的价格。你只能选择某一天买入, 并在未来的某一天卖出。设计一个算法来计算你能获取的最大利润。

思路: 遍历价格数组, 记录当前遍历过的最低价格 (作为潜在买入点), 同时计算每一天卖出可能获得的利润, 通过一次遍历更新全局最大值。

股票收益的核心在于**价格差的最大化**。我们将问题转化为寻找 $P[j] - P[i]$ 的最大值, 其中 $j > i$ 。

1. **状态定义:** 定义 `min_price` 为区间 $[0, i]$ 内的最低价格; `max_profit` 为前 i 天能获得的最大收益。
2. **状态转移:**
 - 更新最小值: `min_price = min(min_price, P[i])`
 - 尝试卖出: 当前收益 `Profit_i = P[i] - min_price`
3. **决策选择:** 如果 `Profit_i` 大于已知的 `max_profit`, 则更新最大利润。
4. **复杂度:** 时间复杂度 $O(n)$, 空间复杂度 $O(1)$ 。

5.2.2 分数背包问题 (Fractional Knapsack Problem)

题面: 给定 n 个物品, 每个物品 i 有价值 v_i 和重量 w_i 。现有一个容量为 W 的背包, 你可以选择将物品的一部分放入背包 (即允许取分数倍)。求背包能装下的最大总价值。

思路: 计算每个物品的单位价值 (性价比), 按照单位价值从高到低进行贪心选择。如果剩余容量不足以装下整个物品, 则切分该物品填满背包。

分数背包与 0/1 背包的区别在于其“可分性”。这种性质使得**贪心策略**能直接达到全局最优解。

1. **计算性价比:** 计算每个物品的单位价值 $\rho_i = \frac{v_i}{w_i}$ 。
2. **排序:** 将所有物品按 ρ_i 从大到小降序排列。

3. 贪心填充:

- 依次选取排序后的物品。
- 若 $W \geq w_i$, 则全额放入, 更新 $W = W - w_i$ 。
- 若 $W < w_i$, 则放入当前物品的 $\frac{W}{w_i}$ 部分, 将背包填满并停止。

4. 核心公式: $V_{total} = \sum_{i \in \text{Full}} v_i + \left(W_{rem} \cdot \frac{v_j}{w_j} \right)$

5.2.3 Dijkstra 算法

Algorithm: Dijkstra($V, E, w, s = 1$)

1. Initialization:

$$dist[1] \leftarrow 0, \quad dist[v] \leftarrow \infty \quad (\forall v \in V \setminus \{1\})$$

$$prev[v] \leftarrow \text{undefined} \quad (\forall v \in V)$$

$$Q \leftarrow \text{MinPriorityQueue}(V) \quad (Y = V, X = \emptyset)$$

2. Iteration:

while $Q \neq \emptyset$:

$$u \leftarrow \text{ExtractMin}(Q) \quad (u \text{ moves from } Y \text{ to } X)$$

for each neighbor v of u :

if $dist[u] + w(u, v) < dist[v]$:

$$dist[v] \leftarrow dist[u] + w(u, v) \quad (\text{Relaxation: } \lambda[v] \downarrow)$$

$$prev[v] \leftarrow u$$

$$\text{DecreaseKey}(Q, v, dist[v])$$

3. Result:

return ($dist, prev$) (where $dist[v] = \delta[v]$ for all $v \in X$)

- $\lambda[v]$: 算法中顶点 v 的当前距离标记 (对应伪代码中的 $dist[v]$)
- $\delta[v]$: 从源点 1 到 v 的真实最短路径长度
- Y : 未处理的顶点集合 (对应优先队列 Q)
- X : 已处理的顶点集合 ($X = V \setminus Y$)

在 Dijkstra 算法中, 当顶点 y 在第 8 步 (ExtractMin 操作) 被选中且 $\lambda[y]$ 有限时, 有:

$$\lambda[y] = \delta[y]$$

第一个离开 Y 的顶点是源点 1 , 初始化时:

$$\lambda[1] = \delta[1] = 0 \quad (\text{显然成立})$$

假设对于所有在 y 之前离开 Y 的顶点 x , 都有:

$$\lambda[x] = \delta[x]$$

下证对 y 也成立。

1. 路径存在性: 由于 $\lambda[y]$ 有限, 必存在从 1 到 y 的路径 P , 其长度为 $\lambda[y]$ 。

2. 关键路径构造: 设 $\pi = 1 \rightarrow \dots \rightarrow x \rightarrow w \rightarrow \dots \rightarrow y$ 是从 1 到 y 的真实最短路径 (π 具有子问题最优性, 否则一定存在一个更短的子路径优化 π), 其中:

- x 是路径 π 上最后一个在 y 之前离开 Y 的顶点
- w 是 x 的后继节点

3. 不等式链:

$$\begin{aligned}\lambda[y] &\leq \lambda[w] \quad (\text{因为 } y \text{ 先于 } w \text{ 被选中}) \\ &\leq \lambda[x] + \text{length}(x, w) \quad (\text{松弛操作保证}) \\ &= \delta[x] + \text{length}(x, w) \quad (\text{归纳假设}) \\ &= \delta[w] \quad (\pi \text{ 的子路径最优性}) \\ &\leq \delta[y] \quad (\text{非负权重下子路径} \leq \text{全路径})\end{aligned}$$

4. 等号成立: 由于 $\lambda[y]$ 是某条路径的实际长度, 而 $\delta[y]$ 是最短长度, 故:

$$\lambda[y] \geq \delta[y]$$

结合上述不等式链, 最终得:

$$\lambda[y] = \delta[y]$$

- 子路径不等式: 保证 $\delta[w] \leq \delta[y]$ (否则 $x \rightarrow w \rightarrow y$ 可能比全局最短路径更长)
- 松弛有效性: 确保 $\lambda[w] \leq \lambda[x] + \text{length}(x, w)$ 不会被负权边破坏
- 反例: 若 $\text{length}(x, w) < 0$, 可能导致 $\delta[w] > \delta[y]$, 破坏归纳链条

朴素的数组实现的 dijkstra 算法的复杂度是 $O(n^2 + m)$, 如果用堆进行优化, 时间复杂度是 $O(n + m) \log n$ (n 是顶点数量, m 是边数量)。如果是稠密图的话, 会使得堆优化速度更慢, 所以在稠密图 (邻接矩阵存储) 情况下, 应该用前者。

5.2.4 Kruskal 算法

Remark

每次把最短的且不构成回路的边加入, 直到成功构建最小生成树。

对于 Kruskal 算法, 算法的时间复杂度取决于排序步骤, 所以是 $O(m \log m)$ 。

- $G = (V, E)$: 带权无向连通图
- T_{mst} : 任意一棵最小生成树
- A_k : 算法在第 k 步迭代后加入的边集 (初始化 $A_0 = \emptyset$)

$\forall k \geq 0, A_k \subseteq T_{\text{mst}}$ (即 A_k 总是某个 MST 边的子集)

1. **基础步骤** ($k = 0$):

$A_0 = \emptyset$, 空集是任何集合的子集, 显然 $A_0 \subseteq T_{\text{mst}}$ 。

2. **归纳步骤**:

假设 $A_k \subseteq T_{\text{mst}}$, 设 $e = (u, v)$ 是第 $k + 1$ 步选择的边:

- 根据算法: e 是当前权重最小且不产生环的边 (即 e 连接 A_k 中两个不同连通分量)
- 根据安全边引理: $A_k \cup \{e\}$ 是某个 MST 边的子集

$\therefore A_{k+1} = A_k \cup \{e\} \subseteq T'_{\text{mst}}$ (T'_{mst} 为新的最小生成树)

3. **终止条件**:

当 $|A_{\text{final}}| = |V| - 1$ 时:

- A_{final} 无环 (算法保证)
- A_{final} 连通所有顶点 (否则存在连接不同分量的安全边, 算法不会终止)
- $A_{\text{final}} \subseteq T_{\text{mst}}$ 且 $|A_{\text{final}}| = |T_{\text{mst}}|$

$\Rightarrow A_{\text{final}}$ 即为一棵最小生成树

下面用反证法证明**安全边引理**

假设存在权重最小的跨分量边 $e \notin T_{\text{mst}}$ (T_{mst} 是包含 A 的最小生成树)

1. 将 e 加入 T_{mst} , 形成唯一环 C (因树加边必成环)

2. 环 C 上必存在边 $e' = (x, y)$ 满足:

- $e' \neq e$
- e' 连接 A 的不同连通分量 (因 e 连接两个分量)
- $w(e') \geq w(e)$ (因 e 是权重最小的跨分量边)

3. 构造新生成树: $T_{\text{new}} = T_{\text{mst}} \cup \{e\} \setminus \{e'\}$

- 总权重: $w(T_{\text{new}}) = w(T_{\text{mst}}) + w(e) - w(e') \leq w(T_{\text{mst}})$
- 当 $w(e) < w(e')$ 时: $w(T_{\text{new}}) < w(T_{\text{mst}})$ (与 T_{mst} 最小性矛盾)
- 当 $w(e) = w(e')$ 时: T_{new} 也是最小生成树, 且包含 $A \cup \{e\}$

5.2.5 Prim 算法

Prim 算法一般复杂度为 $\Theta(m + n^2)$, 可以被堆优化为 $O(m \log n)$.

给出一个含权无向图 $G = (V, E)$, 其中 $|V| = n, |E| = m$:

- 算法 MST 可在 $O(m \log n)$ 时间内找出最小生成树
- **稠密图特例**: 若存在 $\varepsilon > 0$ 使得 $m \geq n^{1+\varepsilon}$, 则时间复杂度可改进为 $O(\frac{m}{\varepsilon})$

设 ε 为固定常数, 其技术意义体现在:

$$\frac{m \log n}{m/\varepsilon} = \varepsilon \log n \Rightarrow \begin{cases} \text{当 } m = n^{1+\varepsilon} \text{ 时, 加速比为 } \Theta(\varepsilon \log n) \\ \varepsilon = \Omega(1) \text{ 时, } O(\frac{m}{\varepsilon}) \text{ 严格优于 } O(m \log n) \end{cases}$$

1. **稠密度控制**: ε 量化图的稠密程度

$$m = n^{1+\varepsilon} \Rightarrow \text{边数 } m \text{ 远超顶点数 } n$$

2. **复杂度改进**: 当 ε 为常数时

$$O\left(\frac{m}{\varepsilon}\right) = O(m) \quad (\text{线性时间优于 } O(m \log n))$$

3. **实际意义**: 对满足 $\exists c > 0, m > n^{1+c}$ 的图, 算法有显著加速

该定理的正确性建立在图论 (稀疏/稠密图分类) 与数据结构 (斐波那契堆优化) 的交叉分析上, 其核心在于发现: 当边数足够多时 ($m \gg n$), 可以通过特殊数据结构设计消除对数因子。

Prim 算法可以正确处理包含负权边的图的最小生成树(MST)问题, 其正确性不受边权值符号的影响。

Proof. 设 $T_{\min}$ 是 G 的最小生成树。将 T 中的 $n-1$ 条边按权重从小到大排列为 $e_{k_1}, e_{k_2}, \dots, e_{k_{n-1}}$, 将 $T_{\min}$ 中的边同样排列为 $e_{g_1}, e_{g_2}, \dots, e_{g_{n-1}}$ 。从头开始比较:

设第一条属于 T 但不属于 $T_{\min}$ 的边为 $e_{k_i} = (u_i, v_i)$, 则在 $T_{\min}$ 中存在一条属于 $T_{\min}$ 但不属于 T 的边 $e_{g_j} = (u_i, v_*)$ 。根据 Prim 算法的性质(2)可知:

$$w(e_{k_i}) \leq w(e_{g_j})$$

在 $T_{\min}$ 中删除 e_{g_j} 并加入 e_{k_i} 得到新树 T_{new} , 满足:

$$\text{cost}(T_{\min}) \geq \text{cost}(T_{\text{new}})$$

由于 $T_{\min}$ 是最小生成树, 又有:

$$\text{cost}(T_{\min}) \leq \text{cost}(T_{\text{new}})$$

故必然有:

$$\text{cost}(T_{\min}) = \text{cost}(T_{\text{new}})$$

即 T_{new} 也是最小生成树。

重复此替换过程直至 $T_{\text{new}} = T$, 最终证明 T 是最小生成树。 ■

5.2.6 活动选择问题

你有 n 个活动构成的集合 $S = \{a_1, a_2, \dots, a_n\}$, 这些活动都需要占用会议室, 且每个时刻会议室只能给一场活动提供服务。

每个活动 a_i 的开始时间为 s_i , 结束时间为 f_i , 其中 $0 \leq s_i < f_i < \infty$ 。如果活动 a_i 占用的时间为区间 $[s_i, f_i)$ 。

如果活动 a_i 和 a_j 的时间区间 $[s_i, f_i)$ 和 $[s_j, f_j)$ 没有重叠, 那么活动 a_i 和 a_j 相互兼容。

目标: 从所有活动中选出一个总时间规模最大相互兼容的活动子集。

如果给定活动序列没有按照结束时间从小到大排序, 先进行排序, 这步预处理非常重要, 没有这步预处理, 后面的贪心策略就无从开展。(其实也可以选择最晚开始的活动, 思路与最早结束大体一致, 不再额外展开)

下面开始使用贪心策略求解, 贪心策略每一步都要作出贪心选择 (greedy choice), 这样只有一个子问题需要处理。下面会分别讨论贪心的递归算法和迭代算法。

下面证明活动选择问题具有最优子结构。

定义 S_{ij} 为 a_i 结束后 a_j 开始前的活动集合, 定义 A_{ij} 为 S_{ij} 中规模最大且相互兼容的活动子集。 A_{ij} 中包含活动 a_k , 和两个子问题 A_{ik} 和 A_{kj} 。定义 $A_{ik} = A_{ij} \cap S_{ik}$ 且 $A_{kj} = A_{ij} \cap S_{kj}$, 这样有 $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$, 所以有 $|A_{ij}| = |A_{ik}| + |A_{kj}| + 1$ 。

用**剪切粘贴**方法可以证明最优解 A_{ij} 一定包含两个子问题 S_{ik} 和 S_{kj} 的最优解。如果你能找到一个由 S_{kj} 相互兼容的活动组成的集合 A'_{kj} , 其中 $|A'_{kj}| > |A_{kj}|$, 那么你就可以用 A'_{kj} 替换 A_{kj} 。此时就能构造出规模为

$$|A_{ik}| + |A'_{kj}| + 1 > |A_{ik}| + |A_{kj}| + 1 = |A_{ij}|$$

的相互兼容的活动集合, 与 A_{ij} 是最优解矛盾。同理也可以用于证明子问题 S_{ik} 。

定义 S_{ij} 的最优解的规模为 $c[i, j]$, 运用动态规划方法可以写出递归式: $c[i, j] = c[i, k] + c[k, j] + 1$, 即:

$$c[i, j] = \begin{cases} 0 & S_{ij} = \emptyset \\ \max_{a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & S_{ij} \neq \emptyset \end{cases}$$

虽然我们可以用动态规划方法求解, 但是我们忽略了活动选择问题的一个重要特征, 这个特征可以大大提高问题求解速度。

考虑任意非空子问题 S_k , 如果 a_m 是 S_k 中结束时间最早的活动, 那么 a_m 在 S_k 的某个最大兼容活动子集中。

下面证明活动选择问题具有贪心选择性质：证明：定义 A_k 是 S_k 的最大兼容活动子集，且 a_j 是 A_k 中结束时间最早的活动。

若 $a_j = a_m$ ，则已证明 $a_m \in A_k$ 。

若 $a_j \neq a_m$ ，令 $A'_k = (A_k - \{a_j\}) \cup \{a_m\}$ ，则 $|A_k| = |A'_k|$ 。又 a_j 是 A_k 中结束时间最早的活动， a_m 是 A'_k 中结束时间最早的活动， $f_m \leq f_j$ 。所以 A'_k 是 S_k 的最大兼容活动子集， $a_m \in A'_k$ 。

我们每次选择结束时间最早的且与当前活动兼容的活动，重复执行这个过程直到没有剩余活动可以选择。所选择的活动的结束时间必然是严格递增的。所以我们只需要按照结束时间单调递增的顺序处理所有活动，每个活动仅需要考察一次。

function RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n):

$m \leftarrow k + 1$

RECURSIVE-ACTIVITY-SELECTOR ($s, f, 0, n$)

当然也可以用动态规划的思想实现，同时用二分的思路对其进行优化，时间复杂度仍为 $O(n \log n)$ ：

function DP-ACTIVITY-SELECTOR(s, f, n) let $c[0..n]$ be new array $c[0] \leftarrow 0$

return $c[n]$

思考活动选择问题的一个变型：每个活动 a_i 除了开始时间和结束时间外，还有一个权值 v_i 。目标不再是求规模最大的活动子集，而是求权值总和最大的活动子集，也就是选择一个活动集合 A ，使得 $\sum_{a_k \in A} v_k$ 最大化。设计一个多项式时间的算法求解该问题。

其实就只需要修改一下动态规划算法的比较项就可以了，不再做更多展开。

5.2.7 区间图着色问题

假设有一组活动，将它们安排到一些讲堂，任意活动都可以在任意讲堂进行，我们希望用最少的教室完成所有活动，设计一个贪心算法求每个活动所在讲堂进行分配。

这个问题可以视为区间图着色问题 (interval-graph coloring problem)。区间图着色问题：构造一个区间图，顶点表示给定的活动，边连接不兼容的活动。要求用最少的颜色对顶点进行着色，相邻顶点的颜色不能相同。

一个朴素的方法是，每当有新活动时，就遍历所有讲堂，查看是否有空闲讲堂。如果没有空闲讲堂，就新开一个讲堂。为了方便选出空闲讲堂，我们可以使用优先队列。

算法执行步骤如下：

按照开始时间对活动进行排序。初始化一个小顶堆，按照结束时间排序，堆顶活动为最早结束的活动。检查每一个活动：如果堆为空，开辟一个新讲堂，选好讲堂后将活动加入堆中，同时将该活动记录到所在讲堂举行的活动名单中。如果堆不为空，和堆顶活动进行比较，如果堆顶活动的结束时间小于或等于当前活动的开始时间，将堆顶活动出队，并将当前活动放在堆顶活动所在讲堂进行，否则开辟一个新讲堂。选好讲堂后将活动加入堆中，同时将该活动记录到所在讲堂举行的活动名单中。所有活动检查结束后，输出所有讲堂举行的活动名单。

按照开始时间 s_i 对活动进行升序排序 初始化一个最小堆 H ，按结束时间排序 初始化空的讲堂列表 $classrooms$

5.2.8 最小延迟调度问题

给定 n 个作业，每个作业 J_i 有一个处理时间 t_i 和截止时间 d_i 。调度方案需为作业排列一个顺序，使得所有作业的延迟（完成时间减去截止时间）的最大值最小化。形式化定义为：

- 作业顺序 $\sigma = (J_{\sigma(1)}, \dots, J_{\sigma(n)})$ 的完成时间 $C_{\sigma(i)} = \sum_{k=1}^i t_{\sigma(k)}$ 。
- 延迟 $L_{\sigma(i)} = \max(0, C_{\sigma(i)} - d_{\sigma(i)})$ 。
- 目标是 minimize $L_{\max} = \max_i L_{\sigma(i)}$ 。

按照截止时间 d_i 对任务进行升序排序，使得 $d_1 \leq d_2 \leq \dots \leq d_n$ 初始化完成时间数组： $f(1) \leftarrow 0$ 初始化计数器： $i \leftarrow 2$

显然，所有无逆序且无空闲时间的调度具有相同的最大延迟。即对于满足 $d_{\sigma(1)} \leq \dots \leq d_{\sigma(n)}$ 的调度 σ ，其 $L_{\max}$ 仅取决于任务顺序，与相同截止时间任务的内部排列无关。

下面证明：按 d_i 升序排列（EDD 策略）的调度 σ^* 是最优解。

若 $d_i > d_j$ 但 i 排在 j 前，则称**相邻任务对** (J_i, J_j) 为**逆序对**。

对任意含逆序对 (J_i, J_j) 的调度 σ ：

$$\text{交换前延迟} : L_i = C + t_i - d_i, \quad L_j = C + t_i + t_j - d_j$$

$$\text{交换后延迟} : L'_j = C + t_j - d_j, \quad L'_i = C + t_j + t_i - d_i$$

其中 C 为 J_i 的开始时间。由于 $d_i > d_j$ 且 $t_j > 0$ ：

$$\max(L'_j, L'_i) \leq \max(L_i, L_j)$$

通过有限次相邻交换可消除所有逆序对：

- 每次交换不增加 $L_{\max}$
- 最终得到无逆序的 EDD 调度
- 由引理 1 知此调度最优

5.2.9 分数背包问题

给出 n 个大小为 $s_1, s_2, \dots, s_n$, 值为 $v_1, v_2, \dots, v_n$ 的物品, 并设背包的容量为 C 。要求找到非负实数 $x_1, x_2, \dots, x_n$, 使得在满足不超过背包总容量的约束条件下, 装进背包物品部分总价值最大。

即, 最大化 $\sum_{i=1}^n x_i v_i$ 且满足 $\sum_{i=1}^n x_i s_i \leq C$

选择单位大小的价值最大的物品即可, 不过多展开了 (过于简单)

5.2.10 带期限作业排序问题

问题: 设有一个单机系统、无其它资源限制且每个作业运行相等时间, 不妨假定每个作业运行 1 个单位时间。现有 n 个作业, 每个作业都有一个截止期限 $d_i > 0$, d_i 为整数。如果作业能够在截止期限之内完成, 可获得 $p_i > 0$ 的收益。问题要求得到一种作业调度方案, 该方案给出作业的一个子集和该作业子集的一种排列, 使得若按照这种排列次序调度作业运行, 该子集中的每个作业都能如期完成, 并且能够获得最大收益。

思路: 将输入的每个作业按收益从大到小进行排序, 用一个数组 vis 表示某时刻是否已经有作业正在运行, $vis[i] = 1$, 表示时间 i 被占用。然后从第 1 个作业开始往后搜索, 将该作业安排到 $vis[d]$ 时间运行, 如果 d 已经有安排, 将从 $d-1$ 开始往前搜索, 直到找到一个未被占用的时间点。如果找不到空时间点, 则跳过此作业, 继续向后搜索直到所有作业都搜索完。

按照收益 p_i 对作业进行降序排序 初始化时间槽数组 $vis[1..max_d]$ 全为 0 初始化结果列表 $res \leftarrow \emptyset$

按效益值降序排列的贪心调度算法总能得到最优解。

Proof. 采用交换论证法, 分三部分证明:

1. 基本设定:

- 设最优解作业集为 I , 贪心解为 J
- 若 $I = J$ 则得证, 故只需考虑 $I \neq J$ 的情况
- 此时必有 $I \not\subseteq J$ (否则 I 非最优) 且 $J \not\subseteq I$ (否则 J 更优)
- 存在 $a \in J - I$ 和 $b \in I - J$, 且由贪心法性质有 $p_a \geq p_b$

2. 调度表调整:

1. 设 S_J 和 S_I 分别为 J 和 I 的可行调度表
2. 对公共作业 $i \in I \cap J$, 调整调度表使得:
 - 若在 S_J 中 i 于 $[t, t+1]$ 执行, 在 S_I 中于 $[t', t'+1]$ 执行

- 当 $t < t'$ 时: 将 S_J 中 $[t', t' + 1]$ 的作业与 i 交换
- 当 $t' < t$ 时: 对 S_I 做对称调整

3. 最终得到新调度表 S'_J 和 S'_I , 使公共作业执行时间一致

3. 最优性保持:

- 在调整后的 S'_I 中, 将作业 b 替换为 a , 形成新解 $I' = (I - \{b\}) \cup \{a\}$
- 由于 $p_a \geq p_b$, 有 $\sum_{i \in I'} p_i \geq \sum_{i \in I} p_i$
- 重复此过程可将 I 逐步转换为 J 而不降低总效益

因此 J 必为最优解。 ■

作业集合 J 可行的充要条件: 存在按 d_i 非降序排列的调度序列 σ , 使得 $\forall i \in J$ 都在 d_i 前完成。

Proof. 必要性由定义显然。充分性证明:

- 设 σ 为按 d_i 非降序排列的调度
- 若存在作业 i_k 在 σ 中第 k 个执行且完成时间 $k > d_{i_k}$
- 则前 k 个作业的 $d_{i_j} \leq d_{i_k} < k$, 导致至少一个作业必然超期
- 矛盾, 故 σ 必为可行调度

■

5.2.11 黑白点最大匹配问题

预处理阶段: 将黑点 B 按 x^B 升序排列 (x 相同则按 y^B 升序) 将白点 W 按 x^W 降序排列 (x 相同则按 y^W 降序)

匹配阶段: $count \leftarrow 0$ $i \leftarrow 1, j \leftarrow 1$

Proof. 我们通过以下步骤证明:

- 设最优匹配为 M^* , 其匹配数为 OPT
- 设算法得到的匹配为 M , 匹配数为 $|M|$
- 定义特征函数 $\phi(M)$ 为匹配中黑点的字典序排列

对于任意最优匹配 M^* , 存在匹配 M' 使得:

1. $|M'| = |M^*|$
2. $\phi(M')$ 与算法输出 M 的字典序相同 (可以通过调整使得相同)

假设存在更优匹配 M^* , 取第一个差异点:

1. 设算法第 k 步选择 (b_i, w_j)
2. 而 M^* 选择 $(b_i, w_{j'})$ 或 $(b_{i'}, w_j)$ (如果不是则通过交换进行修正)
3. **情况 1:** $w_{j'} \neq w_j$

由于算法优先选 最大的 w_j

$$\Rightarrow x_j^W \geq x_{j'}^W$$

$$y_j^W \geq y_{j'}^W$$

可交换 w_j 和 $w_{j'}$ 而不减少匹配数

4. 情况 2: $b_{i'} \neq b_i$

算法选择 最小的 b_i

$$\Rightarrow x_i^B \leq x_{i'}^B$$

$$y_i^B \leq y_{i'}^B$$

用 b_i 替换 $b_{i'}$ 可保持可行性

上述交换均会使 M^* 的特征向量 $\phi(M^*)$ 更接近算法结果 M , 与 M^* 的最优性假设矛盾。 ■

5.2.12 过河问题

Oliver 和 $N - 1$ 个朋友共 N 人需要从河东岸渡到西岸。他们只有一艘小船, 每次最多可载两人。当两个人 i 和 j 一起渡河时, 所需时间为 $\max(T_i, T_j)$ 。求所有人过河的最少总时间。

很明显的贪心, 主要讨论两种情况: 1、让速度最快的往返接送, 每次送两个人所用时间为两倍最快的人所用的时间加上两个最慢的人的所用的时间之和。即 $a[n] + a[1] + a[n - 1] + a[1]$;

2、先让两个最快的人过河, 再让第二快(谁将船送回都不影响)的人送回船, 让两个最慢的人过河, 再将留在河对岸的速度最快的人把船送回。所用时间为 $a[2] + a[2] + a[n] + a[1]$;

Sort $t[1..n]$ in ascending order Initialize $ans \leftarrow 0, lef \leftarrow n$

ans

5.2.13 树上的最小顶点覆盖

给出一种有效的贪心算法, 该算法能在线性时间内找到一棵树中的一个最优顶点覆盖。

表示方案一采用定长数组存储每个结点的孩子结点, 设给定一棵树 T , 其根结点为 $T.root$, 阶数为 $T.degree$ 。对于每个结点 x , 具有颜色属性 $x.color$ 和包含 $T.degree$ 个孩子结点指针的数组 $x.children$ 。该方案借鉴红黑树思想进行结点着色, 规定每个结点非红即黑, 所有空结点默认为黑色, 且红色结点的所有孩子结点必须为黑色。在贪心算法实现中, 首先将所有孩子结点为空结点的非空结点标记为红色, 然后采用自底向上的遍历策略。具体实现采用方法一的后序遍历方式, 其

着色规则为：所有空结点保持黑色；对于非空结点，若其所有孩子结点均为黑色则将该结点着为红色，否则着为黑色。通过自底向上的后序遍历过程，当所有结点完成着色时，最终所有黑色结点的集合 C 即构成该树的最优顶点覆盖解。该方法的正确性依赖于树的后序遍历特性与颜色约束规则的结合，确保在多项式时间内获得满足条件的顶点覆盖。

您的文档标题

作者姓名

2026 年 06 月 16 日

6 chapter6: 图的遍历

6.1 割点

6.2 强连通分量

6.3 A*算法

您的文档标题

作者姓名

2026 年 06 月 16 日

7 chapter7: NP 完全问题

在计算理论中, Church-Turing 论题奠定了可计算性的理论基础, 它明确指出一个问题的可计算性取决于其能否在图灵机上通过有限次计算获得正确结果。这一理论框架将人类面临的所有问题划分为可计算与不可计算两大类, 其中不可计算问题(或称不可解问题)具有本质上的局限性——无论投入多少计算时间与资源, 都无法通过计算机获得解决方案。值得注意的是, 这里的"有限次计算"是一个相对宽松的条件, 主要强调**计算过程必须具有终止性**。

Cook-Karp 论题在此基础上进一步细化了可计算问题的分类标准, 引入了实际可计算性的重要概念。该论题将**多项式时间**计算作为实际可计算性的判定标准, 即在图灵机上通过多项式时间步数能获得正确结果的问题才被视为实际可计算的。这一划分产生了实际可计算问题与难解问题的新分类: 前者具有可行的算法解决方案, 后者虽然理论上仍可通过计算机求解(如采用指数时间算法), 但由于缺乏多项式时间算法而被认为在实际应用中难以处理。这两个论题共同构成了计算复杂性理论的重要基石, 为计算机科学中问题可解性的研究提供了理论基础。

可计算的问题需要满足:

1. 必须是存在解的问题。(例如: 是否有外星生命就不行)
2. 可以通过计算机求解的问题, 可以用一系列机械的数学步骤(算法)来求解。(例如: 证明一个整数 n 的阶乘的末尾数只能为0,1,4,5,6,9种的一个(费马小定理), 这个问题是有解的, 但目前没有找到有效的证明方法, 也不属于可计算问题)
3. 一些需要人类创造力、想象力等方面的问题, 也不属于可计算问题。

算法的复杂性是指解决特定问题时, 具体算法执行所需的时间量度, 这反映了算法本身的效率特性。与之相对, 问题的复杂性则表征问题本身固有的难度级别, 这是问题与生俱来的本质属性。

从根本上看, 问题的复杂度是问题自身内在的、不可改变的特性; 而算法的复杂度则受到多重现实条件的制约, 既包括逻辑层面的算法设计能力, 也包含物理层面的实现条件限制。这种区分揭示了计算理论中问题本质属性与解决方案特性之间的重要分野。

求解一个问题的好的算法的时复杂度函数应该恰好等于问题的计算复杂度。

算法最优性指的是在解决特定问题的所有可能算法中, 效率最高的算法。这种最优性可以从两个重要维度进行衡量: 最坏情况下最优和平均情况下最优。

最坏情况下最优算法是指在解决某问题的所有算法中,不存在其他算法在最坏情况下的时间复杂性比该算法更低。这意味着该算法在最不利输入情况下仍能保持最佳性能。

平均情况下最优算法则是指在解决某问题的所有算法中,不存在其他算法在平均情况下的时间复杂性比该算法更低,这表明该算法在典型输入情况下具有最优表现。这两种最优性标准为算法性能评估提供了重要理论框架。

7.1 计算复杂度的分析

7.1.1 平凡下界

有的问题必须扫描完所有输入才能得到解,算法总要得到所有输入,输出才能解决好,因此算法的输入规模/输出规模是它的平凡下界。

7.1.2 计数所需最少运算数

找最大问题:在 n 个不同的数中找最大的元素,基本元素是元素的比较,则对于任何算法在最坏情况下至少要做 $n - 1$ 次比较。

7.1.3 决策树(Decision Tree)

决策树是一种重要的算法分析工具,它以二叉树的形式直观地展现了算法执行过程。这种特殊的树结构中,每个内部节点都代表着算法执行过程中的一次关键运算操作,比如常见的比较运算;而叶子节点则对应着算法最终输出的结果。值得注意的是,在某些特定问题中,算法的输出结果也可能出现在内部节点上,而不仅限于叶子节点。

当我们分析一个具体算法的决策树时,可以清晰地看到算法处理输入实例的完整流程。算法执行总是从决策树的根节点开始,沿着特定的路径向下遍历。在这个过程中,每个节点都会执行一次基本操作,并根据操作结果(如小于、等于或大于)决定下一步的执行路径。这种执行过程实际上就对应着从根节点到某个叶子节点或内部节点的一条完整路径,完整地记录了算法处理该输入实例的整个过程。

从算法分析的角度来看,决策树为我们提供了深入理解算法行为的窗口。对于给定的算法和固定大小的输入规模 n ,不同的输入实例会导致算法在决策树的不同节点终止。这些终止节点实际上将输入实例进行了分类,每个节点都对应着一类具有相同特征的输入。通过分析决策树中节点的总数,特别是叶子节点的数量,我们可以准确地把握算法需要处理的不同输入类别的数量,这对评估算法的复杂度具有重要

意义。决策树的这种特性使其成为算法设计和分析中不可或缺的重要工具。

对于给定的 n ,任何通过比较对 n 个元素排序的算法的决策树的深度至少是 $\lceil \log n! \rceil$.

$$\begin{aligned}\log n! &= \sum_{j=1}^n \log j \geq \int_1^n \log x \, dx = \log e \int_1^n \ln x \, dx \\ &= \log e (n \ln n - n + 1) \\ &= n \log n - n \log e + \log e \\ &\approx n \log n - 1.5n\end{aligned}$$

存在多项式时间算法的问题是易求解的(tractable), 不存在多项式时间算法的问题是难求解的(intractable).

7.2 判定问题

对于判定问题, 可以使用布尔值来表示问题的答案, 并通过判断输出的是"Yes"还是"No"来确定问题的解。而对于优化问题, 需要确定一个最优解; 而对于计数问题需要计算问题实例的数目。**为了便于研究, 计算复杂性理论研究中将问题限定为判定问题。**

而其他类的问题也都可以通过适当的变换**转化为判定问题**, 且不改变问题的难易程度, 从而利用计算复杂性理论进行分析和研究。

1. 寻找多数元素问题(MAJORITY): 给定一个含有 n 个元素的数组, 寻找并返回数组中的多数元素, 如果没有多数元素, 则返回 none.

可以转换为, 多数元素判定(MAJORITY-D): 给定一个含有 n 个元素的数组, 问数组中是否存在多数元素。

2. 哈密顿回路搜索(HCS): 给定一个无向图 G , 搜索并返回其中一条哈密顿回路, 如果没有哈密顿回路, 则返回"No". (哈密顿回路, 是指经过图中每个顶点恰好一次的回路, 如果图 G 中存在哈密顿回路, 则称 G 为哈密顿图。

可以转化为, 哈密顿图判定问题(HC): 给定一个无向图 G , 判断 G 是否是哈密顿图。

3. 最长公共子序列问题(LCS): 给定取自相同字母集合的两个序列 X 和 Y , 求 X 和 Y 的最长公共子序列。

可以转化为, K 公共子序列问题(KCS): 给定取自相同字母集合的两个序列 X 和 Y 以及正整数 K , 问 X 和 Y 是否存在长度不小于 K 的公共子序列。

4. 货郎担问题(TSP): 给定 n 个城市的集合, 城市 i 和城市 j 之间的距离为整数, 求一条经过每个城市恰好一次最后回到出发点的最短回路。

TSP-D: 给定 n 个城市的集合和一个正整数 K , 城市 i 和城市 j 之间的距离为整数, 问是否存在经过每个城市恰好一次最后回到出发点, 且长度不超过 K 的回路。

7.3 P 类和 NP 类

7.3.1 图灵机模型

图灵机的逻辑结构包括:

- 一个无限长度的带子, 这个带子被分成了一个一个的单元格, 每个格子有包含一个来自字母表的符号 (含有空白)。
- 一个读写头, 可在纸带上左右移动, 并可以读、修改格子上的符号。
- 一个状态寄存器, 用来存储图灵机的当前状态。图灵机的所有可能状态数目是有限的, 并且有一个特殊的状态, 称为停机状态。
- 一个指令表 (控制单元), 根据机器当前所处的状态和带子上当前的符号, 指示机器进行特定的操作。比如: 擦除或者写入一个符号、向左或者向右移动磁头。并改变状态寄存器的值, 令机器进入一个新的状态或保持状态不变。

图灵机工作时, 大体重复如下过程:

1. 读写头读出当前格子的符号
2. 根据当前状态和读到的符号找到对应的控制指令, 并执行以下三个动作:
 1. 读写头在格子上擦除或写入一个数字或符号
 2. 图灵机状态转移到一个新的状态
 3. 读写头向左或向右移动一格

上述过程会一直重复, 直到机器到达一个终止状态, 或者无法继续执行为止。图灵机的状态转移方案可以是唯一的, 也可以不是唯一的。据此, 图灵机模型可以分为: 确定性图灵机(Deterministic Turing Machines, DTM)和非确定性图灵机(Nondeterministic Turing Machines, NTM)

可以用 DTM 模型描述的算法, 称之为确定性算法。设 A 是求解问题 Π 的一个算法, 如果在求解问题 Π 的一个实例时, 在整个执行过程中, 每一步都只有一种选择, 则称 A 是确定性算法。因此如果对于同样的输入, 实例一遍又一遍地执行, 它的输出从不改变。如果 A 的时间是关于输入实例大小的多项式时间, 则称 A 是多项式时间确定性算法。

非确定性图灵机 (NTM) 是 DTM 的一种推广, NTM 和 DTM 的不同之处在于, 在计算的每一时刻, 根据当前状态和读写头所读的符号,

NTM 存在多种状态转移方案，机器可以任意选择其中一种方案继续运行，直至停机。

可以用 NTM 模型描述的算法，称之为不确定性算法。

不确定性算法的非形式化定义：

设 A 是求解问题 Π 的一个算法，如果算法 A 以如下猜测+验证的方式工作，称算法 A 为不确定性(nondeterminism)算法：

- 猜测阶段：对问题的输入实例 x 产生一个任意字符串 y ，在算法的每次运行， y 可能不同，因此猜测是以不确定的形式工作。
- 验证阶段：在这个阶段，用一个确定性算法验证两件事：首先验证猜测的 y 是否是合适的形式，若不是，则算法停下并回答 no；若是合适形式，则继续检查它是否是 x 的解，如果确实是 x 的解，则停下并回答 yes，否则停下并回答 no。

如果猜测和检查都在输入实例的多项式时间完成，则称 A 为多项式时间不确定性算法。

注意：若 A 输出 no，并不意味着不存在一个满足要求的解，也就是说，并不意味着 A 不接受该输入，因为猜测可能不正确。

7.3.2 P 类问题

在计算复杂性理论中，P 类问题是指那些判定问题，其 yes/no 解可以通过确定性算法在多项式时间内求解。具体来说，对于一个判定问题 Π ，如果存在一个非负整数 k ，使得对于输入规模为 n 的实例，能够以 $O(n^k)$ 的时间运行确定性算法得到答案，则该问题属于 P(Polynomial)类问题。

对于一个判定问题，如果能构造出一个多项式求解算法，则可证明该问题属于 P 类。显然，所有易解问题都是 P 类问题。（例如：排序问题、不相交集问题、最短路径问题、2 着色问题、2 可满足问题）

如果对于任意问题 $\Pi \in C$ ， Π 的补也在 C 中，我们说问题类 C 在补运算下是封闭的。比如 2 着色问题的补就是不 2 可着色。P 类问题在补运算下是封闭的。

7.3.3 NP 类问题

定义 1 (NP 类问题的验证定义)：判定问题类 NP 由这样的问题 Π 组成：对于这些问题存在一个确定性算法 A ，该算法在对 Π 的一个实例展示一个断言解时，能在多项式时间内验证解的正确性。具体而言，如果断言解导致答案是 yes，就存在一种方法可以在多项式时间内验证这个解。

定义 2 (NP 类问题的非确定性计算定义): 判定问题类 NP 由这样的判定问题组成: 对于它们存在着多项式时间内运行的不确定性算法。形式化地说, 如果对于判定问题 Π , 存在一个非负整数 k , 对于输入规模为 n 的实例, 能够以 $O(n^k)$ 的时间运行一个不确定性算法, 得到 yes/no 的答案, 则该判定问题 Π 是一个 NP (nondeterministic polynomial) 类问题。

NP 类问题是否对补运算封闭, 即 $NP? = co - NP$ 尚未可知。

P 类问题是易求解的, NP 类问题是易验证的, 显然, $P \subseteq NP$ 目前仍然无法判断是否有或没有 $P = NP$, 但是因为 P 在补运算下封闭, 所以有 $P \subseteq co - NP$ 。

7.4 多项式时间规约和 NP 完全性

NP 完全性 (NP-complete) 是 NP 类的一个特殊子类, 这类判定问题具有以下重要特性: 如果其中任何一个问题被证明存在多项式时间的确定性算法解, 那么 NP 类中的所有问题都将能够通过多项式时间的确定性算法求解。这意味着, 若存在这样的算法, 将直接导致 $NP = P$ 的结论成立。

在计算复杂性理论中, 问题归约是一个核心概念。具体来说, 当一个问题 A 可以归约为问题 B 时, 这意味着我们可以利用解决 B 问题的方法来直接解决 A 问题, 或者说问题 A 能够被"转化"为问题 B 的形式进行处理。

从直观上看, 这种归约关系揭示了两个问题之间的复杂度比较: 若 A 可归约为 B, 则表明 B 的时间复杂度至少与 A 相当, 或者说 B 的求解难度不低于 A。换句话说, 问题 A 的难度不会超过问题 B 的难度, 因为只要能解决 B, 就必然能解决 A。

定义: 设判定问题 $\Pi_1 = \langle D_1, Y_1 \rangle$ 和 $\Pi_2 = \langle D_2, Y_2 \rangle$, 如果存在一个函数 $f: D_1 \rightarrow D_2$ 满足如下条件:

- (1) f 是多项式时间可计算的, 即存在计算 f 的多项式时间算法;
- (2) 对所有 $I \in D_1$, $I \in Y_1 \Leftrightarrow f(I) \in Y_2$

则称 f 是 Π_1 到 Π_2 的多项式时间归约函数。计算 f 的多项式时间算法 F 称为归约算法。

定义: 设判定问题 Π_1 和 Π_2 , 如果存在 Π_1 到 Π_2 的多项式时间归约算法, 则称 Π_1 可多项式时间归约到 Π_2 , 记作 $\Pi_1 \underset{\text{poly}}{\propto} \Pi_2$ 。

Proof. 哈密顿回路问题(HC)可多项式归约到货郎担问题(TSP), 即 $HC \underset{\text{poly}}{\propto} TSP$ 。

1. 问题定义:

- 设 $HC = \langle D_1, Y_1 \rangle$, 其中 D_1 为所有无向图的集合, Y_1 为包含哈密顿回路的图
- 设 $TSP = \langle D_2, Y_2 \rangle$, 其中 D_2 为所有带权完全图与界限值的组合, Y_2 为存在长度不超过 K 的周游路线的实例

2. **构造归约函数**: 定义函数 $f: D_1 \rightarrow D_2$, 对任意 $I \in D_1$ (无向图 $G = \langle V, E \rangle$):

$$f(I) = (G', K)$$

其中:

- G' 为顶点集 V 上的完全图, $E' = \{(u, v) \mid u, v \in V\}$
- 边权赋值规则:

$$d(u, v) = \begin{cases} 1, & \text{若 } (u, v) \in E \\ 2, & \text{若 } (u, v) \notin E \end{cases}$$

- 界限值 $K = |V|$

显然 f 是多项式时间可计算的。

3. **等价性证明**: 对任意周游路线 (经过每个顶点恰好一次):

- 包含 $|V|$ 条边, 每条边权值为 1 或 2
- 路线长度 L 满足 $|V| \leq L \leq 2|V|$

因此有:

$$\begin{aligned} f(I) \in Y_2 &\Leftrightarrow \exists \text{ 周游路线长度 } L \leq K = |V| \\ &\Leftrightarrow L = |V| \quad (\text{因 } L \geq |V|) \\ &\Leftrightarrow \text{路线中所有边权均为 } 1 \\ &\Leftrightarrow \text{该路线所有边都属于 } E \\ &\Leftrightarrow \text{该路线是 } G \text{ 中的哈密顿回路} \\ &\Leftrightarrow I \in Y_1 \end{aligned}$$

综上, $I \in Y_1 \Leftrightarrow f(I) \in Y_2$ 且 f 为多项式时间可计算, 故 $HC \propto_{\text{poly}} TSP$ 得证。 ■

定理: 如果 $\Pi_1 \propto_{\text{poly}} \Pi_2$, 则 $\Pi_2 \in P$ 蕴含 $\Pi_1 \in P$; Π_1 是难解的蕴含 Π_2 是难解的。

也即表明:

定理: 如果 $\Pi_1 \propto_{\text{poly}} \Pi_2$, 则相对于多项式时间, 如果 Π_2 是易解的, 则 Π_1 也是易解的, 即 Π_2 不会比 Π_1 更容易。或者反过来讲, 即 Π_1 不会比 Π_2 更难。

另外, 也容易证明多项式归约具有传递性:

定理：设 Π_1 、 Π_2 和 Π_3 是三个判定问题，且有 $\Pi_1 \underset{\text{poly}}{\propto} \Pi_2$ ， $\Pi_2 \underset{\text{poly}}{\propto} \Pi_3$ ，则 $\Pi_1 \underset{\text{poly}}{\propto} \Pi_3$ 。

7.5 NP 完全性

定义：对于一个判定问题 Π ，如果对所有 $\Pi' \in NP$ ，都有 $\Pi' \underset{\text{poly}}{\propto} \Pi$ ，则称 Π 是 NP 难的（NP-hard, NPH）。

含义：NPH 问题不会比 NP 类中的任何问题容易。

定义：对于一个判定问题 $\Pi \in NP$ ，如果对所有 $\Pi' \in NP$ ，都有 $\Pi' \underset{\text{poly}}{\propto} \Pi$ ，则称 Π 是 NP 完全的（NP-complete, NPC）。

含义：NPC 问题是 NP 类中的 NPH 问题。由此也可以推断，NPC 问题是 NP 类中最难的问题。NP 完全性是判定 P 是否等于 NP 的关键。

NPH 问题和 NPC 问题的唯一差别是：NPC 问题必须是属于 NP 类的问题，而 NPH 问题不一定在 NP 类中。

7.6 Cook-Levin 定理

库克-列文定理。

定理（NP 困难的传递性）：如果存在 $\Pi' \in NPH$ ，使 $\Pi' \overset{\text{poly}}{\rightarrow} \Pi$ ，则 Π 是 NPH 的。

定理（NP 完全性的传递性）：如果 $\Pi \in NP$ ，且存在 $\Pi' \in NPC$ ，使得 $\Pi' \overset{\text{poly}}{\rightarrow} \Pi$ ，则 Π 是 NPC 的。

上述定理表明，如果要证明一个问题 Π 是属于 NPH（NPC）的，不再需要证明所有 NP 问题都多项式时间可归约到 Π ，而是只需要证明存在一个 NPH（NPC）的问题 Π' ， Π' 多项式时间可归约到 Π 即可。

因此，要证明一个问题 Π 是属于 NPC 的，现在只需要证明两点：（1） $\Pi \in NP$ ；（2）存在一个 $\Pi' \in NPC$ ，使得 $\Pi' \overset{\text{poly}}{\rightarrow} \Pi$ 。

可满足性问题是第一个被证明的 NPC 问题，SAT 问题是指：是否存在一组对所有布尔变量的赋值，使得整个合取范式为真。库克列文定理表明，SAT 问题是 NPC 的。

证明一个问题 P_i 是 NPC 问题需要 2 步进行：

1. 证明问题是 NP 类问题。
2. 证明存在一个 NPC 问题 Π' ，使得其可以规约到问题 Π

已知 HC 问题是 NPC 问题，证明 TSP 问题也是 NPC 问题。

Proof. (1) $TSP \in NP$

可设计如下多项式不确定性运行算法: 对任意给定的一个边带权的完全无向图 G 和正整数 K , 随机猜测 G 中所有顶点的一个排列代表一条恰好经过每个城市一次的周游路线, 然后检查这个排列相邻的两个顶点之间边以及排列的首尾顶点之间边的总权值之和是否不超过 K 。如果是, 则回答“Yes”, 否则回答“No”。当 G 中存在长度不超过 K 的周游路线时, 总能猜对一个顶点的排列, 它确实给出一条长度不超过 K 的周游路线, 从而使算法回答“Yes”; 如果 G 中不存在长度不超过 K 的周游路线, 则猜测的任何排列表示的周游路线长度都会大于等于 K , 从而算法总是回答“No”。显然, 猜测一个排列和检查这个排列对应的周游路线的长度是否不超过 K 都可以在多项式时间内完成。因此, TSP 是 NP 类问题。任何一个构成长度不超过 K 的周游路线的顶点排列都是证据。

(2) $HC \leq_{poly} TSP$, 前面已证。 ■

7.6.1 限制法

证明 Π 包含一个已知为 NPC 问题 Π' 作为它的一种特殊情况。

最大可满足性 (Max-SAT) 问题: 任给关于 n 个变元 $x_1, x_2, \dots, x_n$ 的 m 个简单析取式 $C_1, C_2, \dots, C_m$ 以及正整数 K , 问是否存在关于这些变元的赋值 t 使得 $C_1, C_2, \dots, C_m$ 中至少有 K 个为真?

对比 Max-SAT 问题和 SAT 问题, 很容易看出 SAT 是 Max-SAT 问题限定 $K = m$ 的特殊情况。

当 $K = m$ 时, 赋值 t 使 $C_1, C_2, \dots, C_m$ 中至少有 K 个为真就是使 $C_1, C_2, \dots, C_m$ 全部为真, 这当且仅当赋值 t 使 $F = C_1 \wedge C_2 \wedge \dots \wedge C_m$ 为真。

因此当 $K = m$ 时, Max-SAT 问题就退化为 SAT 问题。即 SAT 可以看作是 Max-SAT 的子问题。

7.6.2 局部替换法

一般来讲, 如果已知 $\Pi \in NPC$, Π' 是 Π 在限定一定条件下的子问题, 或者更一般情况, Π' 与 Π 的问题实例的构造有某方面的相似之处时, 往往可以采用局部替换法来证明 Π' 的 NP 完全性。

局部替换法是选择已知 NP 完全问题的实例中的某些元素作为基本单元, 然后把每个基本单元替换成指定结构, 从而得到目标问题的对应实例。

证明三元可满足性问题(3SAT)是 NPC 问题。

显然 3SAT 问题是 SAT 问题的子问题。

对每一个 $I \in SAT$, I 是关于变元 $x_1, x_2, \dots, x_n$ 的合取范式 $F = C_1 \wedge C_2 \wedge \dots \wedge C_m$, 其中 $C_1, C_2, \dots, C_m$ 是简单析取式。

设计转换函数 f , 使得 $f(F) = F' = F'_1 \wedge F'_2 \wedge \cdots \wedge F'_m$, 其中 F'_i 是对应于 C_i 的三元合取范式, 且 C_i 是可满足的当且仅当 F'_i 是可满足的。这样构造的 F' 显然满足 F 是可满足的当且仅当 F' 是可满足的。

根据 C_i 的四种可能情形, 三元合取范式 F'_i 分别构造如下:

1. C_i 仅包含单个文字, 设 $C_i = z_1$, 其中 z_1 为文字, 即某变元或者变元的否定 (下同)。

引入两个新的变元 y_{i1} 和 y_{i2} , 令

$$F'_i = (z_1 \vee y_{i1} \vee y_{i2}) \wedge (z_1 \vee \neg y_{i1} \vee y_{i2}) \wedge (z_1 \vee y_{i1} \vee \neg y_{i2}) \wedge (z_1 \vee \neg y_{i1} \vee \neg y_{i2})$$

显然, 无论怎么赋值, $y_{i1} \vee y_{i2}$ 、 $\neg y_{i1} \vee y_{i2}$ 、 $y_{i1} \vee \neg y_{i2}$ 和 $\neg y_{i1} \vee \neg y_{i2}$ 不可能同时为真, 因此 F'_i 为真当且仅当 z_1 为真, 也即 F'_i 为真当且仅当 C_i 为真。

2. C_i 是两个文字的析取式, 设 $C_i = z_1 \vee z_2$, 其中 z_1 、 z_2 均为文字。引入一个新的变元 y_i , 令

$$F'_i = (z_1 \vee z_2 \vee y_i) \wedge (z_1 \vee z_2 \vee \neg y_i)$$

显然, 无论怎么赋值, y_i 和 $\neg y_i$ 不可能同时为真, 因此 F'_i 为真当且仅当 $z_1 \vee z_2$ 为真, 也即 F'_i 为真当且仅当 C_i 为真。

3. C_i 是三个文字的析取式, 设 $C_i = z_1 \vee z_2 \vee z_3$, 其中 z_1 、 z_2 、 z_3 均为文字。直接令 $F'_i = C_i = z_1 \vee z_2 \vee z_3$, 显然 F'_i 为真当且仅当 C_i 为真。
4. C_i 是三个以上文字的析取式, 设 $C_i = z_1 \vee z_2 \vee \cdots \vee z_k$, $k \geq 4$, 其中 z_1 、 z_2 、 $\dots$ 、 z_k 均为文字。

引入 $k-3$ 个新的变元 y_{i1} , y_{i2} , $\dots$, $y_{i(k-3)}$, 令

$$F'_i = (z_1 \vee y_{i1}) \wedge (\neg y_{i1} \vee z_3 \vee y_{i2}) \wedge (\neg y_{i2} \vee z_4 \vee y_{i3}) \wedge \cdots \wedge (\neg y_{i(k-4)} \vee z_{k-2} \vee y_{i(k-3)}) \wedge (\neg y_{i(k-3)} \vee z_{k-1} \vee z_k)$$

设由真值指派 t 满足 C_i , 则一定存在一个 $j (1 \leq j \leq k)$ 使得 $t(z_j) = 1$ 。将 t 扩展到对所有新变元的赋值上:

1. 如果 $j = 1$ 或者 2 时, 令 $t(y_{is}) = 0$, $1 \leq s \leq k-3$;
2. 如果 $j = k-1$ 或者 k 时, 令 $t(y_{is}) = 1$, $1 \leq s \leq k-3$;
3. 如果 $3 \leq j \leq k-2$ 时, 令

$$t(y_{is}) = \begin{cases} 1, & 1 \leq s \leq j-2 \\ 0, & j-1 \leq s \leq k-3 \end{cases}$$

不难验证, 扩展后的真值指派 t 也满足 F'_i 。因此, 如果 $t(C_i) = 1$, 则 $t(F'_i) = 1$ 。

其实证明到这里就足够了, 但是为了加强问题, 我们进一步证明等价性。

5. 反过来, 设 $t(F'_i) = 1$ 。如果 $t(y_{i1}) = 0$, 则必有 $t(z_1 \vee z_2) = 1$; 如果 $t(y_{i(k-3)}) = 1$, 则必有 $t(z_{k-1} \vee z_k) = 1$; 否则必有 $s (1 \leq s \leq k-3)$

4), 使得 $t(y_{is}) = 1$ 且 $t(y_{i(s+1)}) = 0$, 从而 $t(z_{s+2}) = 1$ (反证可得)。因此, 如果 $t(F'_i) = 1$, 则必存在 $j(1 \leq j \leq k)$ 使得 $t(z_j) = 1$, 从而如果 $t(C_i) = 1$ 。因此, F'_i 为真当且仅当 C_i 为真成立。

综合四种情况可得, F 是可满足的当且仅当 F' 是可满足的。另外, F'_i 为三元合取范式, 所包含的简单析取式的个数不会超过 C_i 中文字的4倍, 每个析取式由3个文字组成。因此, $f(F) = F'$ 可在 $|F|$ 的多项式时间内构造, 也就函数 f 是多项式时间可计算的。因此 f 是 SAT 到 3SAT 的多项式归约函数。

7.6.3 构件设计法

如果限制法和局部替换法都不起效果, 可以考虑构件设计法。基本思想是用目标问题的实例设计某些构件, 能够把这些构件组合起来“实现”已知的 NPC 问题实例。

对于下面3个问题, 容易证明他们是 NP 的。下面会逐一证明 SAT 问题可以规约到他们任意之一。

- 顶点覆盖: 给出一个无向图 $G = (V, E)$ 和一个正整数 k , 是否存在大小为 k 的子集 $C \subseteq V$, 使 E 中的每条边至少和 C 中的一个顶点关联?
- 独立集: 给出一个无向图 $G = (V, E)$ 和一个正整数 k , 是否存在 k 个顶点的子集 $S \subseteq V$, 使得对于每一对顶点 $u, w \in S$, $(u, w) \notin E$?
- 团集: 给出一个无向图 $G = (V, E)$ 和一个正整数 k , G 包含一个大小为 k 的团集吗?(注意一个 G 中大小为 k 的团集是 G 中 k 个顶点的一个完全子图。)

7.6.3.1 顶点覆盖问题是 NPC 问题

给定一个有 m 个子句和 n 个布尔变元 $x_1, x_2, \dots, x_n$ 的可满足性实例 $f = C_1 \wedge C_2 \wedge \dots \wedge C_m$, 按如下方式构造图 G :

1. 对于 f 中的每一个布尔变元 x_i , G 包含一对顶点 x_i 和 $\neg x_i$, 它们有一条边相连;
2. 对于每个子句 C_j 包含的 n_j 个文字, G 包含一个大小为 n_j 的团集 C_j ;
3. 对于在 C_j 中的每个顶点 w , 有一条边连接 w 到1中构造的顶点对 $(x_i, \neg x_i)$ 中它相应的文字。这些边称为连通边;
4. 令 $k = n + \sum_{j=1}^m (n_j - 1)$ 。

显然, 上述构造图 G 的方法可在多项式时间内完成。

引理: f 是可满足的当且仅当构造的图 G 有一个大小为 k 的顶点覆盖集。

证明:

($\Rightarrow$ 方向)

1. 考察 f 中每个文字的赋值: 如果 x_i 为真, 则将步骤(1)构造的对应顶点 x_i 加到顶点覆盖集中; 否则, 将 $\neg x_i$ 对应顶点加入到顶点覆盖集中; 因此在(1)中所有边也已经被覆盖。
2. 由于 f 是可满足的, 因此每个子句中至少有1个文字赋值为真。在步骤(2)构造的每个子句对应的团集 C_j 中与赋值为真的文字对应的顶点 u , 同(1)中对应的顶点 v 之间的连通边 (u, v) 均已经被覆盖(因为 v 已经在覆盖集中)。
3. 将每个团集 C_j 中除了顶点 u 之外的其余 $n_j - 1$ 个顶点也加入顶点覆盖集中, 显然这些顶点能覆盖团集中所有边。
4. 显然, 顶点覆盖集的大小为 k 。因此, 如果 f 是可满足的, 则图 G 有一个大小为 k 的顶点覆盖。

($\Leftarrow$ 方向)

1. 假设图 G 有一个大小为 k 的顶点覆盖集。
2. 显然在步骤(1)构造的每个边 $(x_i, \neg x_i)$ 中至少有1个顶点必须在覆盖集中。因此可知已有 n 个顶点在覆盖集中, 同时, 这些顶点为端点的连通边也由这些顶点所覆盖了。
3. 考虑剩余的 $(n_1 - 1) + (n_2 - 1) + \dots + (n_m - 1)$ 个顶点。不难看出, 对于一个大小为 n_j 的团集, 顶点覆盖集的大小为 $n_j - 1$ 。因此, 对每个团集, 由于连通边除了与连通边有关的顶点外, 其余顶点都包含在顶点覆盖中。
4. 对于 f 中的每个变元 x_i , 如果它对应的顶点在覆盖集中, 则令 $x_i = \text{true}$, 否则, 令 $x_i = \text{false}$ 。这样, 在每个子句对应的团集中, 一定有一个顶点连接 x_i 或 $\neg x_i$ 对应赋值为真的顶点, 才能使连通边被覆盖。
5. 因此, 每个子句中至少有1个文字的赋值为真, 即 f 是可满足的。

7.6.3.2 独立集问题是 NPC 问题

引理: 设 $G = (V, E)$ 是连通无向图, 那么 $S \subseteq V$ 是一个独立集当且仅当 $V - S$ 是 G 的一个顶点覆盖。

证明: 设 $e = (u, v)$ 是 G 中的任意边:

1. S 是独立集 $\Leftrightarrow u$ 和 v 不同时在 S 中
2. $\Leftrightarrow u$ 或 v 至少有一个在 $V - S$ 中
3. $\Leftrightarrow V - S$ 覆盖了边 e
4. 由于 e 是任意边, 因此 $V - S$ 是 G 的顶点覆盖

7.6.3.3 团集问题是 NPC 问题

给定一个有 m 个子句和 n 个布尔变元 $x_1, x_2, \dots, x_n$ 的可满足性实例 $f = C_1 \wedge C_2 \wedge \dots \wedge C_m$ ，构造一个图 $G = (V, E)$ ，其中： - V 是 $2n$ 个文字的所有出现的集合（注意：一个文字是一个布尔变元或它的否定） - $E = \{(x_i, x_j) \mid x_i \text{ 和 } x_j \text{ 位于两个不同的子句，并且 } x_i \neq \neg x_j\}$

图 G 的每个顶点对应于 f 中的每个文字，多次出现的重复表示。若图 G 中两个顶点对应的文字不互补，且不出现在同一子句中，则将其连线。（节点 $a-b$ 连线意味着文字 a 和 b 可同时为真）

引理： f 是可满足的当且仅当 G 有一个大小为 m 的团集。

证明：设 f 有 m 个子句，则有： f 可满足 $\Leftrightarrow m$ 个子句同时为真 $\Leftrightarrow$ 每个子句至少 1 个文字为真 $\Leftrightarrow G$ 中有 m 个顶点之间彼此相连 $\Leftrightarrow G$ 中有 m 个顶点的团集

7.7 本章作业

7.7.1 平面加权图的最小生成树问题的计算复杂度

设平面图 $G = (V, E)$ ， E 中的边 (i, j) 的权值为两个点 i 和 j 之间的距离。
 $|V| = n$ 。计算 G 的一棵最小生成树。

证明，求解该问题的算法类的时间复杂度下界为 $\Omega(n \log n)$ 。

规约证明. 将排序问题规约到最小生成树问题：

1. **输入转换**：给定待排序的 n 个实数 $\{x_1, x_2, \dots, x_n\}$ ，构造平面图 $G = (V, E)$ ：

- 顶点集 $V = \{v_1, \dots, v_n\}$ ，其中每个 v_i 对应实数 x_i
- 边集 $E = \{(v_i, v_j) \mid \forall i \neq j\}$ （完全图）
- 边权定义为 $w(v_i, v_j) = |x_i - x_j|$

2. **MST 的性质**：在如此构造的图中，最小生成树必定是：

$$\text{MST} = \{(v_{(1)}, v_{(2)}), (v_{(2)}, v_{(3)}), \dots, (v_{(n-1)}, v_{(n)})\}$$

其中 $v_{(i)}$ 对应排序后的第 i 小元素（即 $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$ ）。

3. **规约的正确性**：

1. 从 MST 的边可以立即读出排序结果：遍历 MST 边集即可得到有序序列
2. 该转换过程的时间复杂度为 $O(n^2)$ （构造完全图）

4. **复杂度下界**: 由于比较排序问题的下界为 $\Omega(n \log n)$, 而 MST 问题可以解排序问题, 因此:

$$\text{MST 问题的复杂度下界} \geq \text{排序问题的下界} = \Omega(n \log n)$$

特别地, 对于平面图, 该下界仍然成立 (因为规约构造的图可以是平面的)。■

7.7.2 元素唯一性问题

设 S 是由 n 个数构成的数组, 判断 S 中的元素是否唯一。如果唯一, 输出"Yes"; 否则输出"No"。

证明该判定问题的计算复杂度是 $\Theta(n \log n)$ 。

给定 n 维空间 E^n 中的集合:

$$W = \{(x_1, x_2, \dots, x_n) \in \mathbb{R}^n \mid x_i \neq x_j, \forall i \neq j\}$$

需先证明 W 可分解为 $n!$ 个互不相交的连通分量 W_π 。

对每个排列 $\pi \in S_n$ (S_n 为 n 阶对称群), 定义:

$$W_\pi = \{(x_1, \dots, x_n) \mid x_{\pi(1)} < x_{\pi(2)} < \dots < x_{\pi(n)}\}$$

每个 W_π 是连通的, 因为:

1. 作为开凸集: 由线性不等式组定义的解空间是凸的
2. 任意两点可通过直线路径连接: $\forall \mathbf{a}, \mathbf{b} \in W_\pi$, 有 $t\mathbf{a} + (1-t)\mathbf{b} \in W_\pi$ ($0 \leq t \leq 1$)

对任意两个不同排列 $\pi \neq \sigma$, 必有:

$$W_\pi \cap W_\sigma = \emptyset$$

因为:

- 存在至少一对指标 i, j 使得 $\pi^{-1}(i) < \pi^{-1}(j)$ 但 $\sigma^{-1}(i) > \sigma^{-1}(j)$
- 在 W_π 中要求 $x_i < x_j$, 而在 W_σ 中要求 $x_i > x_j$

所有 W_π 的并集等于 W :

$$\bigcup_{\pi \in S_n} W_\pi = W$$

因为:

- 对任意 $\mathbf{x} \in W$, 可通过排序得到唯一排列 π 使 $x_{\pi(1)} < \dots < x_{\pi(n)}$
- 故 $\mathbf{x} \in W_\pi$ 且不属于其他 W_σ

在 $\mathbb{R}^n$ 空间中:

- 每个超平面 $x_i = x_j$ 将空间分成两个半空间

- W_π 对应所有 $\binom{n}{2}$ 个不等式 $x_{\pi(i)} < x_{\pi(j)}$ ($i < j$) 的交集
- 这些不等式定义的区域是 $\mathbb{R}^n$ 中的开凸多面体
- 不同的严格全序排列共有 $n!$ 种
- 每个排列对应唯一的坐标顺序关系
- 因此连通分量数量为 $|S_n| = n!$

集合 W 被超平面 $\{x_i = x_j\}_{i \neq j}$ 划分为 $n!$ 个:

- 互不相交的
- 连通的
- 开凸的

分量, 每个分量对应一个唯一的排列顺序。

7.7.3 2SAT 问题

设计一个多项式时间确定算法求解 2 可满足性问题, 从而证明 2 可满足性问题是 P 类问题。

1. 构建蕴含图:

- 对于每个变量 x , 创建两个顶点: x 和 $\neg x$ 。
- 对于每个子句 $(a \vee b)$, 添加两条有向边:
 - $\neg a \rightarrow b$
 - $\neg b \rightarrow a$

2. 计算强连通分量 (SCC) :

- 使用 Kosaraju 算法或 Tarjan 算法找到图中的所有强连通分量。
- 强连通分量是指图中任意两个顶点相互可达的最大子图。

3. 检查可满足性:

- 对于每个变量 x , 检查 x 和 $\neg x$ 是否在同一个 SCC 中。
- 如果存在这样的 x , 则公式不可满足, 返回“不可满足”。
- 否则, 公式可满足, 返回“可满足”。

4. 构造满足的赋值 (可选) :

- 对 SCC 进行拓扑排序, 得到顺序 $S_1, S_2, \dots, S_k$ 。
- 初始化所有变量为未赋值。
- 按 $i = k$ downto 1 的顺序处理每个 SCC:
 - 如果 S_i 中的任何文字未被赋值:
 - 将 S_i 中的所有文字赋值为真。

- 将它们的否定赋值为假。
- 返回得到的赋值。

Algorithm TARJAN(G) $index \leftarrow 0$ $S \leftarrow \emptyset$ $SCCs \leftarrow \emptyset$
return ($SCCs$)

Function STRONGCONNECT(v) $index[v] \leftarrow index$ $low[v] \leftarrow index$ $index \leftarrow index + 1$ $S.push(v)$ $onStack[v] \leftarrow true$

7.7.4 2 着色问题

设计一个多项式时间的确定算法求解 2 着色问题，从而证明 2 着色问题是 P 类问题。

Algorithm TWO-COLORING(G)

$queue \leftarrow$ 空队列 任选顶点 $s \in V$ $color[s] \leftarrow$ 红色
 $queue.enqueue(s)$
return $color$

7.7.4.1 正确性证明

算法基于以下观察：

- 图是二分图当且仅当可以用两种颜色着色且相邻节点颜色不同
- 使用 BFS 遍历时，若发现相邻节点颜色相同则立即终止
- 颜色冲突检测保证了算法的正确性

7.7.4.2 时间复杂度分析

- 初始化阶段： $O(|V|)$
- BFS 遍历： $O(|V| + |E|)$
- 总时间复杂度： $O(|V| + |E|)$

由于：

- 算法在多项式时间 $O(|V| + |E|)$ 内终止
- 使用确定性图遍历（BFS/DFS）
- 不需要猜测和验证步骤

因此 2-着色问题属于 P 类问题。

7.7.5 规约问题的传递性证明

对判定问题 Π_1 和 Π_2 ，证明如下结论：

如果 $\Pi_1 \propto_{poly} \Pi_2$ ，则 $\Pi_2 \in P$ 蕴含 $\Pi_1 \in P$ ； Π_1 是难解的蕴含 Π_2 是难解的。

Proof. 设 Π_1 和 Π_2 为形式语言，证明分为两部分：

第一部分：正向蕴含

1. 已知 $\Pi_1 \propto_{poly} \Pi_2$ ，即存在多项式时间可计算函数 f ，使得

$$\forall x \in \Sigma^*, x \in \Pi_1 \Leftrightarrow f(x) \in \Pi_2$$

2. 设 $\Pi_2 \in P$ ，则存在多项式时间算法 A_2 判定 Π_2
3. 构造 Π_1 的判定算法 A_1 ：
 - 输入 x ，计算 $f(x)$ （耗时 $p(|x|)$ ， p 为多项式）
 - 运行 $A_2(f(x))$ （耗时 $q(|f(x)|)$ ， q 为多项式）
 - 输出 A_2 的结果
4. 总时间复杂度为 $p(|x|) + q(|f(x)|)$ ，仍为多项式时间

第二部分：逆否命题

1. 原命题的逆否命题为：若 $\Pi_2 \notin \text{难解类}$ ，则 $\Pi_1 \notin \text{难解类}$
2. 假设 Π_2 不是难解的（即 $\Pi_2 \in P$ ）
3. 由第一部分结论直接可得 $\Pi_1 \in P$ ，故 Π_1 不是难解的
4. 特别地，当"难解类"取 P 的补集时：

$$\Pi_2 \notin \bar{P} \Rightarrow \Pi_1 \notin \bar{P}$$

$$\text{即 } \Pi_2 \in P \Rightarrow \Pi_1 \in P$$

综上得证：多项式时间规约具有保持问题复杂度的性质。 ■

7.7.6 集合覆盖问题是 NPC 问题

集合覆盖问题：给出集合 $S = s_1, s_2, \dots, s_n$ ，和它的子集簇 $C = c_1, c_2, \dots, c_m$ ，其中 $\forall c_i, c_j \subseteq S$ ，给定正整数 k ，是否存在 C 的大小不超过 k 的子集 C' ， C' 中的集合的并是 S ？

顶点覆盖问题：给出一个无向图 $G = (V, E)$ ，和一个正整数 k ，是否存在大小不超过 k 的顶点子集 $C \subseteq V$ ，使 E 中的每条边至少和 C 中的一个顶点关联？

请证明：顶点覆盖问题 $\propto_{poly}$ 集合覆盖问题。

通过将顶点覆盖问题 (Vertex Cover) 规约到集合覆盖问题 (Set Cover), 证明集合覆盖问题属于 NPC 类。

7.7.6.1 规约构造

给定顶点覆盖实例 $\langle G = (V, E), k \rangle$, 构造集合覆盖实例 $\langle S, C, k' \rangle$ 如下:

1. 令全集 $S = E$, 即所有边构成的集合
2. 对每个顶点 $v \in V$, 创建子集 $s_v = \{e \in E \mid \text{边 } e \text{ 与顶点 } v \text{ 关联}\}$, 构成子集簇 $C = \{s_v \mid v \in V\}$
3. 保持参数不变, 设 $k' = k$

7.7.6.2 正确性证明

1. 正向证明 (Vertex Cover $\Rightarrow$ Set Cover) :

- 设存在大小为 $j \leq k$ 的顶点覆盖 $V' = \{v_1, \dots, v_j\}$
- 构造对应的子集簇 $C' = \{s_{v_1}, \dots, s_{v_j}\}$
- 对任意边 $e = (u, w) \in E$:
 - 由顶点覆盖性质, u 或 w 至少一个在 V' 中
 - 若 $u \in V'$, 则 $e \in s_u \in C'$
- 故 C' 覆盖 S , 且 $|C'| = j \leq k$

2. 反向证明 (Set Cover $\Rightarrow$ Vertex Cover) :

- 设存在大小为 $j \leq k$ 的集合覆盖 $C' = \{s_{v_1}, \dots, s_{v_j}\}$
- 构造对应的顶点集 $V' = \{v_1, \dots, v_j\}$
- 对任意边 $e = (u, w) \in E$:
 - 存在 $s_v \in C'$ 包含 e
 - 故 e 至少与 $v \in V'$ 关联
- 故 V' 是顶点覆盖, 且 $|V'| = j \leq k$

3. 时间复杂度分析:

- 构造全集 S : $O(|E|)$
- 构建子集簇 C : $O(|V| \cdot \deg(v)) = O(|V| |E|)$
- 总时间复杂度为多项式时间 $O(|V| |E|)$

7.7.6.3 结论

通过上述多项式时间规约，且已知顶点覆盖问题是 NP 完全问题，可证明集合覆盖问题属于 NPC 类。

7.7.7 0-1 背包问题

0-1 背包问题的动态规划算法是一个多项式时间算法吗？解释你的答案。

0-1 背包问题的动态规划算法不是一个多项式时间算法。该算法的运行时间为 $O(nW)$ ，其中 n 是商品数量， W 是背包能容纳的最大商品总重量。 W 的二进制编码长度为 $l = \lceil \lg W \rceil$ ，该算法的运行时间为 $O(n2^l)$ ，随着 l 的增加， $O(n2^l)$ 呈指数级增长。我们通常称 0-1 背包问题为一个伪多项式时间算法⁺(pseudo-polynomial time algorithm)。

7.7.8 子例程时间影响

证明：若一个算法对多项式时间子例程的调用次数为常数，并且每次执行的额外工作量也需要多项式时间，则该算法的运行需要多项式时间。若多项式时间子程序的调用次数修改为多项式，则该算法的运行可能需要指数时间。

首先证明第一个命题，假设输入规模为 n ，调用 c 次多项式时间子例程，其中 c 为常数，因为每次迭代的输出输入为下一次迭代的输入。设第 i 次迭代多项式时间子例程的运行时间为 $O(n^{k_i})$ ，其中 k_i 为常数，多项式时间的额外工作量为 $O(n^{l_i})$ ，其中 l_i 为常数，第一次迭代的运行时间为 $T_1(n) = O(n^{k_1}) + O(n^{l_1}) = O(n^{\max\{k_1, l_1\}})$ 为多项式时间，输出规模为 $n_1 = O(T_1(n)) = O(n^{\max\{k_1, l_1\}})$ ，第二次迭代的运行时间为

$$T_2(n) = O(n_1^{\max\{k_2, l_2\}}) = O((n^{\max\{k_1, l_1\}})^{\max\{k_2, l_2\}}) = O(n^{\max\{k_1, l_1\} \max\{k_2, l_2\}})$$

为多项式时间，输出规模为 $n_2 = O(T_2(n)) = O(n^{\max\{k_1, l_1\} \max\{k_2, l_2\}})$ ，采用归纳法，第 i 次迭代的运行时间 $T_i(n) = O(n^{\prod_{j=1}^i \max\{k_j, l_j\}})$ 为多项式时间，输出规模为

$$n_i = O(T_i(n)) = O(n^{\prod_{j=1}^i \max\{k_j, l_j\}})$$

总的运行时间为

$$\sum_{i=1}^c T_i(n) \leq c \max\{T_1(n), T_2(n), \dots, T_c(n)\}$$

为多项式时间。

然后证明第二个命题，举例如下，假设开始输入规模为 1，每次迭代输出规模为输入规模的两倍， n 次迭代后输出规模为 2^n ，输出规模与运行时间多项式相关，该算法的运行时间为指数时间。证明完毕。

AlgoAnalysis

作者姓名

2026 年 06 月 16 日

8 chapter8: 回溯法

8.1 回溯法基本概念

8.1.1 回溯法定义与三步骤

Definition 8.1.1

回溯法

回溯法 (backtracking) 是一种系统地搜索问题解的方法。回溯法 = 深度优先搜索 + 剪枝。

回溯法求解的三个步骤：

1. **定义解空间**，它包含问题的解
2. **用易于搜索的方式组织解空间** (状态空间树)
3. **深度优先搜索解空间**，获得问题的解

8.1.2 解空间与解表示

问题的解一般可表示为向量 (也称元组) 形式：

$$X = (x_1, x_2, \dots, x_n)$$

其中分量 x_i 的取值范围为有穷且可枚举的集合 S_i 。

Remark

每个 x_i 的所有可能取值的组合构成的集合，称为问题的**解空间**。其中每一个取值组合对应问题实例的一个可能解。

两种约束类型：

- **显式约束**：限定每个 x_i 的取值范围 S_i (例如，N 皇后中 $S_i = \{1, 2, \dots, n\}$)
- **隐式约束**：规定 x_i 之间的相互关系 (例如，N 皇后中 $x_i \neq x_j$, $|x_i - x_j| \neq |i - j|$)

Example

0/1 背包问题， $n = 3$ 时： $S = \{0, 1\}$ ，解空间为：

$(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1), (1, 0, 0), (1, 0, 1), (1, 1, 0), (1, 1, 1)$ 共 $2^3 = 8$ 种可能的解。

8.1.3 状态空间树

解空间的树形式表示称为**状态空间树**。根据问题类型分为两种：

Definition 8.1.2

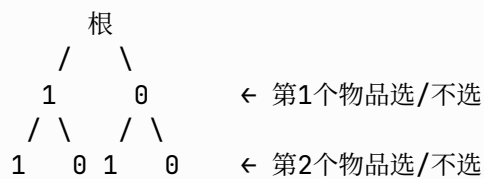
子集树

当所给的问题是从 n 个元素的集合 S 中选出满足某种性质的子集时，解空间树称为子集树。

- 每层 2 个分支（选/不选）
- 叶子数： 2^n
- 典型问题：0/1 背包、子集和

TEXT

TEXT



Definition 8.1.3

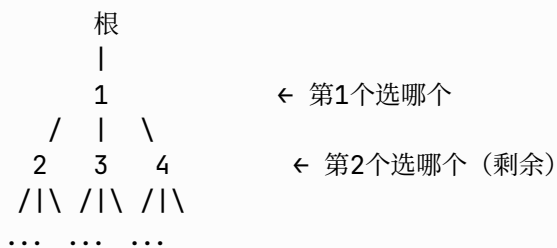
排列树

当所给的问题是确定 n 个元素满足某种性质的排列时，解空间树称为排列树。

- 第 i 层有 $(n - i + 1)$ 个分支（逐层递减）
- 叶子数： $n!$
- 典型问题：旅行商（TSP）、N 皇后（带列约束）、流水作业调度

TEXT

TEXT



8.1.4 解长度类型

- **定长解**：解向量长度固定为 n
 - 3-着色： n 个顶点; 长度 n
 - N 皇后： n 行; 长度 n
 - 0/1 背包： n 个物品; 长度 n
- **变长解**：不同解可能有不同长度

- ▶ 子集和: $\{10, 20, 30\}$ (3 个元素), $\{60\}$ (1 个元素)

变长解可通过改变表示方式转为定长解:

1. 用 n 元布尔向量表示: $x_i = 1$ 表示第 i 个元素被选中
2. $\{10, 20, 30\}$; $(1, 1, 1, 0, 0, 0)$

Remark

为什么要转定长表示? 回溯框架基于定长 n -元组搜索 (k 从 1 递增到 n), 变长解用不定长表示时终止条件不统一, 转成定长布尔向量后统一用子集树框架处理。

8.2 回溯法的基本思想

8.2.1 词典序搜索与三种情况

回溯算法按**词典序** (lexicographic order) 考虑笛卡儿积 $X_1 \times X_2 \times \dots \times X_n$ 的元素。从空向量开始, 选 X_1 最小元素, 逐步深入, 到尽头则回溯。

一般地, 假设算法已检测到部分解 $(x_1, x_2, \dots, x_j)$, 考虑向量 $v = (x_1, x_2, \dots, x_j, x_{j+1})$:

处理方式有三种:

1. **情况 1**: 若 v 是最终解; 记录解, 终止或继续找下一个解
2. **情况 2**: 若 v 是部分解; **前进** ($k \leftarrow k + 1$), 继续选 x_{k+1}
3. **情况 3**: 若 v 既非最终解也非部分解
 - x_{j+1} 还有下一个取值; 取下一个值
 - x_{j+1} 无下一个取值; **回溯**: x_j 取下一个值, 或继续向上回溯

Remark

情况 3 是回溯法的核心——判断出当前路径不可行后, 不是从头再来, 而是回退到最近的一个能换选择的点, 换一条路继续。

8.2.2 结点类型、状态类型与搜索过程

Definition 8.2.1

结点类型 状态类型

三种结点:

- **活结点**: 已生成但儿子结点还未全部生成的结点
- **E-结点** (正在扩展的结点): 当前正在生成其儿子结点的活结点
- **死结点**: 不再进一步扩展或其儿子结点已全部生成的结点

三种状态:

- **问题状态**: 树中的每个结点
- **解状态**: 由根到该结点的路径确定了解空间中的一个元组
- **答案状态**: 由根到该结点的路径确定了问题的一个解

深度优先搜索的 4 步过程:

- 1. 确定解空间的组织结构后, 从根顶点出发, 以 DFS 方式搜索
- 2. 根顶点成为活结点和 E-结点, 向纵深方向移至新顶点
- 3. 若当前 E-结点无法再纵深移动; 成为死结点; 回溯至最近一个活结点
- 4. 重复直至找到解或活结点耗尽

8.3 回溯算法一般框架

8.3.1 递归框架 BACKTRACKREC

Algorithm 8.3.1 “BACKTRACKREC”, 递归回溯框架

输入: 集合 $X_1, X_2, \dots, X_n$ 的清楚的或隐含的描述 **输出:** 解向量 $v = (x_1, x_2, \dots, x_i), 0 \leq i \leq n$

TEXT

TEXT

```
1.  $v \leftarrow ()$ 
2.  $flag \leftarrow false$ 
3.  $advance(1)$ 
4. if  $flag$  then output  $v$ 
5. else output "no solution"

过程  $advance(k)$ 
1. for each  $x \in X_k$ 
2.    $x_k \leftarrow x$ ; 将  $x_k$  加入  $v$ 
3.   if  $v$  为最终解 then  $flag \leftarrow true$ ; exit
4.   else if  $v$  是部分解 then  $advance(k+1)$ 
5. end for
```

Remark

递归框架中的三种动作:

- 第 2 行 (选择): 将当前取值加入解向量 v
- 第 4 行 (前进): 部分解可行; 深入下一层
- 第 5 行 (回溯): `for` 循环结束; 自动回到上层

8.3.2 迭代框架 BACKTRACKITER

Algorithm 8.3.2 “BACKTRACKITER”, 迭代回溯框架

输入: 集合 $X_1, X_2, \dots, X_n$ 的清楚的或隐含的描述 **输出:** 解向量 $v = (x_1, x_2, \dots, x_i), 0 \leq i \leq n$

TEXT

TEXT

```
1.  $v \leftarrow ()$ 
2.  $flag \leftarrow false$ 
3.  $k \leftarrow 1$ 
4. while  $k \geq 1$ 
5.     while  $X_k$  没有被穷举
6.          $x_k \leftarrow X_k$  中的下一个元素; 将  $x_k$  加入  $v$ 
7.         if  $v$  为最终解 then  $flag \leftarrow true$ ; 跳出
8.         else if  $v$  是部分解 then  $k \leftarrow k+1$  // 前进
9.     end while
10.    重置  $X_k$ , 使得下一个元素排在第一位
11.     $k \leftarrow k-1$  // 回溯
12. end while
13. if  $flag$  then output  $v$ 
14. else output "no solution"
```

递归 vs 迭代对比:

操作	递归版	迭代版
前进	<code>advance(k+1)</code>	<code>k ← k+1</code>
回溯	<code>end for</code> 隐含回溯	<code>k ← k-1</code> 显式回溯
状态维护	函数调用栈	显式 k 指针

8.4 回溯法的本质

8.4.1 约束穷举

设 S_i 的大小为 m_i , 则共有 $m = m_1 \cdot m_2 \cdot \dots \cdot m_n$ 个 n -元组。

硬性处理（蛮力）：构造出 m 个 n -元组并逐一测试 $\rightarrow$ 工作量 $O(m)$

回溯处理：用限界函数 $P_{i(x_1, \dots, x_i)}$ 测试正在构造的部分向量；若判定不可行 $\rightarrow$ 略去 $m_{i+1} \cdot m_{i+2} \cdot \dots \cdot m_n$ 个向量 $\rightarrow$ 工作量远小于 $O(m)$

Remark

回溯法比硬性处理做的测试少得多。代价是每次测试需要更复杂的判断——不是单纯检查整个解，而是检查部分解的可扩展性。

8.4.2 多米诺性质

要使回溯法正确（不遗漏解），约束 P 必须满足：

$$P(x_1, x_2, \dots, x_{k+1}) \text{ 为真} \rightarrow P(x_1, x_2, \dots, x_k) \text{ 为真}$$

解的前缀具有继承性——部分解不可行 $\rightarrow$ 扩展后也一定不可行。

违反的后果：若 $P(1, 2) = \text{false}$ 但 $P(1, 2, 3) = \text{true}$ ，则回溯法在 $k = 2$ 时剪枝 $\rightarrow$ 漏掉 $(1, 2, 3)$ 这个解 $\rightarrow$ 算法不正确。

8.4.3 剪枝函数

Definition 8.4.1

剪枝函数

回溯法中用于避免无效搜索的判断函数，统称剪枝函数。分为两种：

- **约束函数（左剪枝）**：在扩展顶点处剪去不满足约束的子树。用于所有问题（判定和优化）。
例如 0/1 背包中 $t_w + w[i] > W \rightarrow$ 不选左分支。
- **限界函数（右剪枝）**：剪去不能得到最优解的子树。仅用于优化问题。例如 0/1 背包中 $\text{bound}(i) \leq \max_v \rightarrow$ 不选右分支。

8.5 回溯法经典应用

8.5.1 子集和问题

Problem 8.5.1

子集和问题

已知 n 个正数 $w_1, w_2, \dots, w_n$ 和整数 M ，找出所有和为 M 的子集。

例如： $w = [5, 10, 12, 13, 15, 18], M = 30$

解： $\{5, 10, 15\}, \{12, 18\}, \{30\}$ （若 $30 \in w$ ）

1. **解表示**：定长 n -元组 $x[1..n]$ ， $x_i \in \{0, 1\}$ 表示 w_i 是否被选中。
2. **解空间**：子集树， 2^n 个叶子。
3. **限界函数 B_k** ：当前考虑第 k 个元素时，记 s 为已选元素之和， r 为剩余元素之和（ w_k 到 w_n 的和）。
左分支（选 w_k ）： $s + w_k = M \rightarrow$ 找到解； $s + w_k + w_{k+1} \leq M \rightarrow$ 选入后即使再加最小的剩余也不会超，可以继续；否则选了这个后面的就都选不了了，绝对不可能到达 M
右分支（不选 w_k ）： $s + r - w_k \geq M$ 且 $s + w_{k+1} \leq M \rightarrow$ 不选 w_k 后后面还够得到 M ，且下一个要试的最小元素不会直接超，值得继续

Remark

排序的作用： w_i 必须按非降序排列，这样 w_{k+1} 才是剩余中的最小值。 $s + w_k + w_{k+1} \leq M$ 的判断才安全——如果最小的都超了，那选更大的更超。

4. 伪代码**Algorithm 8.5.2**

子集和问题的回溯解法

输入： 非降序数组 $w[1..n]$ ，目标和 M **输出：** 解向量 $x[1..n]$

PYTHON

PYTHON

```
def dfs(k, s, r):
    # 选 w[k]
    x[k] = 1
    if s + w[k] == M:
        output(x)
        return
    if s + w[k] + w[k+1] ≤ M:
        dfs(k+1, s + w[k], r - w[k])

    # 不选 w[k]
    if s + r - w[k] ≥ M and s + w[k+1] ≤ M:
        x[k] = 0
        dfs(k+1, s, r - w[k])

# 主调
s = 0
r = sum(w)
if r ≥ M:
    dfs(1, s, r)
```

5. 复杂度

- 时间：最坏 $O(2^n)$ （剪枝不改变最坏上界）
- 空间： $\Theta(n)$ （递归深度 n ）

8.5.2 哈密顿回路问题**Problem 8.5.3**

哈密顿回路

给定一个无向图 $G = (V, E)$ ， n 个顶点，找一条经过每个顶点恰好一次、最后回到起点的回路。

1. **解表示：** 排列 $x[1..n]$ ， $x[k]$ 表示路径上第 k 个顶点，固定 $x[1] = 1$ 。
2. **解空间：** 排列树， $(n-1)!$ 个叶子。

3. 约束条件:

1. $(x[k-1], x[k]) \in E$ (相邻顶点必须邻接)
2. 路径上顶点不能重复 (`visited` 数组维护)
3. $(x[n], x[1]) \in E$ (首尾相连)

4. 伪代码

Algorithm 8.5.4

哈密顿回路问题的回溯解法

输入: 邻接矩阵 `graph[n+1][n+1]` (1-indexed) 输出: 回路 `x[1..n]`

PYTHON

PYTHON

```
def dfs(k):
    if k == n + 1:
        if graph[x[n]][x[1]]:      # 首尾相连
            output(x)              # 找到回路
        return
    for v in range(2, n + 1):      # 枚举剩余顶点
        if not visited[v] and graph[x[k-1]][v]:
            x[k] = v
            visited[v] = True
            dfs(k + 1)
            visited[v] = False      # 回溯
```

5. 复杂度

- 时间: 最坏 $O(n!)$
- 空间: $\Theta(n)$ (递归深度 $n + \text{visited}$ 数组)

8.5.3 0/1 背包问题

Problem 8.5.5

0/1 背包问题

有 n 个物品, 第 i 个物品重量 w_i 、价值 v_i 。给定容量 W 的背包。每个物品要么选中要么不选, 求在总重量不超过 W 的前提下总价值最大的装入方案。

1. 解表示: 定长 n -元组 $x[1..n]$, $x_i \in \{0, 1\}$ 表示第 i 个物品是否装入。

2. 解空间: 子集树, 2^n 个叶子。

3. 剪枝策略:

1. 左剪枝 (约束函数): $t_w + w_i \leq W$ 才尝试选第 i 个物品, 否则超重
2. 右剪枝 (限界函数): $\text{bound}(i) > \max_v$ 才尝试不选第 i 个物品, 其中 $\text{bound}(i) = t_v +$ 剩余物品按单位价值贪心塞满背包的乐观估计。`bound()` 不是算法的解, 它是一个估算函数。它解决的问题是: 在当前节点, 如果我不选物品 i , 后面在不违反约束的前提下, 最多还能拿多少价值? 若乐观估计都不如当前最优解, 那这条路不可能更优, 直接剪掉。

Remark

物品需按 $\frac{v_i}{w_i}$ 降序排列, **bound** 才能给出紧的上界。

4. 伪代码**Algorithm 8.5.6****0/1 背包问题的回溯解法**

输入: 物品列表 $A[1..n]$ (按 $\frac{v_i}{w_i}$ 降序), 容量 W **输出:** 最优价值 $\max_v$ 及对应方案 $x[1..n]$

PYTHON

PYTHON

```
def bound(i, tw, tv):
    # 分数背包松弛: 乐观估计剩余价值上界
    j = i + 1
    while j ≤ n and tw + A[j].w ≤ W:
        tw += A[j].w
        tv += A[j].v
        j += 1
    if j ≤ n:
        tv += (W - tw) * A[j].p    # 装不下整个, 切一部分
    return tv

def dfs(i, tw, tv):
    if i > n:                        # 所有物品决策完毕
        if tv > maxv:
            maxv = tv
            copy(x, bestx)
        return
    # 选第 i 个
    if tw + A[i].w ≤ W:              # 左剪枝
        x[i] = 1
        dfs(i + 1, tw + A[i].w, tv + A[i].v)
    # 不选第 i 个
    if bound(i, tw, tv) > maxv:      # 右剪枝
        x[i] = 0
        dfs(i + 1, tw, tv)
```

5. 复杂度

- 时间: 最坏 $O(2^n)$ (剪枝不改变最坏上界)
- 空间: $\Theta(n)$

8.5.4 流水作业调度问题

Problem 8.5.7

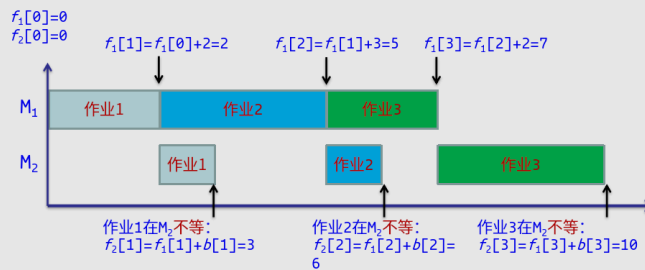
流水作业调度

有 n 个作业，每个作业必须先 在 M_1 上加工 a_i 时间，再到 M_2 上加工 b_i 时间。求作业的加工顺序，使得从第一个作业在 M_1 上开始到最后一个作业在 M_2 上完成的总时间最少。

一个示例，假设有 3 个作业：

作业编号	1	2	3
M_1 时间 a	2	3	2
M_2 时间 b	1	1	3

现在的调度方案为 $x = (1, 2, 3)$ ，即按作业 1、2、3 的顺序执行。首先将 f_1 和 f_2 数组所有元素初始化为 0。该调度方案的总时间计算如下：



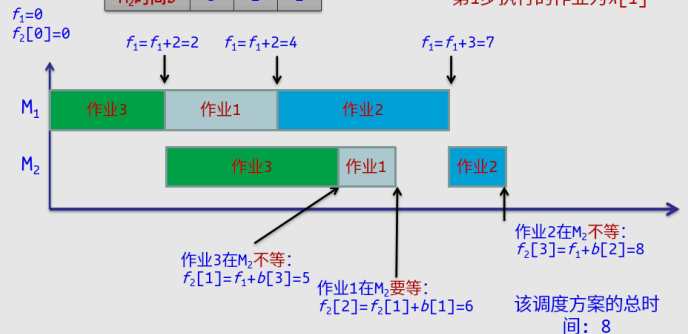
该调度方案的总时间：10

再看看另外一种调用方案 $x = (3, 1, 2)$ 。该调度方案的总时间计算：

作业编号	3	1	2
M_1 时间 a	2	2	3
M_2 时间 b	3	1	1

$x = (x[1], x[2], \dots, x[i], \dots, x[n])$

第 i 步执行的作业为 $x[i]$



- 解表示：**排列 $x[1..n]$ ， $x[i]$ 表示第 i 步加工的作业编号。
- 解空间：**排列树， $n!$ 个叶子。
- 时间计算公式：**

两条流水线的工作方式：

- M_1 从不空转——作业一个接一个按顺序做，累计时间 f_1 直接累加 $a[x[i]]$
- M_2 可能空转、也可能让 M_1 等——因为每个作业必须先做完 M_1 才能到 M_2

所以第 i 个作业在 M_2 上的开始时间是： $\max(M_1 \text{ 做完当前作业的时间}, M_2 \text{ 做完上一个作业的时间})$

Remark

两个机器相互等待的两种情况

情况 A — M_2 等 M_1 ： $f_1 > f_2[i-1]$ M_1 做完当前作业时， M_2 还在做上一个作业，当前作业在 M_2 入口排队。 $f_2[i] = f_1 + b[x[i]]$

情况 B — M_1 等 M_2 ： $f_1 \leq f_2[i-1]$ M_2 早就做完上一个作业了，但当前作业还在 M_1 上没传过来， M_2 空转等待。 $f_2[i] = f_2[i-1] + b[x[i]]$

合并： $f_2[i] = \max(f_1, f_2[i-1]) + b[x[i]]$

总时间即为最后一个作业离开 M_2 的时刻 = $f_2[n]$

- 剪枝策略：**

剪枝 1 (基础)： $f_2[i] < \text{best}_f$ 。当前部分调度在 M_2 上的完成时间已经 $\geq$ 已知最优总时间，不可能更优，直接砍掉。

剪枝 2 (下界 bound 进一步剪枝)：对于已选择的作业，它们的 b 时间和为 $\sum_b$ ，所有 n 个作业的 b 时间和为 tot ，剩余未选作业的 b 时间和为 $tot - \sum_b$ 。

即使剩余作业在 M_2 上一个接一个完全不停顿地做（乐观估计），总时间至少也是 $f_2[i] + (tot - \sum_b)$ 。若这个下界已经 $\geq best_f$ ，则剪枝。

Algorithm 8.5.8

流水作业调度进一步剪枝的 bound 函数

PYTHON

PYTHON

```
def bound(i, tot):
    # tot = sum(b[1..n]), 所有 b 时间和
    sum_b = sum(b[x[j]] for j in range(1, i+1)) # 已选作业的 b 时间和
    return f2[i] + (tot - sum_b) # 乐观估计的总时间下界
```

dfs 中的剪枝条件更新为： $bound(i, tot) < best_f$

Remark

bound 剪枝比单纯的 $f_2[i] < best_f$ 更强——它考虑了剩余作业在 M_2 上的时间，可以更早地判断这条路没前途。

5. 伪代码

Algorithm 8.5.9

流水作业调度问题的回溯解法

输入： $a[1..n]$ (M_1 时间), $b[1..n]$ (M_2 时间) **输出**：最优调度 $best_{x[1..n]}$ 和最少总时间 $best_f$

PYTHON

PYTHON

```
def bound(i, tot):
    # 总时间下界：当前完成时间 + 剩余所有作业的 b 时间和
    sum_b = sum(b[x[j]] for j in range(1, i+1))
    return f2[i] + (tot - sum_b)

def dfs(i, tot):
    if i > n: # 得到一个完整排列
        if f2[n] < bestf:
            bestf = f2[n]
            copy(x, bestx)
        return
    for j in range(i, n + 1): # 枚举剩余作业
        swap(x[i], x[j])
        f1 += a[x[i]]
```

```

f2[i] = max(f1, f2[i-1]) + b[x[i]]
if bound(i, tot) < bestf: # 剪枝 (用 bound 代替 f2[i] < bestf)
    dfs(i + 1, tot)
f1 -= a[x[i]]           # 回溯
swap(x[i], x[j])

```

6. 复杂度

- 时间：最坏 $O(n!)$
- 空间： $\Theta(n)$

8.5.5 货郎担问题（旅行商问题）

Problem 8.5.10

货郎担问题

有 n 个城市，已知任意两城市 i, j 之间的距离 $d[i][j]$ 。从城市 1 出发，每个城市恰好访问一次，最后回到城市 1，求总路程最短的回路。

1. **解表示**：排列 $x[1..n]$ ， $x[k]$ 表示第 k 步访问的城市，固定 $x[1] = 1$ 。
2. **解空间**：排列树， $(n-1)!$ 个叶子。
3. **剪枝**：
 1. 当前已走路程 c_l ，若 $c_l \geq \text{best}_l$ 直接剪掉
 2. 到达叶子时检查 $d[x[n]][x[1]]$ （能否回起点），更新 best_l
4. **伪代码**

Algorithm 8.5.11

货郎担问题的回溯解法

输入：距离矩阵 $d[1..n][1..n]$ **输出**：最短回路 $\text{best}_{x[1..n]}$ 和最短距离 best_l

PYTHON

PYTHON

```

def dfs(k):
    if k == n + 1:           # 所有城市走完
        if d[x[n]][x[1]] < INF: # 能回到起点
            total = cl + d[x[n]][x[1]]
            if total < bestl:
                bestl = total
                copy(x, bestx)
        return
    for j in range(k, n + 1): # 枚举剩余城市
        swap(x[k], x[j])
        cl += d[x[k-1]][x[k]]
        if cl < bestl:         # 剪枝

```

```
        dfs(k + 1)
    cl -= d[x[k-1]][x[k]]      # 回溯
    swap(x[k], x[j])
```

5. 复杂度

- 时间：最坏 $O(n!)$
- 空间： $\Theta(n)$

AlgoAnalysis

作者姓名

2026 年 06 月 16 日

9 chapter9: 分支限界法

9.1 引言

9.1.1 分支限界与回溯的异同

Definition 9.1.1

分支限界法

分支限界法 (branch and bound) 和回溯法一样, 都是通过系统地搜索解空间树来求解的算法。区别在于搜索方式:

- **回溯法**: 深度优先搜索 (DFS) 遍历状态空间树, 用约束函数和限界函数剪枝
- **分支限界法**: 广度优先或最小成本优先搜索, 每访问一个 E-结点, 生成它的所有儿子结点, 再利用限界函数剪枝

回溯法用栈 (递归栈或显式栈) 管理活结点, 分支限界法用队列 (FIFO) 或优先队列 (LC) 管理活结点。

9.1.2 三种检索策略

分支限界法根据活结点表的数据结构分为三种检索策略:

1. **FIFO 检索**: 活结点表采用普通队列 (first in first out)。每次取出队首结点作为 E-结点, 生成的所有儿子入队尾。等价于 BFS 的层序遍历, 选择方式死板、盲目。
2. **LIFO 检索**: 活结点表采用栈 (last in first out)。每次取出栈顶结点作为 E-结点, 新儿子压栈。类似于 D-检索。
3. **LC-检索 (最小成本检索)**: 对每个活结点计算成本估计值 $\hat{c}(\cdot)$, 从优先队列中选取 $\hat{c}$ 最小的活结点作为下一个 E-结点。使最有可能通向答案的结点优先被扩展。

9.1.3 成本估计函数

Definition 9.1.2

结点成本函数 $c(\cdot)$

对于状态空间树上的结点 X :

- 若 X 本身是答案结点, 则 $c(X)$ 是从根到 X 的路径成本
- 若 X 不是答案结点且子树 X 不含任何答案结点, 则 $c(X) = \infty$
- 否则 $c(X)$ 等于子树 X 中具有最小成本的答案结点的成本

然而, 精确计算 $c(X)$ 需要搜索子树 X ——和解原问题一样困难。因此引入成本估计函数 $\hat{c}(X)$:

$$\hat{c}(X) = f(h(X) + \hat{g}(X))$$

其中:

- $h(X)$ 是从根结点到结点 X 的已发生成本 (已知)
- $\hat{g}(X)$ 是从 X 到最近答案结点所需附加成本的估计 (启发式)
- f 是一个非降函数, 用于平衡纵深搜索与宽度搜索

若 X 是答案结点或叶子结点, 则 $c(X) = \hat{c}(X)$ 。

9.1.4 15-谜问题

Problem 9.1.3

15-谜问题

在一个 4×4 的棋盘上放有 15 块编号为 1 至 15 的牌和一块空格, 每次只允许将空格相邻的牌移到空位上。给定一个初始排列, 判断能否通过合法移动到达目标排列 (按序排列, 空格在右下角)。

9.1.4.1 可达性判定: 奇偶不变量

并非所有排列都能通过合法移动还原。可达性由以下定理判定:

Definition 9.1.4

可达性判定

给 4×4 棋盘方格编号 $1 \sim 16$, 位置 i 是目标排列中放 i 号牌的方格位置, 位置 16 是空格位置。设:

- $POSITION(i)$: 编号 i 的牌在初始状态下的位置号 ($1 \leq i < 16$), $POSITION(16)$ 为空格的位置号
- $LESS(i)$: 满足 $j < i$ 且 $POSITION(j) > POSITION(i)$ 的 j 的数目 (实质是逆序对计数)
- X : 若初始空格在棋盘阴影区域中 (通常为 (行 + 列) 和为奇数的方格), 则 $X = 1$; 否则 $X = 0$

定理: 当且仅当 $\sum_{i=1}^{16} LESS(i) + X$ 为偶数时, 目标状态可由初始状态到达。

Remark

这个判定的数学基础是: $\sum LESS(i) + X$ 的奇偶性是合法移动下的**不变量**。横向移动交换空格与相邻非空白牌, 同时改变 $LESS$ 的奇偶性; 纵向移动跨行排列时 $LESS$ 和 X 同时翻转, 二者抵消。因此一旦初始状态算出的总和为奇数, 可立即判定无解, 无需搜索。

9.1.4.2 三种检索策略对比

PPT 分别演示了 FIFO、DFS、LC 三种策略处理 15-谜问题:

- **FIFO 检索:** 一层层展开所有可能移动, 结点膨胀极快, 无方向性
- **DFS 检索:** 沿某一支深入, 可能长时间找不到解
- **LC-检索 (最优):** 定义 $\hat{c}(X) = h(X) + \hat{g}(X)$, 其中:

- $h(X)$ = 根到结点 X 的路径长度（实际已走步数）
- $\hat{g}(X)$ = **不在其目标位置的非空白牌数目**（易于计算的启发式）

可以证明 $\hat{c}(X) \leq c(X)$ ，即 $\hat{c}$ 是 c 的**下界**。LC-检索据此优先扩展最可能接近目标的结点。

9.2 LC-分支限界策略

9.2.1 基本思想

LC-分支限界的核⼼策略：

1. 使用满足 $\hat{c}(X) \leq c(X)$ 的成本估计函数 $\hat{c}$ ，给出从任一结点 X 出发的解的下界
2. 维护当前最优解的成本**上界** U （初始可置 ∞ 或用启发式估计）
3. 剪枝条件：若 $\hat{c}(X) > U$ ，则结点 X 可以被杀死——因为 $c(X) \geq \hat{c}(X) > U$ ，其子树不可能包含优于当前解的解

Remark

LC-检索与 FIFO/LIFO 的本质区别在于取下一个 E-结点时不是机械地取头/取顶，而是按 $\hat{c}$ 值排序，让最有希望的结点领先扩展。这就是“有智力的”搜索。

9.2.2 LC-SEARCH 算法框架

Algorithm 9.2.1

LC-SEARCH

输入： 状态空间树 T ，成本估计函数 $\hat{c}$ **输出：** 从根到答案结点的路径（或“无解”）

TEXT

TEXT

```

1. if T 是答案结点 then 输出 T; return
2. E ← T
3. 初始化活结点表为空
4. repeat
5.     for E 的每个儿子 X
6.         if X 是答案结点 then
7.             输出从 X 到 T 的路径; return
8.         end if
9.         ADD(X)                // X 加入活结点表
10.        PARENT(X) ← E        // 记录父结点
11.    end for
12.    if 活结点表为空 then
13.        输出"无解"; return
14.    end if

```

```

15.     LEAST(E)           // 取 hat(c) 最小的活结点为新的 E-结点
16. end repeat

```

9.3 经典应用

9.3.1 货郎担问题 (TSP)

Problem 9.3.1

货郎担问题 (旅行商问题 TSP)

给出 n 个城市的集合和一个定义在每对城市间的耗费函数 (可以是距离、时间、票价等非负值), 找出一个闭合回路, 使每个城市恰好被访问一次且总耗费最小。解空间为排列树, 包含 $(n-1)!$ 条可能的回路。

9.3.1.1 归约矩阵

Definition 9.3.2

归约矩阵

对耗费矩阵 A , 每一行减去该行的最小值、每一列减去该列的最小值, 使每行每列至少有一个 0 元素。因为每条周游路线在每行每列恰好选一条边, 行/列减去常数 r 会使所有周游路线的总耗费减少 r 。

总归约代价 = 所有行归约代价之和 + 所有列归约代价之和, 且为最优解的**下界**。

PPT 例子 (5 城市) 总归约代价为 63, 即根结点 $\hat{c}(\text{根}) = 63$ 。

9.3.1.2 边的选择与惩罚值 r_R

对归约矩阵中每个值为 0 的边 (i, j) , 计算**强制不选该边的惩罚**:

$$r_R = \min_{k \neq j} \{A(i, k)\} + \min_{k \neq i} \{A(k, j)\}$$

含义: 不选 (i, j) 时, 城市 i 必须选择该行其他出边 (最小值为第一项), 城市 j 必须从其他城市接收入边 (最小值为第二项)。

Remark

r_R 是算法中的**关键复用量**: 选边时用它判断哪条 0 边最适合做分裂点 (r_R 越大 $\rightarrow$ 不选它的惩罚越高 $\rightarrow$ 这条边越可能是最优解中的必经之路); 右儿子的成本公式中也用 r_R 作为增量来抬高右子树的 $\hat{c}$, 迫使搜索优先走左子树 (包含该边的方向)。

PPT 中图例对根结点选 (3,5) 边时 $r_R = 23$ ，但右子结点的矩阵漏掉了列归约（第 5 列未归约），导致图上 bound 标注为 75，实际应为 $63 + 23 = 86$ 。考试时需注意： r_R 包含行和列两项，缺一不可。

9.3.1.3 左右儿子的估计成本

以边 (i, j) 分裂后：

左儿子 S_L (包含 (i, j)) :

- 删第 i 行（城市 i 已出发）和第 j 列（已到达城市 j ）
- 禁止回头： $A(j, i) \leftarrow \infty$
- 对不全为 ∞ 的行列重新归约，得归约代价 r_L

$$\hat{c}(S_L) = \hat{c}(R) + A(i, j) + r_L$$

右儿子 S_R (不含 (i, j)) :

- $A(i, j) \leftarrow \infty$ （禁止选该边）
- 重新归约第 i 行和第 j 列，得归约代价 r_R

$$\hat{c}(S_R) = \hat{c}(R) + r_R$$

9.3.1.4 搜索流程

Algorithm 9.3.3

TSP 分支限界搜索

输入： $n \times n$ 耗费矩阵 **输出：** 最小成本周游路线

TEXT

TEXT

1. 归约矩阵，得总归约代价作为根 $\hat{c}$
2. 根入优先队列（按 $\hat{c}$ 排序）
3. repeat
4. 取出 $\hat{c}$ 最小的 E-结点
5. 在 E 的归约矩阵中找出所有 0 边，计算每条边的 r_R
6. 选取 r_R 最大的 0 边 (i, j) 作为分裂边
7. 生成左儿子（包含 (i, j) ）：删行删列 $\rightarrow$ 归约 $\rightarrow$ 算 $\hat{c}$ $\rightarrow$ 入队
8. 生成右儿子（不含 (i, j) ）：置 ∞ $\rightarrow$ 归约 $\rightarrow$ 算 $\hat{c}$ $\rightarrow$ 入队
9. until 某叶结点的 $\hat{c} \leq$ 所有活结点的 $\hat{c}$

每次分裂锁定一条“必须走”或“必须不走”的边，深度最多 n 层可得完整解。由于 LC-检索始终扩展 $\hat{c}$ 最小的结点，且 $\hat{c}$ 是下界，故首次到达的叶结点即为最优解。

9.3.2 0/1 背包问题

Problem 9.3.4

0/1 背包问题

有 n 个物品，第 i 个物品重量 w_i 、价值 v_i 。背包容量为 W 。每个物品要么选中 (1) 要么不选 (0)，求总重量不超过 W 时总价值最大的方案。解空间为子集树，共 2^n 个叶子。

9.3.2.1 上界函数 bound()

0/1 背包是最大化问题，需用**上界**限界。思路是：假设剩余物品可以按任意比例装入——即分数背包松弛，其最优解 $\geq$ 0/1 背包的最优解，故是合法上界。

Algorithm 9.3.5

bound() 上界函数

输入：当前结点 e （已装重量 $e.w$ ，已装价值 $e.v$ ，层次 $e.i$ ） **输出：**上界值 $e.ub$

PYTHON

PYTHON

```
def bound(e):
    i = e.i + 1
    sumw = e.w
    sumv = e.v
    while i ≤ n and sumw + w[i] ≤ W:
        sumw += w[i]
        sumv += v[i]
        i += 1
    if i ≤ n:
        # 剩余物品只能部分装入
        e.ub = sumv + (W - sumw) * v[i] / w[i]
    else:
        # 剩余物品全部能装入
        e.ub = sumv
```

9.3.2.2 剪枝条件

1. **左剪枝（约束函数）**，选当前物品： $e.w + w[e.i + 1] \leq W$ ，否则超重剪枝
2. **右剪枝（限界函数）**，不选当前物品： $e_2 \cdot ub > \max v$ 才保留，否则不可能超过当前最优解

9.3.2.3 优先队列式分支限界法

优先队列式使用大根堆（按 ub 排序）， ub 越大越先出队。

Algorithm 9.3.6

0/1 背包优先队列式

输入: $n, W, w[1..n], v[1..n]$ 输出: 最大价值 maxv 及最优解向量 $\text{bestx}[1..n]$

PYTHON

PYTHON

```
def bfs():
    e = Node(i=0, w=0, v=0, x=[0]*(n+1))
    bound(e)
    heap = [(-e.ub, e)]          # 大根堆 (存负值实现)

    while heap:
        _, e = heappop(heap)     # 取 ub 最大的结点

        if e.w + w[e.i+1] ≤ W:   # 左孩子
            e1 = Node(i=e.i+1, w=e.w+w[e.i+1], v=e.v+v[e.i+1], x=e.x.copy())
            e1.x[e1.i] = 1
            bound(e1)
            if e1.i == n:        # 叶子
                if e1.v > maxv:
                    maxv = e1.v; bestx = e1.x
            else:
                heappush(heap, (-e1.ub, e1))

        # 右孩子
        e2 = Node(i=e.i+1, w=e.w, v=e.v, x=e.x.copy())
        e2.x[e2.i] = 0
        bound(e2)
        if e2.ub > maxv:         # 右剪枝
            if e2.i == n:
                if e2.v > maxv:
                    maxv = e2.v; bestx = e2.x
            else:
                heappush(heap, (-e2.ub, e2))
```

9.3.2.4 复杂度

无论队列式还是优先队列式, 最坏情况均需搜索整个解空间树, 时间和空间复杂度均为 $O(2^n)$ 。

9.3.3 最大团集问题

Problem 9.3.7

最大团集问题

给定无向图 $G = (V, E)$ 。

- 团: 顶点子集 $U \subseteq V$, 其中任意两顶点间都有边

- **极大团**: 不被更大的完全子图包含的团
- **最大团**: 顶点数最多的团

解空间为子集树, 共 2^n 个叶子 (每个顶点选或不选)。

9.3.3.1 上界函数

$$\text{upperSize} = \text{cliqueSize} + n - \text{level} + 1$$

其中 cliqueSize 为当前团已包含的顶点数, level 为当前层数 (从 1 开始), $n - \text{level} + 1$ 为剩余未决定的顶点数。这是一个乐观估计——假设剩余顶点全部可以加入团中。

9.3.3.2 优先队列式分支限界法

Algorithm 9.3.8

最大团优先队列式

输入: 邻接矩阵 $a[1..n][1..n]$ ($a[i][j] = 1$ 表示有边) **输出**: 最大团大小 bestn 及顶点选择向量 $\text{bestx}[1..n]$

PYTHON

PYTHON

```
def bound(cliqueSize, level):
    return cliqueSize + n - level + 1

def bfs():
    heap = [(0, 0, 0, [0]*(n+1))] # (-upperSize, level, cliqueSize, x)
    bestn = 0

    while heap:
        _, level, cliqueSize, x = heappop(heap)

        if level > n: # 到达叶子
            if cliqueSize > bestn:
                bestn = cliqueSize
                bestx = x.copy()
            continue

        # 左儿子: 选顶点 level
        ok = True
        for j in range(1, level):
            if x[j] == 1 and a[level][j] == 0:
                ok = False # 与已选顶点无边相连
                break
        if ok:
            x1 = x.copy()
```

```

x1[level] = 1
ub = bound(cliqueSize + 1, level + 1)
heappush(heap, (-ub, level+1, cliqueSize+1, x1))

# 右儿子: 不选顶点 level
ub = bound(cliqueSize, level + 1)
if ub > bestn:                # 右剪枝
    x2 = x.copy()
    x2[level] = 0
    heappush(heap, (-ub, level+1, cliqueSize, x2))

```

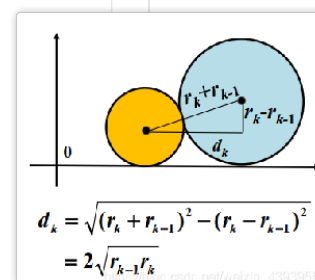
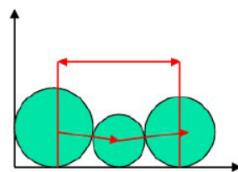
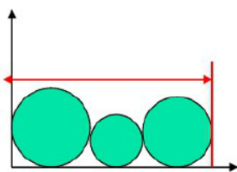
上界函数 $\text{upperSize} = \text{cliqueSize} + n - \text{level} + 1$ 剪枝能力较弱, 但计算简单。

9.3.4 圆排列问题

Problem 9.3.9

圆排列问题

给定 n 个圆的半径 $r[1..n]$ (可能相等), 将它们排成一行, 使每个圆均与底边相切。求一种排列, 使所有圆在水平方向上的总宽度最小。



9.3.4.1 圆心横坐标计算

两个相切圆 i 和 j 的圆心水平距离由勾股定理得出: $2\sqrt{r_i r_j}$ 。第 i 个圆的圆心坐标取它与所有已排圆相切所得的最大值。

9.3.4.2 总宽度计算

Algorithm 9.3.10

总宽度计算

PYTHON

PYTHON

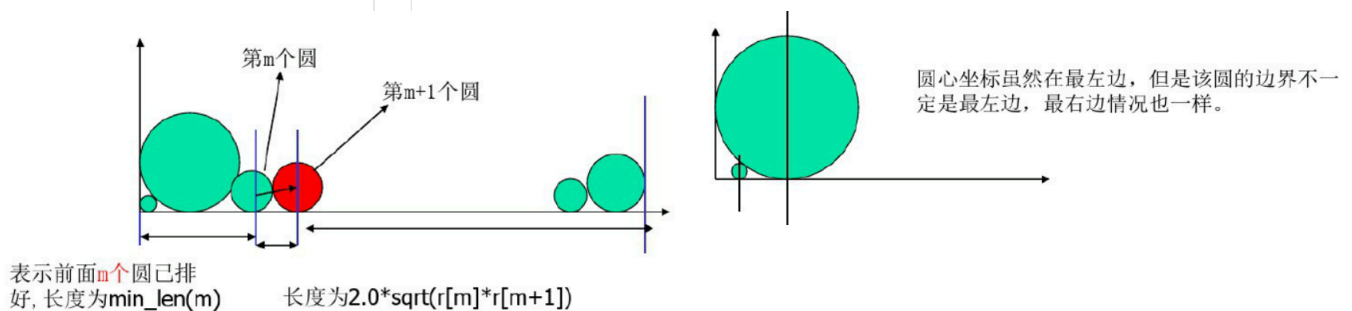
```
def compute():
    low = 0
    high = 0
    for i in range(1, n+1):
        if x[i] - r[i] < low:
            low = x[i] - r[i]
        if x[i] + r[i] > high:
            high = x[i] + r[i]
    if high - low < min:
        min = high - low          # 更新最优解
```

9.3.4.3 三种剪枝策略

1. **镜像对称剪枝**: 排列 $(1, 2, \dots, n)$ 与 $(n, \dots, 2, 1)$ 互为镜像, 总宽度相同。仅需计算 $\frac{n!}{2}$ 种排列, 即固定第一个圆, 在 2 到 $n-1$ 之间排列。
2. **相同半径剪枝**: 若有 k 个圆半径相同, 互换位置不改变总宽度, 只计算一个。交换前先判断半径是否相等。
3. **下界约束加强**: 对已排 m 个圆的状态:

$$\text{min_len}(m) + 2\sqrt{r_m r_{m+1}} + \text{len_r}(n-1-m) \geq \text{min}$$

其中 $\text{len_r}(n-1-m) = r_{m+2} + \dots + r_n$ 是剩余圆半径之和的松弛下界, 若此下界 $\geq$ 当前最优宽度则剪枝。



9.3.4.4 优先队列式分支限界法

Algorithm 9.3.11

圆排列优先队列式 B&B

输入: n , 半径数组 $r[1..n]$ **输出:** 最小圆排列长度 min 及最优排列 bestx

初始化:

1. 对 n 个圆的编号做排列 $p = [1, 2, \dots, n]$
2. 最优值 $\text{min} = \infty$
3. 圆心横坐标数组 $x[1..n]$

4. 固定第一个圆 $x[1] = 0$

镜像对称剪枝：仅排列第 2 至第 $n-1$ 个位置，第一个圆固定在第 1 位，最后一个圆在剩余圆中自动确定。

递归搜索过程（从第 2 层开始）：

TEXT

TEXT

```

过程 dfs(level)
1.  if level > n then
2.      调用 compute() 计算总宽度
3.      若 high-low < min 则更新 min 和 bestx
4.      return
5.  end if
6.  for i ← level to n-1          // 枚举剩余圆（固定最后一个）
7.      // 相同半径剪枝
8.      if i > level 且 r[p[i]] = r[p[level]] then 跳过
9.      swap(p[level], p[i])      // 将第 i 个圆换到当前位置
10.     x[level] ← center(level)  // 计算当前圆的圆心横坐标
11.     计算当前已排总宽度 cur_width = max(x[j] + r[j]) - min(x[j] - r[j])
12.     // 下界约束剪枝
13.     r_sum ← 剩余圆半径之和
14.     lb ← cur_width + 2√(r[level]·r[level+1]) + r_sum
15.     if lb ≥ min then
16.         swap(p[level], p[i])  // 恢复，剪枝
17.         continue
18.     end if
19.     dfs(level + 1)            // 前进
20.     swap(p[level], p[i])      // 回溯
21. end for

```

其中 `center(level)` 计算第 `level` 个圆的圆心横坐标（取与所有已排圆相切距离的最大值），`compute()` 遍历所有 n 个圆计算 `high - low` 更新 `min`。

Remark

圆排列的分支限界实现在排列树上做 LC-检索，配合三种剪枝策略（镜像对称、相同半径、下界约束）大幅缩小搜索空间。最坏复杂度仍为 $O(n!)$ ，但实际运行中剪枝效果显著。

您的文档标题

作者姓名

2026 年 06 月 16 日

10 chapter10: 随机算法

10.1 导论

1. 与确定性算法相比，随机算法所需运行时间或空间通常较小；
2. 随机算法易于理解和实现。
3. **Las Vegas 算法²**：总是给出正确的解，或者无解

通常需要分析**算法的平均运行时间**，运行时间本身是一个随机变量。

期望运行时间是输入规模的多项式且总能给出正确答案的随机算法称为**有效的拉斯维加斯算法**，也称为 **ZPP 类算法**(zero-error probabilistic polynomial time)算法，即零错误概率多项式时间随机算法。

4. **Monte Carlo 算法**：总是给出解，偶尔可能是错误解，可以通过多次运行原算法，且满足每次运行时随机选择都相互独立，使产生错误解的概率减到任意小。

通常需要分析**算法的出错概率**，总是在**多项式时间内运行**且出错概率**不超过 $1/3$** 的随机算法称为**有效的*蒙塔卡洛型随机算法**，也称为 **BPP 类算法**(bounded-error probabilistic polynomial time)，即错误概率有限的多项式时间随机算法。

错误类型：

- **弃真型错误**：在求解一个判定问题时把本应该接受的输入误判为拒绝。
- **取伪型错误**：就是把本应该拒绝的输入误判为接受。

根据是否同时出现两类错误，蒙特卡洛型算法又分为：

1. **单侧错误的蒙特卡洛型算法**：在所有输入上，要么犯弃真的错误，要么犯取伪的错误，不同时出现两种不同的错误。
 - 有效的**弃真型**单侧错误随机算法称为 **RP 类算法**(Randomized Polynomial-time)
 - 有效的**取伪型**单侧错误随机算法称为 **coRP 类算法**(Complementary Randomized Polynomial-time, RP 的补类)
2. **双侧错误的蒙特卡洛型算法**：在所有输入上可以同时出现两种不同的错误。

²Las Vegas 和 Monte Carlo 不是一种算法的名称，而是对一类随机算法的特性的概括。不太可能为 **NP 完全问题设计出多项式时间的随机算法**

对于随机算法 A, 其在大小为 n 的一个实例 I 上的运行时间可能因不同次执行而异。因此, 更自然的性能度量是 **A 在实例 I 上的期望运行时间**, 也即算法 A 反复求解实例 I 所花费的平均时间。

- 对于确定算法, 求解期望运行时间时, 概率分布定义在**输入**上
- 对于随机算法, 求解期望运行时间时, 概率分布定义在**算法**上

期望时间: 对于大小为 n 的特定输入 I , 算法的期望时间就是在随机数的不同序列上的时间的平均:

$$ET(A, I) = \sum_{\text{random sequence } R} \text{probability}(R) * \text{time}(A, I, R)$$

最坏情况下的期望时间:

$$W_ET(A) = \max_{\text{input } I \text{ with size } n} (\text{random sequence } R \text{ probability}(R) * \text{time}(A, I, R))$$

10.2 随机算法举例

这是一个很不错的讲解栏目: <https://www.luogu.com.cn/article/wegx023s>

10.2.1 随机快速排序

其实是在快速排序的基础上, 增加了**随机选取**的步骤。

(,,)

(,,)

(, +1,)

(,,)

$\leftarrow (,)$ interchange and

(,,)

(,+1,)

该算法在最坏情况的运行时间仍然是 $\Theta(n^2)$, 但这个最坏情况不是由于输入形式, 而是由于随机数生成器每次非常不幸地选择了不好的划分元素, 这种情况不大可能发生。算法的期望运行时间为 $\Theta(n \log n)$, 是一个 ZPP 类算法

10.2.2 随机选择

$\leftarrow - + 1$

let $= \lfloor p/5 \rfloor$. Divide into groups of 5 elements each. If 5 doesn't divide ,

then discard the remaining elements.

Sort each of the groups individually and extract its median element.

Let the set of medians be M

$\leftarrow (M, 1, \lfloor n/2 \rfloor)$ Partition into three arrays:

$A_1 \leftarrow \{A \mid A < mm\}, A_2 \leftarrow \{A \mid A = mm\}, A_3 \leftarrow \{A \mid A > mm\}$

$v \leftarrow$

$x \leftarrow A[v]$

Partition $A[low...high]$ into three arrays: $A_1 \leftarrow \{A \mid A < x\}, A_2 \leftarrow \{A \mid A = x\}, A_3 \leftarrow \{A \mid A > x\}$

在最坏情况下, 随机数生成器在第 j 次迭代中选择第 j 小的元素, 因此只丢弃一个元素。由于将 A 划分为 A_1 、 A_2 和 A_3 所需的比较次数正好为 n , 因此算法在最坏情况下进行的元素比较总数为 $\sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$ 。设 $C(n)$ 为对输入数组有 n 个元素时进行元素比较运算的期望次数。由于随机选择的 v 可以等概率假设为 $1, 2, \dots, n$ 中的任何一个, 因此根据 $v < k$ 或 $v > k$, 有两种情况需要考虑。

- $v < k$: 剩余元素的数量为 $n - v$ (v 右侧的元素数量为 $n - v$)
- $v > k$: 剩余元素的数量为 $v - 1$ (v 左侧的元素数量为 $v - 1$)

因而有:

$$C(n) = n + \frac{1}{n} \sum_{v=1}^n \begin{cases} C(v-1) & \text{if } v < k \\ C(n-v) & \text{if } v > k \end{cases}$$

由数学归纳法, 假设 $C(m) \leq 4m$ 对所有 $m < n$ 成立, 下证 $C(n) \leq 4n$:

$$\begin{aligned} C(n) &\leq n + \frac{1}{n} \sum_{v=1}^n 4 \cdot \max(n-v, v-1) \\ &\leq n + \frac{8}{n} \cdot \sum_{v=1}^{\frac{n}{2}} (n-v) \\ &\leq n + \frac{8}{n} \cdot \frac{3n^2}{8} \\ &= 4n \end{aligned}$$

又由于每个元素至少被检查一次, 因此 $C(n) \geq n$, 因此算法的期望运行时间是 $\Theta(n)$, 是 ZPP 类算法。

10.2.3 多选问题

设有 n 个元素的数组 A , 并且设 $K[1..r] (1 \leq r \leq n)$ 是一个由 1 到 n 之间的 r 个正数构成的有序数组, 即一个排序数组。

多选问题就是对于所有 i , $1 \leq i \leq r$, 选择 A 中的第 $K[i]$ 小的元素。假定 $A[1..n]$ 中所有元素各不相同。如果 $r = 1$, 该问题即为选择问题。如果 $r = n$, 该问题即为排列问题。

设 $A[1..n] = [a_1, a_2, \dots, a_n]$; $K[1..r] = [k_1, k_2, \dots, k_r]$, 设中间排位为 $k = k_{\frac{r}{2}}$, 可使用 SELECT 算法发现第 k 小的元素 a 。接着根据 a 把 A 划分成两个子序列 A_1 和 A_2 , 其中 A_1 中元素个数大于等于 a , A_2 中元素个数小于等于 a 。令 $K_1 = [k_1, k_2, \dots, k_{\frac{r}{2}}]$, $K_2 = [k_{\frac{r}{2}+1}, k_{\frac{r}{2}+2}, \dots, k_r]$ 。递归调用, 一个关于 A_1 和 K_1 的递归调用, 一个关于 A_2 和 K_2 的递归调用。

multiselect(A, K)

$r \leftarrow |K|$

Multiselect 算法的时间复杂度是 $\Theta(n \log r)$ 。这是因为调用 SELECT 算法的复杂度是 $\Theta(n)$, 所以有递推式

$$T(n, r) = \Theta(n) + T(n_1, \frac{r}{2}) + T(n_2, \frac{r}{2}), n_1 + n_2 \leq n$$

递归深度是 $\log r$, 所以时间复杂度是 $\Theta(n \log r)$ 。

将多选问题要寻找的元素称为靶标。随机一致地挑选一个元素 a (而不是通过 SELECT 精心选择), 将数组 A 关于 a 分割成两部分: 一部分元素小于 a , 另一部分元素大于 a 处理逻辑:

如果两部分均含有靶标, 则递归调用算法继续求解; 否则, 丢弃不含有靶标的部分, 对含有靶标部分继续求解

该算法以概率 $1 - O(\frac{1}{n})$ 的时间复杂度为 $O(n \log r)$ 。 r 个靶标的分配可以看作 r 次独立伯努利试验, 其分布近似二项分布 $B(r, 1/2)$ 。根据 Hoeffding 不等式, 靶标分配的偏离概率满足:

$$P\left(\left|\text{左侧靶标数} - \frac{r}{2}\right| \geq \epsilon r\right) \leq 2 \exp(-2\epsilon^2 r)$$

当 $r \geq c \log n$ 时, 此概率上界为 $O(1/n)$ 。

10.2.4 随机取样

首先将所有 n 个元素标记为未选中。接下来, 重复以下步骤, 直到选择了 m 个元素:

1. 生成一个介于 1 和 n 之间的随机数 r
2. 如果 r 被标记为未选中, 则将其标记为选中并将其添加到样本中

算法的期望运行时间 $T(n) = \Theta(n)$ 如果 $m > \frac{n}{2}$, 那么应该考虑反面, 改为随机选择 $n - m$ 个整数丢弃, 将剩下的整数添加到样本中。

设第 k 次选择时已选中 k 个元素, 则:

- 成功选中新元素的概率 $p_k = \frac{n-k}{n}$

- 失败（重复选中）的概率 $q_k = 1 - p_k = \frac{k}{n}$

根据几何分布，第 $k+1$ 个新元素的期望尝试次数为：

$$E_k = \frac{1}{p_k} = \frac{n}{n-k}$$

每次尝试的时间成本为 $\Theta(1)$ （仅需检查布尔数组 S ）。

$$\begin{aligned} T(n) &= \sum_{k=0}^{m-1} E_k \cdot \Theta(1) \\ &= \sum_{k=0}^{m-1} \frac{n}{n-k} \\ &= n \left(\frac{1}{n} + \frac{1}{n-1} + \dots + \frac{1}{n-m+1} \right) \\ &= n \cdot (H_n - H_{n-m}) \quad (\text{调和数展开}) \end{aligned}$$

利用调和数近似 $H_n \approx \ln n + \gamma$ ：

$$T(n) \approx n(\ln n - \ln(n-m)) = n \ln \left(\frac{n}{n-m} \right)$$

- 当 $m = O(n)$ 时（如 $m = \sqrt{n}$ ）： $T(n) \approx n \cdot \frac{m}{n} = \Theta(m)(\ln x \approx x - 1)$
- 当 $m = \Theta(n)$ 时（如 $m = n/2$ ）： $T(n) = \Theta(n)$

因此对于任意 $m < n$ ，最坏情况下 $T(n) = \Theta(n)$ 。

10.2.5 素数测试

易知除 2, 3 以外的所有素数都是 $6k \pm 1$ 的形式，因此可以用 $6k \pm 1 \leq \sqrt{n}$ 来对 n 是否是素数进行测试。

- 随机抽取一个数 a** ($1 \leq a < n$)；
- 检查等式**：验证涉及 a 和 n 的特定数论等式（与所选测试方法对应）。若等式不成立，则：
 - n 是合数（Composite）
 - a 称为合数的**证据**（Witness）
 - 测试终止
- 重复迭代**：返回步骤 1，直到达到预设置信度。

现在我们先介绍幂运算：

Let the binary digits of n be $b_k, b_{k-1}, \dots, b_0$

$c \leftarrow 1$

c

如果每次乘法花费 1 个时间单元, 算法的运行时间是 $\Theta(\log n)$ 。但如果是任意大的整数, 两数相乘的时间是 $O(\log^2 n)$, 则算法的运行时间是 $O(\log^3 n)$ 。

利用费马定理: 如果 n 是素数, 那么对于所有的 $a \not\equiv 0 \pmod{n}$, 都有 $a^{n-1} \equiv 1 \pmod{n}$, 如果通过 a 的测试, 则成 n 是基 2 的伪素数, 例如 341。而对于小于 100000 的数中, 只有 78 个数计算出错, 这是因为费马定理的逆并不正确。一类被称为**卡迈克尔数**的合数 n , 它对于所有相对于 n 互素的正整数 a 均满足费马定理, 他们在小于 $1e8$ 的范围内只有 255 个。

引理: 如果合数 n 不是一个卡迈克尔数, 那么算法 (随机选择 a 并验证费马定理) 将检测出 n 为合数的概率至少是 $\frac{1}{2}$ 。

若 n 是合数但非卡迈克尔数, 则存在至少一个 $b \perp n$ 使得 $b^{n-1} \not\equiv 1 \pmod{n}$, 则定义集合

$$W = \{a \in \{1, 2, \dots, n-1\} \mid a^{n-1} \not\equiv 1 \pmod{n}\}$$

而对于 W 在整数集 Z_n^* 上的补集 G , 易知 G 是 Z_n^* 的一个乘法子群, 那么由拉格朗日定理知 $|G|$ 整除 $|Z_n^*| = \phi(n)$, 所以 $|G| \leq \frac{1}{2}\phi(n)$, 所以 $|W| \geq \frac{1}{2}\phi(n)$, 引理得证。

再考虑 Miller-Rabin 算法, 设 n 为奇素数, 则存在 q, m 使得:

$$n-1 = 2^q m (q \leq 1)$$

构造序列:

$$a^m \pmod{n}, a^{2m} \pmod{n}, a^{4m} \pmod{n}, \dots, a^{2^{q-1}m} \pmod{n}$$

由费马定理知, 最后一项 $a^{n-1} \equiv 1 \pmod{n}$ 。而如果 $x^2 \equiv 1 \pmod{n}$ 的根是 $x = \pm 1$ 称为平凡根, 其他根是不平凡的, 如果存在非平凡根, 那么一定是合数。

所以检测该序列中某项是 1, 那么前一项必须是 1 或者 $n-1$ 否则 n 是合数。

$$q \leftarrow 0, m \leftarrow n-1$$

$$a \leftarrow \text{Random}(2, n-1)$$

$$x_0 \leftarrow \text{EXPMOD}(a, m, n)$$

每次测试出错 (把合数判定为素数) 概率至多是 $\frac{1}{2}$, 这样我们重复运行 k 次, 就可以将出错的概率降低到 2^{-k} , 那么取 $k = \lceil \log n \rceil$, 失败的概率不超过 $2^{-\lceil \log n \rceil} \leq \frac{1}{n}$, 而这就是 **Miller-Rabin 算法**。

10.2.6 验证串的相等性

对于一个比特串 w , 设 $I(w)$ 是 w 表示的一个整数, 那么可以选一个素数 p , 这样得到一个指纹函数:

$$I_p(x) = I(x)(\text{mod } x)$$

对于 $I_p(x) = I_p(y)$, 而 $x \neq y$ 的现象, 我们称之为**假匹配**。每次验证时重新选择 p , 并重复选择 p , 可以增强 $x = y$ 的可信度。

我们知道, 设 $\pi(n)$ 是小于 n 的不同素数的个数, 那么 $\pi(n)$ 趋近于 $\frac{n}{\ln n}$. 对于两个长度为 n 的串 x, y , 如果发生假匹配, 那么 $p \mid (I(x) - I(y))$, 所以其概率为:

$$\frac{|\{p \leq M \mid p \text{ is prime and } p \mid I(x) - I(y)\}|}{\pi(M)} \leq \frac{\pi(n)}{\pi(M)}$$

关键观察: 假匹配发生时, 素数 p 必须满足 $p \mid (I(x) - I(y))$ 。由于 x 和 y 是长度为 n 的比特串, $I(x)$ 和 $I(y)$ 的绝对值不超过 $2^n - 1$, 因此

$$|I(x) - I(y)| \leq 2^n - 1.$$

(这个结论需要在 $n > 11$ 后成立, 这是由切比雪夫函数完成估计的, 最终 n 以内的素数的乘积的增长速率约为 $e^{0.9n}$, 是高于 $2^n - 1$ 这个速率的, 所以该估计是正确的)

而如果取 $M = n^2$, 重复执行 k 次算法, 这样最后概率最大是 $(\frac{2}{n})^k$.

10.2.7 模式匹配

确定模式串是否出现在文本串中(假设都是0,1串)。利用随机算法比较模式指纹 $I_p(Y)$ 和文本块 $I_p(X(j))$, 让文本块的指纹要易于计算, 最好还可以通过 $X(j)$ 计算出 $X(j+1)$

仿照上一题思想, 我们把这里的指纹就假定为选定的串对应的数的大小, 那么有: $I_p(X(j+1)) = (2I_p(X(j)) - 2^{mx_j + x_j+m}) \pmod{p}$

在小于 M 的素数中随机选择素数 p

$j \leftarrow 1$

计算 $I_p(Y), I_p(X_j)$

易知该算法的时间复杂度是 $O(m + n)$

10.2.8 货郎担问题

假设有一个旅行商人要拜访 n 个城市, 路径选择需满足以下限制:

1. 每个城市只能拜访一次;
2. 最终必须回到出发城市。

优化目标: 选择总路程最短的路径

$bestPath \leftarrow []$ $bestDistance \leftarrow \infty$ $path \leftarrow$ 随机排列 $(1, 2, \dots, n)$
 $distance \leftarrow$ 计算路径总长度 $(path, D)$

10.3 随机游走算法

常用有限马尔科夫链来分析随机游走算法。

马尔可夫特性：随机过程在给定 t 时刻的状态为 X_t ，该过程的后续状态及其出现的概率与 t 之前的状态无关。也就是说，过程当前的状态包括了过程所有的历史信息，该过程的进一步发展完全由当前状态所决定，与当前状态之前的历史无关，这种性质也称为**无后效性**或**无记忆性**。

马尔科夫不等式：设 X 为只取非负的随机变量，数学期望 $E(X) = \mu$ ，对于任意 $\varepsilon > 0$, $P(X \geq \varepsilon) \leq \frac{\mu}{\varepsilon}$

1. **Integral method** (for continuous case):

$$E(X) = \int_0^{\infty} xf(x)dx \geq \int_{\varepsilon}^{\infty} xf(x)dx \geq \int_{\varepsilon}^{\infty} \varepsilon f(x)dx \geq \varepsilon P(X \geq \varepsilon)$$

2. **Indicator method** (general): Define $Y = \varepsilon \cdot \mathbf{1}_{\{X \geq \varepsilon\}}$, then:

$$E(Y) = \varepsilon P(X \geq \varepsilon) \leq E(X)$$

设一个离散的随机序列 $\{X_0, X_1, \dots\}$ ，其中每个 X_i 都从有限集中取值，若它满足

$$P(X_{i+1} = x_{i+1} \mid X_i = x_i, X_{i-1} = x_{i-1}, \dots, X_0 = x_0) = P(X_{i+1} = x_{i+1} \mid X_i = x_i) = p_{x_i, x_{i+1}}$$

这称之为**有限马尔科夫链**

对于有限马氏链，不妨设 X_t 取值的状态空间为 $\{0, 1, \dots, n\}$ ，于是 $p_{i,j}$ (从状态 i 转移到状态 j 的概率) 可组成一个 $n+1$ 阶方阵

$$P = [p_{i,j}],$$

称为**一步转移矩阵**，每一行元素之和等于1。

设 $p_i(t)$ 表示在时刻 t 处于状态 i 的概率，则

$$p_i(t) = p_0(t-1)p_{0i} + p_1(t-1)p_{1i} + \dots + p_{n-1}(t-1)p_{(n-1)i} + p_n(t-1)p_{ni}$$

设 $\mathbf{p}(t)$ 表示向量 $(p_0(t), p_1(t), \dots, p_n(t))$ ，有

$$\mathbf{p}(t) = \mathbf{p}(t-1)P$$

对于任意 m ，定义 m 步转移矩阵

$$P^{(m)} = [p_{i,j}^{(m)}],$$

其中

$$p_{i,j}^{(m)} = P(X_{t+m} = j \mid X_t = i).$$

因此, $P^{(m)} = P^{m, \infty} \text{boldsymbol{p}}(t + m) = \text{boldsymbol{p}}(t) P^{(m)}$.

10.4 二元布尔可满足问题

随机初始化赋值 σ : 对每个变元 x_i 随机赋True或False

定理: 若输入公式是不可满足的, 则上述随机游走算法正确宣布不可满足; 若输入公式是可满足的, 则上述算法以 $1 - \frac{1}{2^m}$ 概率找到可满足赋值。

定义马尔可夫链 $\{Y_0, Y_1, Y_2, \dots\}$ 如下:

$$\begin{aligned} Y_0 &= X_0 \\ P(Y_{t+1} = 1 \mid Y_t = 0) &= 1 \\ P(Y_{t+1} = j + 1 \mid Y_t = j) &= \frac{1}{2} \\ P(Y_{t+1} = j - 1 \mid Y_t = j) &= \frac{1}{2} \quad (j > 0) \end{aligned}$$

显然有:

$$\begin{aligned} P(Y_{t+1} = j + 1 \mid Y_t = j) &= \frac{1}{2} \geq P(X_{t+1} = j + 1 \mid X_t = j) \\ P(Y_{t+1} = j - 1 \mid Y_t = j) &= \frac{1}{2} \geq P(X_{t+1} = j - 1 \mid X_t = j) \end{aligned}$$

从任意状态出发, Y_t 比 X_t 花费更长的期望时间进入状态 n 。因此 Y_t 进入状态 n 的时间作为算法期望运行时间的上界。

设 h_j 表示从状态 j 到状态 n 的期望运行时间, 则有:

$$\begin{aligned} h_n &= 0 \\ h_0 &= h_1 + 1 \\ h_j &= \frac{h_{j+1} + 1}{2} + \frac{h_{j-1} + 1}{2}, \quad 0 < j < n \end{aligned}$$

由归纳法可证, 对于所有 j , 有:

$$h_j \leq n^2 - j^2 \leq n^2$$

因此有:

$$E[X \text{ 从 } X_0 \text{ 到达 } n \text{ 的时间}] \leq E[Y \text{ 从 } Y_0 \text{ 到达 } n \text{ 的时间}] \leq n^2$$

取 $\varepsilon = 2n^2$, 根据马尔可夫不等式, 则有:

$$P(X \text{ 从 } X_0 \text{ 到达 } n \text{ 的时间} \geq 2n^2) \leq \frac{n^2}{2n^2} = \frac{1}{2}$$

证明:

- **不可满足情形:** 显然, 算法无法找到可满足赋值, 因此正确宣布不可满足。
 - **可满足情形:** 算法有可能找不到可满足赋值而出错, 下面证出错概率为 $\frac{1}{2^m}$ 。
1. **汉明距离定义:** 设 σ^* 为最优赋值, 当前赋值 σ 与 σ^* 的汉明距离 $d(\sigma, \sigma^*)$ 表示两者取值不同的变量个数。
 2. **改进概率:** 对每个不满足子句, 至少以 $\frac{1}{2}$ 概率减少 $d(\sigma, \sigma^*)$ (因子句最多 2 个文字)。
 3. **收敛分析:** 经过 $2n^2m$ 步迭代, 未到达 σ^* 的概率:

$$P(\text{失败}) \leq \left(1 - \frac{1}{2n}\right)^{2n^2m} \approx e^{-nm} \leq \frac{1}{2^m}$$

10.5 本章作业

10.5.1 优惠券收集

有 n 种优惠券, 每种优惠券数目不限。每次实验随机抽取一张优惠券, 优惠券来自 n 种中任何一种的概率均等。计算需要多少次实验才能使得收集的优惠券包含 n 种中每一种至少一张。

设 X_i 表示在已收集 $i-1$ 种优惠券后, 首次抽到第 i 种新优惠券所需的次数 ($i = 1, 2, \dots, n$)。

在已收集 k 种优惠券时, 抽到新优惠券的概率为 $p_k = \frac{n-k}{n}$, 因此 X_{k+1} 服从参数为 p_k 的几何分布, 其期望为:

$$E(X_{k+1}) = \frac{1}{p_k} = \frac{n}{n-k}$$

集齐所有 n 种优惠券的总次数 T 为各阶段次数之和:

$$T = X_1 + X_2 + \dots + X_n$$

总期望为:

$$E(T) = \sum_{k=0}^{n-1} \frac{n}{n-k} = n \sum_{k=1}^n \frac{1}{k} = nH_n$$

当 n 较大时, $H_n \approx \ln n + \gamma$ 。因此:

$$E(T) \approx n \ln n + \gamma n$$

集齐 n 种优惠券的平均实验次数为:

$$E(T) = nH_n = n\left(1 + \frac{1}{2} + \dots + \frac{1}{n}\right)$$

对于大 n ，可近似为 $n \ln n + \gamma n$ ，其中 $\gamma \approx 0.577$ ，

10.5.2 用公平硬币随机序列生成

假定你有一枚公平硬币，请设计一个有效的随机算法用来生成整数 $1, 2, \dots, n$ 的一个随机排列。并分析你的算法的时间复杂性。

初始化数组 A ： $A[i] \leftarrow i$ 对 $i = 1$ 到 n

考虑 i 次枚举后才找到 $\sum_{i=1}^{\infty} q^i = \frac{q}{1-q} = 1-p$ 解得 $q = 1 - \frac{1}{2^p}$

在最优情况下，所有的 j 都被接受，最优时间复杂度是 $\sum_{i=2}^n \lceil \log_2 i \rceil \leq C_{\text{opt}}(n)$

有：

$$C_{\text{opt}}(n) \leq \sum_{i=2}^n \log_2 n = O(n \log n)$$

也有：(此处省略分部积分的步骤)

$$C_{\text{opt}}(n) \geq \sum_{i=2}^n \log_2 i = \sum_{i=2}^n \int_{i-1}^i \log_2 x dx \geq \sum_{i=2}^n \int_{i-1}^i \log_2 x dx = \int_1^n \log_2 x dx = O(n \log n)$$

因此由夹逼定理有： $C_{\text{opt}}(n) = O(n \log n)$

下面考虑最坏情况 $C_{\text{wor}}(n)$

对于 i 情况下单次 j 的取值，拒绝概率为：

$$P_{\text{reject}}(i) = \frac{2^{\lceil \log_2 i \rceil} - i}{2^{\lceil \log_2 i \rceil}} = 1 - \frac{i}{2^{\lceil \log_2 i \rceil}} \triangleq p_i$$

考虑 i 情况下 k 次 j 的取值被拒绝的概率是 p_i^k ，每次被拒绝都会多进行 $\lceil \log_2 i \rceil$ 次硬币投掷实验，因此 k 次取值后产生的次数为： $p_i^k \cdot k \lceil \log_2 i \rceil$ ，因此 i 情况下的总投掷次数为：(跳过等差乘等比级数的数学推导)

$$w_i = \sum_{k=1}^{\infty} p_i^k \cdot k \lceil \log_2 i \rceil = \frac{p_i(1+p_i)}{(1-p_i)^2} \lceil \log_2 i \rceil = \frac{(2^{\lceil \log_2 i \rceil} - i)(2^{\lceil \log_2 i \rceil + 1} - i)}{i^2} \leq \frac{i \cdot 3i}{i^2} = 3$$

因为拒绝而多出的总次数为：

$$C_{\text{penalty}}(n) = \sum_{i=2}^n w_i \leq 3n = O(n)$$

所以最坏时间复杂度为： $C_{\text{wor}}(n) = C_{\text{penalty}}(n) + C_{\text{opt}}(n) = O(n \log n)$ 综上，该算法的时间复杂度是 $O(n \log n)$ (当然可以改成如果 $j > i$, 就 $j \% i$, 这样也就省了这步计算了)

10.5.3 矩阵互逆

给定两个 $n \times n$ 方阵 A 和 B , 判定它们是否互逆 (即 $AB = I$)。算法基于蒙特卡罗方法, 通过随机采样验证 AB 矩阵元素的性质:

- 对角线元素应为 1 (允许误差 ϵ)
- 非对角线元素应为 0 (允许误差 ϵ)
- **时间复杂度:** $O(T \cdot n)$
 - 每次采样需要 $O(n)$ 时间计算矩阵元素
 - 共进行 T 次采样
- **空间复杂度:** $O(1)$ (仅需存储临时变量)

10.5.4 三元可满足问题

请设计随机游走算法求解三元可满足问题, 并分析算法的期望运行时间。

随机初始化变量赋值 $\mathbf{x} = (x_1, \dots, x_n)$

- **递推关系** (3 文字子句):

$$h_j = \frac{2}{3}h_{j+1} + \frac{1}{3}h_{j-1} + 1$$

- **特征根:**

$$r_1 = 1, r_2 = -\frac{1}{2} \Rightarrow h_j \approx 3^j$$

- **指数下界:**

$$E[T] \geq 3^n - 1 = \Omega(3^n)$$

10.5.5 拉斯维加斯型与蒙特卡洛型的期望运行时间推导关系

设 A 是一个蒙特卡洛随机算法, 满足:

- 期望运行时间不超过 $T(n)$
- 返回正确解的概率为 $p(n)$

- 任何解的正确性可在 $T'(n)$ 时间内验证

我们构造拉斯维加斯算法 A' 如下：

1. 运行 A （耗时 $\leq T(n)$ ）
2. 验证解的正确性（耗时 $\leq T'(n)$ ）
3. 若正确则返回解, 否则重复验证

$$E[\text{时间}] \leq \frac{T(n) + T'(n)}{p(n)}$$

期望时间公式成立因为：

$$E[\text{总时间}] = \sum_{k=1}^{\infty} [(1 - p(n))^{k-1} p(n) \cdot k(T(n) + T'(n))] = \frac{T(n) + T'(n)}{p(n)}$$

该转换在保证结果正确性的前提下，仅增加了多项式时间开销

您的文档标题

作者姓名

2026 年 06 月 16 日

11 chapter11: 近似算法

11.1 导入

11.1.1 组合优化问题

组合优化问题 Π 不是最小化问题就是最大化问题。它由三个部分组成：

1. 一个实例集合 D_Π ；
2. 对于每个实例 $I \in D_\Pi$ ，存在 I 的一个候选解的有限集合 $S_\Pi(I)$ ；
3. 实例 $I \in D_\Pi$ 的每个解 $\sigma \in S_\Pi(I)$ ，存在一个值 $f_\Pi(\sigma)$ ，称为 σ 的解值。
 - 如果 Π 是最小化问题，那么实例 $I \in D_\Pi$ 的最优解 σ^* 满足：

$$\forall \sigma \in S_\Pi(I), f_\Pi(\sigma^*) \leq f_\Pi(\sigma)$$

- 最大化问题的最优解类似定义：

$$\forall \sigma \in S_\Pi(I), f_\Pi(\sigma^*) \geq f_\Pi(\sigma)$$

- 最优解值记为 $OPT(I) = f_\Pi(\sigma^*)$ 。

11.1.2 近似算法

最优化问题 Π 的一个近似算法 A 是一个（**多项式时间**）算法，给定一个实例 $I \in D_\Pi$ ，算法输出某个解 $\sigma \in S_\Pi(I)$ 。算法 A 得到的解也称为近似解，值 $f_\Pi(\sigma)$ 记为 $A(I)$ 。

数学表达：

- 问题实例： $I \in D_\Pi$
- 可行解集： $S_\Pi(I)$
- 近似解： $\sigma \in S_\Pi(I)$
- 解值表示： $A(I) = f_\Pi(\sigma)$

11.1.3 装箱问题

- **问题定义**：给定一个大小在 0 和 1 之间的物品集合，将这些物品装到容量为 1 的箱子中，要求使用箱子数目最少（最小化问题）。
- **实例集** D_Π ：由所有实例 $I = \{s_1, s_2, \dots, s_n\}$ 组成，其中 $\forall j (1 \leq j \leq n)$ ，有 $s_j \in (0, 1]$ 。

- **解集合** $S_{\Pi}(I)$: 由子集集合 $\sigma = \{B_1, B_2, \dots, B_k\}$ 构成, 满足:
 1. σ 是 I 的不相交划分
 2. $\forall j (1 \leq j \leq k), \sum_{s \in B_j} s \leq 1$
- **解值**: $f_{\Pi}(\sigma) = |\sigma| = k$ (使用的箱子数量)。
- 最优解是符合条件的具有最小子集数的 σ 。
- 令 A 是为每一物品分配一个箱子的 (平凡) 算法。根据定义 (A 的时间复杂度是多项式级别的), A 是一个近似算法。但显然, A 不是一个好的近似算法。

11.1.4 差界

- 对于问题的所有实例 I , 希望近似算法 A 满足:

$$|A(I) - OPT(I)| \leq K \quad (K \text{ 为常数})$$

- **定义**: 若对每个实例 I 都有

$$OPT(I) - K \leq A(I) \leq OPT(I) + K \Leftrightarrow |A(I) - OPT(I)| \leq K$$

则称算法 A 具有**有界误差**或**差界** K

- 这也称为近似算法的**绝对性能保证** (absolute performance guarantee)
- 适用于绝对误差敏感的场景 (如资源分配问题中固定容量的偏差)
- 只有很少的 NP 困难问题已知差界

11.1.5 相对性能界

- **定义**: 设 A 是求解优化问题 Π 的近似算法, I 是 Π 的实例

$$R_A(I) = \max\left(\frac{A(I)}{OPT(I)}, \frac{OPT(I)}{A(I)}\right) \geq 1$$

- 称为解 $A(I)$ 的**近似度** (近似比)
- 提供了算法解质量的统一测度, 也称**相对近似度**
- **绝对近似度**:

$$R_A = \inf\{r \geq 1 \mid R_A(I) \leq r, \forall I \in D_{\Pi}\}$$

表示所有实例上的最大近似度

- **渐进近似度**:

$$R_A^{\infty} = \inf\{r \geq 1 \mid \exists N, R_A(I) \leq r, \forall I \in D_{\Pi}, |I| \geq N\}$$

表示足够大实例上的近似度

11.1.6 近似方案

- **定义**：最优化问题的**近似方案**是算法簇 $\{A_\epsilon\}_{\epsilon>0}$ ，满足

$$R_{A_\epsilon} \leq 1 + \epsilon$$

- 近似方案是近似度收敛于 1 的一系列算法
- 可视为算法 A ，输入实例 I 和误差界 ϵ ，满足

$$R_A(I, \epsilon) \leq 1 + \epsilon$$

- **多项式近似方案(PAS)**：近似方案 $\{A_\epsilon\}_{\epsilon>0}$ 中，每个 A_ϵ 在 $|I|$ 的多项式时间内运行。注意：PAS 的运行时间对 $|I|$ 是多项式，但对 $1/\epsilon$ 可能非多项式
- **完全多项式近似方案(FPAS)**是近似方案 $\{A_\epsilon\}_{\epsilon>0}$ ，每个 A_ϵ 在 $|I|$ 和 $1/\epsilon$ 的多项式时间内运行
- **伪多项式时间算法**：在 L 的多项式时间内运行的算法，其中 L 是输入实例的最大数值。示例：0/1 背包问题的动态规划算法（运行时间 $O(nC)$ ）是伪多项式时间算法。注：若算法在 $\log L$ 的多项式时间内运行，则是严格多项式时间算法
- **FPAS 构造方法**：对存在伪多项式时间算法 A 的 NP 难问题：
 1. 对实例 I 的数值进行标度变换和舍入，得到 I'
 2. 将 A 应用于 I' 得到近似解

类型	时间复杂度	解质量	典型问题
PTAS	$O(n^{f(1/\epsilon)})$	$(1 + \epsilon)$ -近似	欧几里得 TSP
FPTAS	$O((1/\epsilon)^k n^c)$	$(1 + \epsilon)$ -近似	0-1 背包
伪多项式算法	$O(N^k n^c)$	可能为精确解	子集和问题

表 5 近似算法对比

11.1.7 可近似性分类

1. **完全可近似的** (Fully Approximable):
 - 定义：对任意小的 $\epsilon > 0$ ，存在 $(1 + \epsilon)$ -近似算法
 - 典型问题：0/1 背包问题
2. **可近似的** (Approximable):
 - 定义：存在具有常数比的近似算法（如 2-近似）
 - 典型问题：最小顶点覆盖问题、多机调度问题
3. **不可近似的** (Inapproximable):

- 定义：除非 $P=NP$ ，否则不存在具有常数比的近似算法
- 典型问题：货郎问题（TSP）

11.2 可近似算法举例

11.2.1 装箱问题

会有四种启发式方法，最先匹配(FF，分配第一个能装的箱子)，最佳匹配(BF，分配能装的箱子里面剩余空间最小的)，递减最先匹配(FFD)，递减最佳匹配(BFD)。下面我们对 FF 进行近似度分析：

假定：箱子容量为 1 个单位体积，物品大小 $s_i \leq 1$ ($i = 1, 2, \dots, n$)。

定义：

- $FF(I)$ 表示在实例 I 下，使用 FF 算法装入物品时所用的箱子数目
- $OPT(I)$ 表示最优装入时所用的箱子数目

FF 装箱规则说明：

- 根据 FF 装箱规则，**至多有一个非空箱子**所装的物品体积 $< \frac{1}{2}$
- 若存在两个这样的箱子 b_i, b_j ($i < j$)，由于它们装的物品体积均 $< \frac{1}{2}$ ，根据算法规则， b_j 中的物品应被装入 b_i 而非新箱子

给定若干个物品，判断它们是否可由两个箱子装下是 NP-完全的。

Proof. 可由划分问题 (Partition problem) 规约。划分问题是 Karp 的 21 个 NP-完全问题之一，该问题表述为：给定若干个正整数，问可否将其划分成和相等的两个部分；换言之，给定 $c_1, c_2, \dots, c_n \in \mathbb{Z}^+$ ，划分问题问是否存在一个 $S \subseteq \{1, 2, \dots, n\}$ 使得

$$\sum_{i \in S} c_i = \sum_{i \notin S} c_i$$

根据一个划分问题实例，我们可以构造一个装箱问题，令物品的尺寸为：

$$a_i = \frac{2c_i}{\sum_{j=1}^n c_j}, \quad i = 1, 2, \dots, n$$

同时我们假设这些 $a_i \in (0, 1]$ 。由于 $\sum_{i=1}^n a_i = 2$ ，如果这些物品能用两个箱子装下，那么这两个箱子都正好装满，这也同时解答了划分问题。这样我们就把一个已知的 NP-完全问题规约为了两个箱子的装箱问题。而显然一个给定的装箱问题的解可在多项式时间内验证，因此该问题是 NP-完全的。 ■

要证明问题 L 是 NP-完全的, 只需证明: 存在性: 已知的某个 NP-完全问题 (如划分问题) 可规约到 L 无关性: 不要求 L 的所有实例都满足某种特性, 只需特定构造的实例能覆盖原问题的解空间

我们之前衡量近似算法采用的是绝对近似比, 定义为最坏情况下算法的求解值与问题的最优值之间的比率。对任意实例 I , 用 $A(I)$ 表示针对实例 I 运行算法 A 后所用箱子数; $OPT(I)$ 表示装完实例 I 中所有物品所用的最少箱子数。若存在常数 $\alpha \geq 1$, 对任意实例 I 有:

$$A(I) \leq \alpha \cdot OPT(I)$$

则算法 A 的近似比至多是 α 。

除非 $P = NP$, 否则装箱问题不存在多项式时间算法具有小于 $\frac{3}{2}$ 的绝对近似比。

证明 (反证法). 假设存在近似比小于 $\frac{3}{2}$ 的多项式时间近似算法 A , 分类讨论:

- 当 $OPT(I) = 2$ 时:

$$A(I) < \frac{3}{2} \times 2 = 3 \implies A(I) = 2 = OPT(I)$$

- 当 $OPT(I) \geq 3$ 时:

$$A(I) \geq 3$$

此时算法 A 可精确判定"是否能用两个箱子装下"的问题, 而该问题是 NP-完全的。若 $P \neq NP$, 则导致矛盾。 ■

虽然 $OPT(I) = 2$ 的情况直接揭示了矛盾的核心 (算法 A 被迫精确求解 NP-完全问题), 但必须验证 $OPT(I) \geq 3$ 时的行为, 才能确保反证假设的全局一致性。这是复杂性理论证明中“规约+反证”方法的严谨性体现。

对任意常数 $\alpha \geq 1$, 任意实例 I , 若存在常数 k 满足:

$$A(I) \leq \alpha \cdot OPT(I) + k \quad (\text{AAR}) \quad (11.2.1)$$

则称所有满足上式的 α 的下确界为算法 A 的**渐进近似比**。其中:

- α 决定当 $OPT(I)$ 充分大时 $A(I)$ 与 $OPT(I)$ 的比值
- k 可以是:
 - 固定常数, 或
 - $o(OPT(I))$ (满足 $\lim_{OPT(I) \rightarrow \infty} \frac{k}{OPT(I)} = 0$)
- 当 $k = 0$ 时, 渐进近似比退化为**绝对近似比**

对于任意的实例 I , 有:

$$FF(I) \leq \frac{17}{10} OPT(I) + \frac{4}{5}$$

(常数 ppt 中为 2, BF 同为 $\frac{17}{10}$)

后续再做展开: <https://zhuanlan.zhihu.com/p/685478438>

对于任意的实例 I , 有:

$$FFD(I) \leq \frac{11}{9}OPT(I) + 4$$

(BFD 同为 $\frac{11}{9}$)

11.2.2 ETSP 问题

给定一个平面上 n 个点的集合 S , 找出一个在这些点上的最短长度的游程 t 。这里一个游程是恰好访问每个点一次的一条回路。

ETSP 问题是 TSP 问题的特殊情况, 点之间的距离是欧式距离, 满足三角不等关系。对于最近邻(NN)法: 采用贪心法, 设 p_1 是任意一个起始点, 下一个访问距离 p_1 最近的点, 比如 p_2 , 再继续访问距离 p_2 最近的点, 直到回到 p_1 。

对于 ESTP 问题所有满足三角不等书的 n 个城市的实例 I , 有:

$$NN(I) \leq \frac{1}{2}(\lceil \log_2 n \rceil + 1)OPT(I).$$

而且, 对于每一个充分大的 n , 存在满足三角不等式的 n 个城市的实例 I 使得:

$$NN(I) > \frac{1}{3}(\log_2(n+1) + \frac{4}{3})OPT(I).$$

很明显这里对于 NN 的近似比: $R_{NN}(I) = \frac{NN(I)}{OPT(I)} = O(\log n)$ 显然 $R_{NN} = \infty$. 所以 NN 算法不是常数近似比的算法, 不能实际应用。

11.2.3 最小顶点覆盖问题

开始时初始化顶点集合 C 为空集, 然后重复执行以下操作——任意选取图中的一条边 (u, v) , 将这两个顶点 u 和 v 加入集合 C 中, 同时从图中删除这两个顶点及其所有关联边; 持续进行这样的操作直到图中所有的边都被删除为止, 最终得到的集合 C 就是所求的顶点覆盖。

算法挑选的边构成图的极大匹配 M (即不存在其他匹配 M' 满足 $M \subset M'$); 由于覆盖 M 中的边至少需要 $|M|$ 个顶点, 因此最小顶点覆盖的大小至少为 $|M|$, 而 MVC 算法实际得到的覆盖大小为 $2|M|$, 由此可推导出近似比 $R_{MVC} \leq 2$ 。MVC 是 2-可近似算法。

设 $G = (V, E)$, 其中顶点集 $V = \{u, v\}$, 边集 $E = \{(u, v)\}$ 。近似顶点覆盖算法 $APPROX-VERTEX-COVER$ 返回的解只能是 $\{u, v\}$,

而最优解为 $\{u\}$ 或 $\{v\}$ ，此时该算法返回顶点覆盖的规模恰为最优覆盖的两倍。

考虑以下启发式算法来解决顶点覆盖问题：重复选择度数最高的顶点，然后删除其所有关联边。举一个例子来说明这种启发式算法的近似比不为 2。（提示：考虑一个二分图，左边顶点集中的点的度数相同，右边顶点集中的点的度数不相同。）

解答：

构造二分图 $G = (V, E)$ ，其中：

$$\begin{aligned} V &= L \cup R \\ L &= \{l_1, l_2, l_3, l_4, l_5\} \\ R &= \{r_1, r_2, \dots, r_{11}\} \end{aligned}$$

边集 E 包含以下边：

$$\begin{aligned} &\{ (l_1, r_1), (l_1, r_3), (l_1, r_4), (l_1, r_6), (l_1, r_7), \\ &\quad (l_2, r_1), (l_2, r_2), (l_2, r_4), (l_2, r_6), (l_2, r_8), \\ &\quad (l_3, r_1), (l_3, r_2), (l_3, r_3), (l_3, r_5), (l_3, r_9), \\ &\quad (l_4, r_1), (l_4, r_2), (l_4, r_3), (l_4, r_5), (l_4, r_{10}), \\ &\quad (l_5, r_1), (l_5, r_2), (l_5, r_3), (l_5, r_4), (l_5, r_{11}) \} \end{aligned}$$

11.2.4 多机调度问题

给定有穷作业集 A 和 m 台相同的机器，作业 a 的处理时间为正整数 $t(a)$ ，每一项作业可以在任一台机器上处理。如何把作业分配给机器才能使完成所有作业的时间最短？即如何把 A 划分成 m 个不相交的子集 A_i 使得 $\max \left\{ \sum_{a \in A_i} t(a) \mid i = 1, 2, \dots, m \right\}$ 最小。

贪心法 G-MPS：按输入的顺序分配作业；把每一项作业分配给当前负载最小的机器；如果当前负载最小的机器有 2 台或 2 台以上，则分配给其中的任意一台（比如标号最小的一台）。

对于多机调度问题的每一个有 m 台机器的实例 I ，贪心算法 G-MPS 满足：

$$\text{G-MPS}(I) \leq \left(2 - \frac{1}{m}\right) \text{OPT}(I)$$

其中 $\text{OPT}(I)$ 表示最优调度长度。

Proof. 首先建立两个基本事实：

1. $\text{OPT}(I) \geq \frac{1}{m} \sum_{a \in A} t(a)$ (m 台机器的最大负载不小于平均负载)
2. $\text{OPT}(I) \geq \max_{a \in A} t(a)$ (最大负载不小于任一作业的处理时间)

设机器 M_j 的负载最大, 即 $G-MPS(I) = t(M_j)$ 。令 b 是最后被分配给 M_j 的作业。根据贪心算法规则, 在分配 b 时 M_j 的负载最小, 因此有:

$$t(M_j) - t(b) \leq \frac{1}{m} \left(\sum_{a \in A} t(a) - t(b) \right)$$

由此可得:

$$\begin{aligned} G-MPS(I) &= t(M_j) \\ &\leq \frac{1}{m} \left(\sum_{a \in A} t(a) - t(b) \right) + t(b) \\ &= \frac{1}{m} \sum_{a \in A} t(a) + \left(1 - \frac{1}{m} \right) t(b) \\ &\leq OPT(I) + \left(1 - \frac{1}{m} \right) OPT(I) \quad (\text{由事实 2}) \\ &= \left(2 - \frac{1}{m} \right) OPT(I) \end{aligned}$$

给定 m 台机器和 $m(m-1)+1$ 项作业, 其中前 $m(m-1)$ 项作业的处理时间都为1, 最后一项作业的处理时间为 m 。算法方案是将前 $m(m-1)$ 项作业平均分配给 m 台机器, 每台分配 $m-1$ 项, 最后一项任意分配给一台机器, 此时 $G-MPS(I) = 2m-1$ 。最优分配方案则是将前 $m(m-1)$ 项作业平均分配给 $m-1$ 台机器, 每台分配 m 项, 最后一项分配给剩下的机器, 此时 $OPT(I) = m$ 。由此可得 $G-MPS$ 是2-近似算法。

递降贪心法 DG-MPS: 首先按处理时间从大到小重新排列作业 (处理时间长的作业优先处理), 然后运用 G-MPS。

时间复杂度分析: DG-MPS 增加排序时间 $O(n \log n)$, 仍然是多项式时间。

对于多机调度问题的每一个有 m 台机器的实例 I , 递降贪心算法 DG-MPS 满足:

$$DG-MPS(I) \leq \left(\frac{3}{2} - \frac{1}{2m} \right) OPT(I)$$

其中 $OPT(I)$ 表示最优调度长度。

Proof. 设作业按处理时间从大到小排列为 $a_1, a_2, \dots, a_n$, 考虑负载最大的机器 M_j 和最后分配给 M_j 的作业 a_i , 存在两种情况:

1. **情况 1:** M_j 只有一个作业

- 此时 $i = 1$, 即 a_i 是处理时间最大的作业
- 显然 $DG-MPS(I) = t(a_1) = OPT(I)$

2. 情况 2: M_j 有至少两个作业

- 则 $i \geq m + 1$ (因为前 m 个作业已分配给不同机器)
- 由最优解性质可得:

$$\text{OPT}(I) \geq t(a_m) + t(a_{m+1}) \geq 2t(a_{m+1}) \geq 2t(a_i)$$

- 对 DG-MPS 的负载进行分析:

$$\begin{aligned} \text{DG-MPS}(I) &= t(M_j) \\ &\leq \frac{1}{m} \left(\sum_{k=1}^n t(a_k) - t(a_i) \right) + t(a_i) \\ &= \frac{1}{m} \sum_{k=1}^n t(a_k) + \left(1 - \frac{1}{m} \right) t(a_i) \\ &\leq \text{OPT}(I) + \left(1 - \frac{1}{m} \right) \cdot \frac{1}{2} \text{OPT}(I) \\ &= \left(\frac{3}{2} - \frac{1}{2m} \right) \text{OPT}(I) \end{aligned}$$



11.3 不可近似问题

对于不要求满足三角不等式的货郎问题, 不存在常数近似比的近似算法, 除非 $P = NP$ 。

Proof. 假设存在这样的近似算法 A , 其近似比 $r \leq K$ (K 为常数)。我们将通过构造性证明这与 NP 完全问题 (哈密顿回路问题) 的关系。

构造方法

对任意给定的图 $G = \langle V, E \rangle$, 构造货郎问题实例 I_G 如下:

- 城市集合为 V , 其中 $|V| = n$
- 距离函数定义为:

$$d(u, v) = \begin{cases} 1, & \text{若 } (u, v) \in E \\ Kn, & \text{否则} \end{cases}$$

关键性质

1. 若 G 有哈密顿回路:

$$\text{OPT}(I_G) = n, \quad A(I_G) \leq r \text{OPT}(I_G) \leq Kn$$

2. 若 G 无哈密顿回路:

$$\text{OPT}(I_G) > Kn, \quad A(I_G) \geq \text{OPT}(I_G) > Kn$$

基于算法 A 可设计如下判定算法:

1. 构造 I_G (耗时 $O(n^2)$)
2. 对 I_G 运行 A (多项式时间)
3. 若 $A(I_G) \leq Kn$ 输出"Yes", 否则输出"No"

该算法可在多项式时间内判定哈密顿回路问题(HC)。由于HC是 NP 完全的, 故 $P = NP$, 与普遍接受的复杂性假设矛盾。 ■

11.4 完全可近似问题

例如对于01背包问题: 如果用贪心算法(G-KK), 不断尝试单位价值最大的物品:

对于0-1背包问题的任何实例 I , 贪心算法 G 和最优解 $\text{OPT}(I)$ 满足:

$$\text{OPT}(I) < 2G - KK(I)$$

其中 $KK(I)$ 表示背包的剩余容量。

Proof. 设物品按单位重量价值 $\frac{v_i}{w_i}$ 从大到小排列为 $u_1, u_2, \dots, u_n$, u_k 是贪心算法 G 中第一件未装入背包的物品。根据贪心策略的特性:

$$\begin{aligned} \text{OPT}(I) &< G - KK(I) + v_k \quad (\text{最优解最多比贪心解多放入 } u_k \text{ 的价值}) \\ &\leq G - KK(I) + v_{\max} \quad (v_k \leq \max_{1 \leq i \leq n} v_i) \\ &\leq 2G - KK(I) \quad (\text{因 } v_{\max} \leq G) \end{aligned}$$

其中关键不等式成立因为:

- 贪心解 G 至少包含一个最大价值物品 (否则可以替换)
- $KK(I)$ 表示贪心解装入后的剩余容量

■
 $\varepsilon \leftarrow \frac{1}{k}$ 将物品按 $\frac{v_i}{w_i}$ 降序排列: $\frac{v_1}{w_1} \geq \dots \geq \frac{v_n}{w_n}$ 输出
 $\max\{A_0(I), \dots, A_k(I)\}$

对任意 $k \geq 1$, PTAS 算法满足:

- 时间复杂度: $O(kn^{k+1})$
- 近似比: $R_k = \frac{\text{OPT}(I)}{A_\varepsilon(I)} < 1 + \frac{1}{k} = 1 + \varepsilon$

Proof. 时间复杂度分析:

- 子集枚举量: $\sum_{j=0}^k \binom{n}{j} = O(kn^k)$

- 每次 G-KK 计算: $O(n)$
- 总时间: $O(kn^{k+1})$

近似比证明: 设最优解 X , 分两种情况:

1. 若 $|X| \leq k$: 算法必得 X (因枚举所有 $\leq k$ 的子集)
2. 若 $|X| > k$:

- 取 $Y \subset X$ 为前 k 个最大价值物品
- 设 $Z = X \setminus Y = \{u_{k+1}, \dots, u_r\}$ (保持 $\frac{v_j}{w_j}$ 降序)
- 设 u_m 为 Z 中第一个未被算法装入的物品
- 定义 W 为算法在 u_m 前装入的非 $\{u_1, \dots, u_m\}$ 物品

关键推导:

$$C' = C - \sum_{j=1}^k w_j - \sum_{j=k+1}^{m-1} w_j$$

$$C'' = C' - \sum_{u_j \in W} w_j < w_m \quad (\text{因 } u_m \text{ 未装入})$$

$$\begin{aligned} OPT(I) &\leq \sum_{j=1}^k v_j + \sum_{j=k+1}^{m-1} v_j + C' \frac{v_m}{w_m} \\ &< \sum_{j=1}^k v_j + \sum_{j=k+1}^{m-1} v_j + \sum_{u_j \in W} v_j + v_m \quad (\text{因 } \frac{v_j}{w_j} \geq \frac{v_m}{w_m}) \\ &< A_\epsilon(I) + \frac{OPT(I)}{k+1} \quad (\text{因 } (k+1)v_m \leq OPT(I)) \\ &\Rightarrow R_k < 1 + \frac{1}{k} = 1 + \epsilon \end{aligned}$$



- **PTAS (Polynomial-Time Approximation Scheme):** 对任意小正数 ϵ , 在输入规模 n 的多项式时间内找到 $(1 + \epsilon)$ -近似解。运行时间关于 n 的多项式, 但可能关于 $1/\epsilon$ 指数增长。
- **FPAS/FPTAS (Fully Polynomial-Time Approximation Scheme):** 运行时间同时是 n 和 $1/\epsilon$ 的多项式, 具有实际可行性。

对于存在伪多项式时间算法的 NP 难问题, 构造 FPTAS 的一般方法:

1. 设原问题实例 I 的输入值为 $x_1, \dots, x_k$
2. 伪多项式算法 A 运行时间为 $T(n, \sum x_i)$
3. 进行标度变换: $x'_i = \lfloor \frac{x_i}{f(\epsilon)} \rfloor$
4. 得到新实例 I'
5. 用算法 A 求解 I' 得解 S'

6. 证明 S' 是 I 的 $(1 + \varepsilon)$ -近似解

$K \leftarrow \frac{C}{2(k+1)n}$ $C' \leftarrow \lfloor C/K \rfloor$ 创建新实例 $I' = (C', \{(s'_i, v'_i)\}_{i=1}^n)$ $S \leftarrow \text{KNAPSACK}(I')$ S

性质:

- 构成算法簇: 对每个 k , $\varepsilon = 1/k$, 记为 A_ε
- 时间复杂度: $\Theta(n^2/\varepsilon)$

对每个整数 $k > 1$, $\varepsilon = 1/k$ 和任意实例 I :

$$\text{OPT}(I) < (1 + \varepsilon)A_\varepsilon(I)$$

时间复杂度为 $\Theta(n^2/\varepsilon)$ 。

Proof. **时间复杂度分析:**

$$\begin{aligned} T(n) &= \Theta(nC'/K) \\ &= \Theta\left(n \cdot \frac{C}{K} \cdot \frac{1}{K}\right) \\ &= \Theta\left(n \cdot \frac{2(k+1)n}{K}\right) \\ &= \Theta(kn^2) = \Theta(n^2/\varepsilon) \end{aligned}$$

近似比证明: 设 I' 为标度变换后的实例:

- $\text{OPT}(I') \geq \sum_{i \in S^*} \lfloor v_i/K \rfloor$ (S^* 是最优解)
- 有不等式链:

$$\text{OPT}(I) - K \cdot \text{OPT}(I') \leq Kn \quad (\text{最优解至多 } n \text{ 项})$$

$$\text{OPT}(I) - A_\varepsilon(I) \leq Kn \quad (\text{近似解定义})$$

$$A_\varepsilon(I) \geq \text{OPT}(I) - Kn$$

$$= \text{OPT}(I) - \frac{C}{2(k+1)}$$

- 假设 $\text{OPT}(I) \geq C/2$ (否则易得最优解), 则:

$$\frac{A_\varepsilon(I)}{\text{OPT}(I)} \geq 1 - \frac{1}{2(k+1) \text{OPT}(I)/C}$$

$$> 1 - \frac{1}{k+1} = \frac{k}{k+1}$$

$$\Rightarrow R_{A_\varepsilon} < \frac{k+1}{k} = 1 + \frac{1}{k} = 1 + \varepsilon$$



11.5 课后作业

11.5.1 独立集问题

给出一个独立集问题的近似算法，即找到最大的顶点子集，子集中顶点互不相连。并请证明或否定该近似算法的近似度是有界的。

使用贪心算法：

1. 初始化独立集 $S = \emptyset$ 。
2. 从图中选择一个度数最小的顶点 v ，将其加入 S 。
3. 将 v 及其所有邻居从图中删除。
4. 重复步骤 2-3，直到图中没有剩余顶点。

每次选择一个顶点加入 S 时，最多会"排除" $\Delta + 1$ 个顶点（包括该顶点及其邻居）。因此，贪心算法得到的独立集大小至少为 $\frac{OPT}{\Delta+1}$ ，其中 OPT 是最优解的大小。

11.5.2 最大子集和问题

最大子集和问题：给定含有 n 个正整数的集合 $S = \{s_1, s_2, \dots, s_n\}$ 和一个正整数 C ，要求找出 S 的一个子集，使得它们的和最大且不超过 C 。请给出求解该问题的完全多项式近似方案。要求分析算法时间复杂度和近似比。

运用动态规划思想：

1. 设定近似参数 $\varepsilon \in (0, 1)$ ，计算缩放因子 $k = \frac{\varepsilon \cdot \max(S)}{n}$ 。
2. 对集合 S 中每个元素 s_i ，计算缩放后的值 $s'_i = \lfloor \frac{s_i}{k} \rfloor$ 。
3. 使用动态规划求解缩放后的问题：
 - 定义 $dp[i][j]$ 表示前 i 个元素能否组成和 j
 - 初始化 $dp[0][0] = \text{True}$
 - 状态转移： $dp[i][j] = dp[i-1][j] \vee dp[i-1][j - s'_i]$
4. 在满足 $\sum_{i \in T} s'_i \leq \lfloor \frac{C}{k} \rfloor \triangleq J_{max}$ 的所有子集 T 中，利用 $dp[n][..]$ 来找到使 $\sum_{i \in T} s_i$ 最大的解。

而对于该算法的时间复杂度与近似比：

• 参数缩放阶段：

- 计算最大值 $\max(S)$ ： $O(n)$
- 计算缩放因子 $k = \frac{\varepsilon \cdot \max(S)}{n}$ ： $O(1)$

▸ 缩放所有元素 $s'_i = \lfloor \frac{s_i}{k} \rfloor$: $O(n)$

• **动态规划阶段:**

▸ 最大缩放和计算:

$$J_{\max} = \left\lfloor \frac{C}{k} \right\rfloor = O\left(\frac{nC}{\varepsilon \max(S)}\right)$$

▸ 状态转移次数:

$$O(n \cdot J_{\max}) = O\left(n \cdot \frac{nC}{\varepsilon \max(S)}\right)$$

• **总时间复杂度:**

$$T(n, \varepsilon) = O\left(\frac{n^2 C}{\varepsilon \max(S)}\right)$$

注意到 $\max(S) \leq C$, 因此:

$$T(n, \varepsilon) \leq O\left(\frac{n^2}{\varepsilon}\right)$$

显然, 这是一个完全多项式近似算法。

• **缩放误差分析:**

$$\begin{aligned} \forall s_i : k \cdot s'_i \leq s_i \leq k(s'_i + 1) \\ \sum_{i \in T} s_i - k \sum_{i \in T} s'_i \leq nk = \varepsilon \max(S) \end{aligned}$$

• **近似比推导:**

$$ALG \geq k \cdot OPT' \geq OPT - nk = OPT - \varepsilon \max(S)$$

$$\begin{aligned} R_{ALG} &= \max\left(\frac{ALG}{OPT}, \frac{OPT}{ALG}\right) \\ &= \frac{OPT}{ALG} \\ &\leq \frac{OPT}{OPT - \varepsilon \max(S)} \\ &= \frac{1}{1 - \frac{\varepsilon \max(S)}{OPT}} \end{aligned}$$

若 $\max(S) \leq OPT$, 则易知 $R_{ALG} \leq 1 + \varepsilon$, 若 $\max(S) > OPT$, 则对该算法进行修正, 将所有满足 $s_i > C$ 的 s_i 均删除, 再次进行计算, 即可转化为上一种情况。

其中 ALG 为该算法得到的解, OPT 为最优解, OPT' 为缩放后问题的最优解

11.5.3 无向图最大割集问题

设无向图 $G = (V, E)$, $V_1 \cup V_2 = V$, $V_1 \cap V_2 = \emptyset$ 。若边 $(u, v) \in E$ 且 $u \in V_1$, 则称 (u, v) 为割边; 否则称为非割边。求最大割集问题: 任给无向图 $G = (V, E)$, 求 G 的边数最多的割集。

求最大割集的局部改进算法 MCUT 思想如下:

初始令 $V_1 = V$, $V_2 = \emptyset$ 。重复以下操作:

1. 若存在顶点 u , 在 u 关联的边中非割边数 $>$ 割边数
 - 若 $u \in V_1$, 则将 u 移至 V_2
 - 若 $u \in V_2$, 则将 u 移至 V_1
2. 直到不存在这样的顶点时停止

最终得到的 (V_1, V_2) 即为解。对任意实例满足:

$$OPT(I) \leq 2 \cdot MCUT(I)$$

其中 $OPT(I)$ 表示最优解的边数。

设 C 为算法得到的割边数 $MCUT(I)$, 设顶点 u 的割边与非割边数分别是 $c(u)$ 和 $nc(u)$, 那么:

每条割边连接 V_1 和 V_2 , 因此 $\sum_{u \in V} c(u) = 2C$ 。因为 $c(u) \geq nc(u)$, 所以对于每个顶点 u , 其度数 $d(u) = c(u) + nc(u) \leq 2 \times c(u)$, 那么 $c(u) \geq \frac{d(u)}{2}$, 所以 $2C = \sum_{u \in V} c(u) \geq \frac{1}{2} \sum_{u \in V} d(u) = |E|$, 所以 $OPT(I) \leq |E| \leq 2C = 2 \cdot MCUT(I)$

11.5.4 平面 TSP 问题的最小权匹配

对 TSP 问题所有满足三角不等关系的实例 I , 已知有最小权匹配 (MM) 近似算法求解, 且 $MM(I) < \frac{3}{2} OPT(I)$, 请给出具有该近似比的紧实例。

这个题肯定需要让顶点数 n 趋于无穷, 使得 $length(T) \rightarrow OPT(I)$, $length(M) \rightarrow \frac{1}{2} OPT$ 即可, 这里我们必须约定我们找到的最小生成树满足某些特殊的性质, 使得 n 个点中度数为奇数的点的数量是接近 n 的, 这样就可以使得 $length(M) \rightarrow \frac{1}{2} OPT$ 。而且依靠这个最小生成树得到的欧拉图, 必须是能够变成哈密尔顿回路的, 即不存在割点。

按照以上原则进行构造即可。

- **PTAS (Polynomial-Time Approximation Scheme):** 对任意小正数 ϵ , 在输入规模 n 的多项式时间内找到 $(1 + \epsilon)$ -近似解。运行时间关于 n 的多项式, 但可能关于 $1/\epsilon$ 指数增长。

- **FPAS/FPTAS (Fully Polynomial-Time Approximation Scheme)**: 运行时间同时是 n 和 $1/\varepsilon$ 的多项式, 具有实际可行性。

对于存在伪多项式时间算法的 NP 难问题, 构造 FPTAS 的一般方法:

1. 设原问题实例 I 的输入值为 $x_1, \dots, x_k$
2. 伪多项式算法 A 运行时间为 $T(n, \sum x_i)$
3. 进行标度变换: $x'_i = \lfloor \frac{x_i}{f(\varepsilon)} \rfloor$
4. 得到新实例 I'
5. 用算法 A 求解 I' 得解 S'
6. 证明 S' 是 I 的 $(1 + \varepsilon)$ -近似解

$K \leftarrow \frac{C}{2(k+1)n}$ $C' \leftarrow \lfloor C/K \rfloor$ 创建新实例 $I' = (C', \{(s'_i, v'_i)\}_{i=1}^n)$ $S \leftarrow \text{KNAPSACK}(I')$ S

性质:

- 构成算法簇: 对每个 k , $\varepsilon = 1/k$, 记为 A_ε
- 时间复杂度: $\Theta(n^2/\varepsilon)$

对每个整数 $k > 1$, $\varepsilon = 1/k$ 和任意实例 I :

$$\text{OPT}(I) < (1 + \varepsilon)A_\varepsilon(I)$$

时间复杂度为 $\Theta(n^2/\varepsilon)$ 。

Proof. **时间复杂度分析:**

$$\begin{aligned} T(n) &= \Theta(nC'/K) \\ &= \Theta\left(n \cdot \frac{C}{K} \cdot \frac{1}{K}\right) \\ &= \Theta\left(n \cdot \frac{2(k+1)n}{K}\right) \\ &= \Theta(kn^2) = \Theta(n^2/\varepsilon) \end{aligned}$$

近似比证明: 设 I' 为标度变换后的实例:

- $\text{OPT}(I') \geq \sum_{i \in S^*} \lfloor v_i/K \rfloor$ (S^* 是最优解)
- 有不等式链:

$$\text{OPT}(I) - K \cdot \text{OPT}(I') \leq Kn \quad (\text{最优解至多 } n \text{ 项})$$

$$\text{OPT}(I) - A_\varepsilon(I) \leq Kn \quad (\text{近似解定义})$$

$$A_\varepsilon(I) \geq \text{OPT}(I) - Kn$$

$$= \text{OPT}(I) - \frac{C}{2(k+1)}$$

- 假设 $\text{OPT}(I) \geq C/2$ (否则易得最优解), 则:

$$\begin{aligned}
\frac{A_\epsilon(I)}{\text{OPT}(I)} &\geq 1 - \frac{1}{2(k+1) \text{OPT}(I)/C} \\
&> 1 - \frac{1}{k+1} = \frac{k}{k+1} \\
\Rightarrow R_{A_\epsilon} &< \frac{k+1}{k} = 1 + \frac{1}{k} = 1 + \epsilon
\end{aligned}$$



11.5.5 最大团问题

给定近似参数 ϵ ，假设存在一个近似比为常数 ρ 的算法 A 用于求解最大团问题。对于图 $G = (V, E)$ ，设其最大团大小为 m ，我们通过以下步骤构造新的近似算法：

计算 $k = \lceil 1/\log_c(1 + \epsilon) \rceil$ 构造图 $G^{(k)}$ 使用算法 A 在 $G^{(k)}$ 中找到大小为 n 的团

将 $G^{(k)}$ 的团映射回 G ，得到团 C

C

$$\begin{aligned}
\frac{m}{n^{1/k}} &\leq c^{1/k} \\
\text{令 } k &= \frac{1}{\log_c(1 + \epsilon)} \\
\Rightarrow \frac{m}{n^{1/k}} &\leq 1 + \epsilon \\
\Rightarrow n^{1/k} &\geq \frac{m}{1 + \epsilon} \\
k &= \frac{1}{\log_c(1 + \epsilon)} = \frac{\ln c}{\ln(1 + \epsilon)} \\
&\geq \frac{\ln c}{\epsilon} \quad (\text{因为 } \ln(1 + \epsilon) \leq \epsilon)
\end{aligned}$$

因此算法 A' 满足：

- 近似比： $1 + \epsilon$
- 时间复杂度：关于 $|V|$ 和 $1/\epsilon$ 的多项式

上述构造证明了最大团问题存在完全多项式时间近似模式(FPTAS)，其中：

$$\text{输出团大小} \geq \frac{m}{1 + \epsilon}$$

且运行时间关于输入规模和 $1/\epsilon$ 均为多项式级别。

11.5.6 证明 TSP 是不可近似问题

反证法. 假设存在常数 $\rho \geq 1$, 使得一般旅行商问题(TSP)存在多项式时间 ρ -近似算法 A 。我们将通过以下步骤导出矛盾:

11.5.6.1 步骤 1: 问题转化

给定哈密顿环问题实例 $G = (V, E)$, 构造对应的 TSP 实例:

- 完全图 $G' = (V, E')$, 其中 $E' = \{(u, v) : u, v \in V \text{ 且 } u \neq v\}$
- 代价函数定义为:

$$c(u, v) = \begin{cases} 1 & \text{若 } (u, v) \in E \\ \rho |V| + 1 & \text{否则} \end{cases}$$

11.5.6.2 步骤 2: 代价分析

- **情况 1:** 若 G 有哈密顿环 H , 则对应 TSP 回路代价:

$$C_{\text{opt}} = \sum_{(u,v) \in H} c(u, v) = |V|$$

- **情况 2:** 若 G 无哈密顿环, 则任何 TSP 回路至少包含一条非 E 中的边, 其最小代价为:

$$C_{\min} \geq (\rho |V| + 1) + (|V| - 1) > \rho |V|$$

11.5.6.3 步骤 3: 算法行为

对构造的 TSP 实例 $\langle G', c \rangle$ 运行算法 A :

- 若 G 有哈密顿环, A 输出回路代价 $C_A \leq \rho C_{\text{opt}} = \rho |V|$
- 若 G 无哈密顿环, A 输出回路代价 $C_A > \rho |V|$

11.5.6.4 步骤 4: 矛盾导出

通过比较 C_A 与 $\rho |V|$ 可判定 G 是否存在哈密顿环。这意味着:

$$\text{哈密顿环问题} \in P$$

根据定理 34.13 (哈密顿环的 NP 完全性) 和定理 34.4 ($P=NP$ 的充分条件), 将导致 $P = NP$, 与前提 $P \neq NP$ 矛盾。 ■

构造完全图 $G' = (V, E')$, 其中 $E' = \{(u, v) : u, v \in V \text{ 且 } u \neq v\}$ 定义代价函数:

$$c(u, v) = \begin{cases} 1 & \text{若 } (u, v) \in E \\ \rho |V| + 1 & \text{否则} \end{cases}$$

运行 ρ -近似算法 A 于 $\langle G', c \rangle$

理论意义

该证明揭示了：

- TSP 的不可近似性直接源于其与 NP 完全问题的等价关系
- 近似算法存在性将导致计算复杂性层级坍塌 ($P = NP$)
- 对任意 $\rho \geq 1$ 的推广通过代价函数的伸缩性实现

11.5.7 瓶颈旅行商问题

瓶颈旅行商问题 (bottleneck traveling-salesman problem) 要求找到一条回路, 满足该回路中代价最大的边的代价是所有回路中代价最大的边的代价中最小的。假设代价函数满足三角形不等式, 证明: 该问题有一个近似比为 3 的多项式时间近似算法。

给定无向完全图 $G = (V, E)$, 根据思考题 21-4 的结论, 可以在关于 $|V|$ 和 $|E|$ 的线性时间内求出一棵瓶颈生成树 T 。因为 G 是完全图, 根据练习题 34.2-11 的结论, 一定能由 T 在关于 $|V|$ 和 $|E|$ 的多项式时间内构造出一个哈密顿环 H , 且 H 上任意两个连续顶点 u 和 v 间在 T 上的距离不超过 3 条边。设 B_T 表示瓶颈生成树中代价最高的边。

近似比证明. 根据三角形不等式 (如图 35.2-4 所示):

$$\begin{aligned} c(v_1^{(1)}, v_2^{(2)}) &\leq c(v_1^{(1)}, v_2^{(1)}) + c(v_2^{(1)}, v_2^{(2)}) \\ &\leq c(v_1^{(1)}, u) + c(u, v_2^{(1)}) + c(v_2^{(1)}, v_2^{(2)}) \\ &\leq 3c(B_T) \end{aligned}$$

因此对任意边 $(u, v) \in H$, 有 $c(u, v) \leq 3c(B_T)$ 。

设 H 中代价最大的边为 B_H , 则:

$$c(B_H) \leq 3c(B_T)$$

设最优瓶颈回路 H^* 中代价最大的边为 B_H^* , 将其移除后得到生成树。根据瓶颈生成树的定义:

$$c(B_T) \leq c(B_H^*)$$

联立(1)(2)式可得:

$$c(B_H) \leq 3c(B_H^*)$$

11.5.8 最优回路不会自我交叉

假设旅行商问题的一个实例的所有顶点是一组平面上的点, 并且代价 $c(u, v)$ 是点 u 和 v 之间的欧氏距离。证明: 一条最优回路不会自我交叉。采用反证法, 假设存在一条最优回路存在一个自我交叉, 如图 35.2-5 所示, 设 (v_1, v_2) 和 (v_3, v_4) 交叉位于 x , 有

$$c(v_1, v_2) + c(v_3, v_4) = c(v_1, x) + c(v_2, x) + c(v_3, x) + c(v_4, x)$$

因为所有顶点都位于欧式平面上且代价 $c(u, v)$ 是点 u 和 v 之间的欧氏距离。所以 c 满足三角形不等式, 有 $c(v_1, x) + c(v_3, x) > c(v_1, v_3)$ 且 $c(v_2, x) + c(v_4, x) > c(v_2, v_4)$ 。代入前面的等式, 有 $c(v_1, v_2) + c(v_3, v_4) > c(v_1, v_3) + c(v_2, v_4)$, 用边 (v_1, v_3) 和 (v_2, v_4) 替换原回路中的 (v_1, v_2) 和 (v_3, v_4) 得到一个代价更小的回路, 与原回路是最优回路相矛盾。因此, 一条最优回路不会自我交叉。证明完毕。

11.5.9 TSP 不可近似性

证明对于任意常数 $c \geq 0$, 一般旅行商问题不存在具有近似比为 $|V|^c$ 的多项式时间的近似算法。

证明: 采用反证法。假设对于某个常数 $c \geq 0$, 一般旅行商问题存在近似比为 $|V|^c$ 的多项式时间算法 A 。我们将说明如何使用 A 来在多项式时间内求解哈密顿环问题的一个实例。根据定理 34.13, 哈密顿环问题是 NP 完全的, 根据定理 34.4, 若哈密顿环问题存在一个多项式时间算法, 则 $P = NP$ 。

设 $G = (V, E)$ 是哈密顿环问题的一个实例, 使用假想的近似算法 A 来检测 G 是否包含一个哈密顿环。按照如下方法将 G 转化为旅行商问题的一个实例, 设 $G' = (V, E')$ 为关于 V 的完全图, 即 $E' = \{(u, v) : u, v \in V \text{ and } u \neq v\}$ 。其代价函数为:

$$c(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E, \\ |V|^{c+1} + 1 & \text{otherwise.} \end{cases}$$

可以在关于 $|V|$ 和 $|E|$ 的多项式时间内构造出 G' 和 c 。

现在考虑旅行商问题的一个实例 $\langle G', c \rangle$ 。若 G 中存在一个哈密顿环 H , 则 c 赋予 H 中每条边的代价为 1, H 的代价为 $|V|$, $\langle G', c \rangle$ 中有一条回路代价为 $|V|$ 。若 G 中不存在一个哈密顿环, 则 G' 中的回路必须包含不属于 E 的边, 该回路的代价至少为 $(|V|^{c+1} + 1) + (|V| - 1) = |V|^{c+1} + |V| = |V|^c (|V| + 1) > |V|^c \cdot |V|$ 。即该回路不是 G 中的一个哈密顿环的因子超过 $|V|^c$ 。

假设我们用近似算法 A 来求解旅行商问题 $\langle G', c \rangle$ 。因为 A 保证能够返回的回路代价不超过最优回路代价的 $|V|^c$ 倍。若 G 中包含一个哈

密顿环，则 A 返回一个满足上述要求的回路。若 G 中不包含一个哈密顿环，则 A 返回一个代价大于 $|V|^{c+1}$ 的回路。因此，可以使用 A 在多项式时间内求解哈密顿环问题。